# Mapping the Spoken Word: A Geo-Spatial Machine Learning Approach to Irish Accent Recognition Final Project Report

**TU856**
**BSc in Computer Science**

**Cianán Ó Finn**

**C22393223**

**Supervisors: Akshay Vellanki, Fadoua Rafii**

School of Computer Science

Technological University, Dublin

**28/03/2026**

# Abstract

This project investigates the feasibility of classifying regional Hiberno-English accents using machine learning and presenting the results through a geospatial interface. While accent classification is well studied at a global scale, limited work has focused on fine-grained variation within Ireland, where accents form a continuum with overlapping features.

A custom dataset of geographically labelled Irish speech was constructed from publicly available sources, including Dáil Éireann and Northern Irish political recordings. The data was pre-processed and used to train and evaluate multiple models across binary and multi-class tasks. Two feature extraction approaches were explored: Mel-Frequency Cepstral Coefficients (MFCCs) and embeddings from a pre-trained wav2vec 2.0 model.

Results showed that wav2vec-based models consistently outperformed MFCC-based approaches, although overall performance remained limited, particularly for multi-class tasks, due to dataset constraints such as size, imbalance, and speaker diversity. The final system integrates selected models into a web-based application, allowing users to submit audio and receive predictions visualised on an interactive map of Ireland. The project demonstrates that regional accent classification is feasible to a degree, but strongly constrained by data, highlighting opportunities for future improvement in dataset expansion and model development.

# Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Cianán Finn

17/04/2026

# Acknowledgements

I began this course four years ago, inspired by my father, who was studying data science at the time. He was incredibly supportive and always encouraged me to do my best and strive for excellence.

Unfortunately, as great men often, he passed away too early, during my second year of university. This had a profound impact on my life, both personally and academically. My results suffered during this time, and I took time away from my studies to be with him.

This report represents my effort to return to my studies and rebuild my academic performance. It was not easy to reach this point again, but I am proud of the work I have done to get here.

Rest in peace, Dad. Bíonn muid ag smaoineamh ort fós, gach uile lá.

# Table of Contents

# 1. Introduction

## 1.1. Project Background

The Hiberno-English accent refers to the variety of English spoken in Ireland, shaped by historical interaction with the Irish language, Gaeilge. The term originates from Hibernia, the Roman name for Ireland, often interpreted as “land of winter” (Etymonline, 2026). The name itself is derived from the Proto-Celtic root Īweriū, which is thought to mean “fertile land” (Matasović, 2009). English varies significantly across regions worldwide, reflecting social, cultural, and geographic influences, from class-based distinctions within the United Kingdom (The Guardian, 2022) to regional dialect variation in the United States (Anglophonia, 2021; Bauer, 2013). Within this global context, Hiberno-English stands out as a distinct dialect, influenced by translation patterns and linguistic structures inherited from Gaeilge.

Within the island of Ireland, distinct sub-dialects have developed, often corresponding to geographic or administrative boundaries, such as city-specific accents or broader provincial variations. Historically, limited mobility between regions contributed to the emergence of localised speech patterns, reinforcing regional linguistic identity. This project aims to explore whether such subtle variation can be systematically detected using machine learning techniques.

Machine learning techniques have been widely applied to speech analysis, including accent classification tasks, where models are trained to identify patterns within audio data (Sabuj et al., 2023). A common approach involves extracting acoustic features such as Mel-Frequency Cepstral Coefficients (MFCCs), which aim to represent speech in a manner aligned with human auditory perception (Davis and Mermelstein, 1980). These features are then used in supervised learning models to classify speech samples. While such approaches have demonstrated strong performance when distinguishing between accents across large geographic regions, their effectiveness decreases as the linguistic and geographic distance between accents becomes smaller, resulting in greater overlap between classes.

In the context of Hiberno-English, this presents a particularly challenging problem. Regional accents exist within a relatively small geographic area and share many overlapping phonological features, forming what has been described as a continuum of variation rather than clearly separable categories (Hickey, 2007). Despite ongoing research in accent classification, relatively limited work has focused specifically on fine-grained regional classification within Ireland. One contributing factor is the lack of publicly available datasets containing geographically labelled Irish speech data, which restricts both training and evaluation (Radford et al., 2022).

Furthermore, while accent classification systems have been developed, few approaches integrate geospatial visualisation of results, limiting both how interpretable it is and practical application. This creates a clear gap for the project: not just whether regional Irish accents can be classified, but whether the result can be shown in a way that is easier to interpret geographically./

## 1.2. Project Description

This project, “Mapping the Spoken Word: A Geo-Spatial Machine Learning Approach to Irish Accent Recognition”, presents a research-driven system that combines experimental machine learning with

a user-facing web application for visualisation. The project investigates the feasibility of classifying regional Hiberno-English accents and representing the results geographically, within the constraints of limited and imperfect speech data.

At its core, the system accepts speech input from users through a web-based interface, either via audio recording or file upload. This audio is processed and analysed using machine learning models trained on geographically labelled Irish speech data. In the final implementation, feature representations are extracted using a pre-trained wav2vec 2.0 model (Baevski et al., 2020), which has been shown to capture high-level acoustic and temporal characteristics of speech more effectively than traditional handcrafted features. These representations are then passed to trained classifiers to predict the most likely region of origin of the speaker.

The system is designed to support multiple classification tasks with varying levels of granularity. These include binary classifications, such as Leinster vs Rest, Ulster vs Rest, and Northern Ireland vs Republic of Ireland, as well as multi-class classifications, such as Four Provinces and Ulster vs Leinster vs Rest. Each task is evaluated across multiple machine learning models, with the best-performing configurations selected for integration into the frontend. This allows users to explore how classification performance varies depending on task complexity and regional grouping.

To enhance interpretability, the system incorporates geospatial visualisation using the Leaflet JavaScript library (Leaflet, 2011). Prediction results are displayed both textually and visually, with the predicted region highlighted as a polygon on a map of Ireland. This provides an intuitive link between the model's output and its geographic meaning, improving user understanding of the results.

In addition, the system maintains a history of previous predictions using browser-based local storage (Mozilla Developer Network, 2024), allowing users to revisit past results and provide feedback on prediction accuracy. This introduces an interactive element to the application and supports informal evaluation of model performance from a user perspective.

The final system combines dataset preparation, feature extraction, classification, and geographic visualisation within a single web-based application. It demonstrates both the technical challenges associated with fine-grained accent classification and the value of geographic visualisation as a tool for interpreting model outputs.

A detailed system flow diagram illustrating the interaction between the frontend, backend, and machine learning components is presented in Section 3.4.

## 1.3. Project Aims and Objectives

The primary aim of this project is to investigate the feasibility of classifying regional Hiberno-English accents using machine learning techniques and to present the results through a geospatial interface.

To achieve this aim, the following objectives were defined:

1. To collect and construct a valid, high-quality dataset of geographically labelled Hiberno-English speech data.
2. To preprocess and standardise the audio data (e.g. segmentation, resampling and cleaning) to ensure consistency of the data for model training.
3. To investigate and evaluate suitable machine learning approaches for regional accent classification, for both traditional feature-based methods and modern deep-learning techniques.

4. To design and implement reusable feature extraction pipelines, including both MFCC-based features and embeddings from a pre-trained wav2vec2 model.
5. To train and evaluate multiple classification models across a range of tasks, including both binary and multi-class classification.
6. To analyse the models' performance using evaluation metrics (e.g. balanced accuracy, F1 macro score), and identify the most effective approaches for fine grained accent classification.
7. To design and develop a web-based application that enables users to upload or record speech samples and receive accent predictions.
8. To integrate the trained models into the application, allowing users to select from multiple models at runtime.
9. To implement geospatial visualisation of predictions using Leaflet, to make the visualised results be more intuitive.

## 1.4. Project Scope

The aim of this project was to research, design and implement a machine learning system capable of regionally classifying Hiberno-English accents and then visualising the results geographically. The primary focus of the project was on the development of backend components, including dataset construction, audio processing, feature extraction, and the evaluation of multiple machine learning models.

The scope of the system includes the development of a processing pipeline that accepts and audio sample, extracts relevant features and applies the trained machine learning models to predict the most likely region of origin of the speaker. The initial work involved working with MFCCs (Mel-Frequency Cepstral Coefficients) for feature extraction; however, due to limited performance on fine-grained tasks, this approach was extended to use embeddings from a pre-trained wav2vec 2.0 model, which improved both accuracy and consistency. The final system performs classification at a regional level, including both binary distinctions and province-level classification, and maps predictions to corresponding geographic regions using polygon-based visualisation.

A significant portion of the project was dedicated to dataset creation and preparation. Due to the lack of publicly available geographically labelled Irish speech data, a custom dataset was constructed from multiple sources, including Dáil Éireann recordings, Oireachtas proceedings, Northern Irish political speech, and the Mozilla Common Voice dataset (Mozilla, 2023). This process required substantial manual effort, including audio sourcing, speaker isolation, quality validation, segmentation, and metadata construction. Despite these efforts, the resulting dataset remains limited in size, balance, and speaker diversity when compared to typical datasets used in speech classification research (Radford et al., 2022), which constrained overall model performance.

The scope of the project was intentionally limited to demonstrating the feasibility of regional accent classification within Ireland, rather than achieving production level accuracy. The system was treated as an experimental platform rather than a production tool. The main goal was to test feature extraction methods, model behaviour, and task design under the data constraints. While a functional web-based interface was developed to support interaction and visualisation, advanced features such as large-scale deployment, user authentication, and persistent backend data storage were considered outside the scope of this project and instead mentioned in Section 7 for future development.

In summary, this project focuses on exploring the challenges of fine-grained accent classification within Ireland and evaluating how geospatial visualisation can support the interpretation of model predictions, within the constraints of limited and imperfect data.

## 1.5. Thesis Roadmap

The rest of this report will be structured as follows:

- Chapter 2 – Literature Review
  This chapter examines existing research, technologies, and systems related to accent classification, speech processing, and geospatial visualisation, identifying key gaps that this project aims to address.

- Chapter 3 – System Analysis
  This chapter outlines the system requirements, stakeholder considerations, and the overall architecture of the proposed solution.

- Chapter 4 – System and Experiment Design
  This chapter describes the design decisions behind both the machine learning pipeline and the system architecture, including the chosen methodology and planning approach.

- Chapter 5 – System and Experiment Development
  This chapter details the implementation of the dataset, experimental pipeline, backend services, and frontend application.

- Chapter 6 – Testing and Evaluation
  This chapter presents the results of system testing and model evaluation, analysing performance across different classification tasks and feature extraction approaches.

- Chapter 7 – Conclusions and Future Work
  This chapter summarises the key findings of the project and outlines potential areas for further development.

# 2. Literature Review

## 2.1. Introduction

This chapter reviews existing research, technologies, and systems relevant to the development of this project, a geo-spatial accent recognition system. The project combines three key areas: web-based technologies, machine learning approaches to accent classification, and the study of regional linguistic variation. While each of these domains has been explored extensively in isolation, there is relatively limited work that integrates them into a single, cohesive system.

The aim of this literature review is to examine existing solutions and academic research across these areas, with a particular focus on their applicability to regional accent classification within Ireland. This includes an evaluation of machine learning techniques used in speech and accent recognition, as well as the technologies used for processing, representing, and visualising geographic data. In addition, the challenges associated with speech datasets—particularly in terms of availability, quality, and geographic labelling—are considered, as these play a critical role in determining system performance.

By bringing these areas together, this chapter aims to identify gaps in both existing research and practical systems, particularly in relation to combining fine-grained accent classification with geospatial visualisation in a single application. This gap is the foundation for the system developed in this project.

## 2.2. Alternative Existing Solutions

A number of tools and systems have already been developed in the area of speech recognition and accent analysis. however, most of the systems focus on general speech-to-text transcription, or large region accent detection, rather than fine-grained regional classification, paired with geographic visualisation.

One such system is Google Cloud's Speech-to-Text (Google Cloud, 2023). This system focuses on converting spoken word into text, supporting multiple different languages. While it includes some level of accent robustness, it still struggles with regional dialects (The Guardian, 2016), rather focusing on its primary goal of accurate transcription rather than identifying speaker's regionality. As a result, it doesn't have functionality to accent classification or geographic interpretation.

Amazon's Transcribe (Amazon Web Services, 2023) provide automatic speech recognition for services with a focus on business scalability and integration into cloud applications. Similarly to Google, it focuses on transcription and does not attempt to classify or visualise accents. These too tools show how industry-level tools put more value into usability and accuracy regarding transcription, rather than linguistic analysis, at least publicly.

Research driven tools, such as systems based on wav2vec 2.0 (Baevski et al., 2020) have demonstrated strong performance in extracting useable representations from raw audio. While these models are only tools, not end-to-end products, they are key elements of many modern speech processing systems. Their ability to capture high-level speech features makes them highly relevant for accent classification tasks, even though they do not provide built-in geographic visualisation.

Canúint (RTÉ Archives, 2025) is a project that was developed by RTÉ in collaboration with Dublin City University to document Gaeilge speakers across the island of Ireland. The system presents recordings on an interactive map of Ireland, where users can select markers on the map, which represent speakers, and listen to archived samples. Although the project focuses on Gaeilge rather than the Hiberno-English accent, it demonstrates an effective approach to visualising regional speech data through a simple and intuitive interface. Its use of map visualisation provided the inspiration of the visual design and user interaction patterns implemented in this project.

Vocal Image (Vocal Image 2023) is an accent training application designed to help users improve their pronunciation and communication skills. Its accent test feature allows users to submit speech and receive an estimated country of origin, which is displayed visually on a world map. This makes it one of the closest existing systems to the concept explored in this project. However, its predictions are limited to broad national categories and do not attempt to classify accents at a regional level. As such, while it provides useful inspiration for front-end interaction and visual feedback, it does not address fine-grained regional accent classification which was the aim of this system.

The alternative existing solutions reviewed show a clear divide between technical capability and practical application. Industry tools such as Google Speech-to-Text and Amazon Transcribe prioritise transcription accuracy and scalability, while research tools such as wav2vec focus on feature extraction without providing user-facing systems. At the same time, interactive platforms such as Canúint and Vocal Image demonstrate effective approaches to visualising speech data, but lack the ability to perform fine-grained regional accent classification. None of the systems examined combine speech input, machine learning-based regional accent prediction, and geo-spatial visualisation within a single platform. This gap forms the basis for the system developed in this project.

## 2.3. Technologies Researched

Several technologies were evaluated during development, covering frontend implementation, backend APIs, audio preprocessing, machine learning, and map visualisation.

**Web Technologies**

The frontend of the system was built using the default web technologies, including HTML, CSS, and JavaScript. These technologies were widely supported for developing the interactive web app planned. JavaScript was used for managing the application's state, handling user interactions, and communicating with the backend through HTTP requests.

To support persistent the client-side storage, local storage was used using the browser's Storage API (Mozilla Developer Network, 2024). This allows prediction history and user feedback to be stored locally within the browser, avoiding the need for a backend database while maintaining user privacy.

**Backend Frameworks**

The backend system was implemented using FastAPI (FastAPI, 2018), which is a modern Python web framework designed for high performing APIs. It was selected due to its simplicity and support for asynchronous request handling. It allowed for the RESTful endpoints to be developed and changed easily as changes occurred, making it well suited for handling the audio uploads and returning the prediction results.

Comparing it to the more traditional frameworks such as Django which was originally used in the prototype for the interim report, FastAPI provides a more lightweight approach, which aligns with the experimental and modular nature of the project.

**Audio Processing Libraries**

How the audio data is handled and processed is critical to the system. The librosa library (McFee et al., 2015) was used to load, process, and transform audio signals. It provides functionality for resampling audio, converting between formats, and extracting features such as Mel-Frequency Cepstral Coefficients (MFCCs).

MFCCs, originally proposed by Davis and Mermelstein (1980), are widely used in speech processing as they try to replicate the human auditory perception. Their use in this project formed the basis of the initial feature extraction approach before the project transitioned to more the modern method of wav2vec 2.0 (Baevski et al., 2020).

**Machine Learning Frameworks**

As the system was developed with an experimental ideology in mind, a range of different machine learning techniques were explored including Logistic Regression, Support Vector Machines, Random Forests and k-Nearest Neighbour (see Figure B1.1.1, Appendix B). These models were decided through researching academic sources (Subuj, H.H. et al. , 2023) which confirmed that they work well using MFCCs.

Expanding the research domain, modern representation-based methods were also investigated, leading to wav2vec (Baevski et al., 2020) being used. This deep learning model enabled the extraction of high-level representations directly from the raw audio, with its ability to be able to capture complex acoustic patterns making it very relevant to accent classification tasks, especially while dealing with a dataset with such subtle differences.

These technologies allowed for direct comparison between handcrafted feature extraction methods and learned representations, which was a central focus of the experimental pipeline.

**Geospatial Technologies**

A core element of the system is the visualisation of the predicted labels geographically. The Leaflet JavaScript library (Leaflet, 2011) was used to implement an interactive map in the frontend, enabling the rendering of geographic data and supporting dynamic interaction, which allowed for the highlighting of predicted region polygons.

The data of the boundaries of these polygons was represented using GeoJSON sourced from the Irish Government's public data (Department of Public Expenditure Infrastructure Public Service, 2026). GeoJSON (Internet Engineering Task Force, 2015 ) is a standard format for encoding spatial data structures. QGIS (QGIS, 2002) was used to prepare and refine these datasets for efficient use in the application.

OpenStreetMap (OpenStreetMap, 2004) provided the base map data used in the visualisations. These technologies together enabled the integration of the developed machine learning predictions to be visualised in intuitive geographic visualisations.

## 2.4. Other Relevant Research

Regional accent classification has been widely studied within the field of speech processing, particularly in contexts where accent groups are clearly distinguishable. Much of this research focuses on classification tasks involving accents from different countries or continents, where linguistic variation is more pronounced. In these scenarios, machine learning models have demonstrated strong performance, as clear phonetic differences between accent groups make classification more straightforward (Sabuj et al., 2023).

However, this level of performance is not representative of more complex, fine-grained classification tasks. As the linguistic distance between accent groups decreases, distinctions between them become less clearly defined, making classification significantly more challenging. This is particularly evident in the context of Irish English. Hickey (2007) describes Irish English as a continuum of regional variation, where overlapping phonological features are common across regions. This means that accents cannot always be separated into clearly defined categories, which directly impacts the performance of classification models. This helps to explain the confusion observed between regions such as Leinster, Munster, and Connacht during the model evaluation phase of this project.

In terms of technical approaches, traditional accent classification systems have relied heavily on feature extraction methods such as Mel-Frequency Cepstral Coefficients (MFCCs). Originally introduced by Davis and Mermelstein (1980), MFCCs aim to represent audio signals in a way that reflects human auditory perception. These features have been widely adopted due to their efficiency and ability to capture key spectral characteristics of speech. When combined with classical machine learning models, such as those explored by Sabuj et al. (2023), MFCC-based approaches can perform well, particularly in controlled environments with clearly separable classes. However, they are limited in their ability to capture more complex temporal and contextual aspects of speech, which are often necessary for distinguishing subtle accent differences.

More recent research has shifted towards representation-based approaches using deep learning. One of the most significant developments in this area is wav2vec 2.0 (Baevski et al., 2020), which enables the extraction of high-level representations directly from raw audio. Unlike MFCCs, which rely on manually designed features, wav2vec learns feature representations through self-supervised training on large speech datasets. This allows the model to capture more complex characteristics of speech, including temporal dynamics such as rhythm, intonation, and pronunciation patterns. These properties make wav2vec-based approaches particularly well suited to accent classification tasks, where subtle variations in speech delivery are often more informative than static spectral features alone.

Despite these advancements, the effectiveness of modern machine learning approaches remains strongly dependent on the availability of large, high-quality datasets. Radford et al. (2022) highlight that state-of-the-art speech models are typically trained on extremely large datasets containing thousands of hours of audio, enabling them to generalise effectively across a wide range of speakers and accents. In contrast, smaller and more specialised datasets introduce significant limitations in both model performance and generalisability.

This challenge is particularly evident in research focused on Irish accent classification. Nolan (2024) found that performance was constrained by both dataset size and the subtle nature of regional variation within Ireland. While some models achieved moderate success, the results highlighted the

difficulty of distinguishing between closely related accent groups. These findings align closely with the challenges encountered in this project, where limited data and overlapping accent features reduced classification performance, particularly in multi-class tasks.

## 2.5. Existing Final Year Projects

The review of existing final year projects in the area of machine learning showed a strong emphasis on model development and performance comparison, often using structured datasets. Many projects focus on applying standard classification techniques, such as logistic regression and decision trees, and evaluating their performance using metrics such as accuracy and ROC analysis. For example, Barr (2025) compared classification trees and logistic regression models using the Growing Up in Ireland dataset, while Bolger (2022) explored similar methods for predicting user behaviour. These projects demonstrate a typical machine learning workflow centred on model evaluation, but are generally limited to isolated experimentation and do not extend to full system development or real-world application.

More domain-specific work has been carried out in the area of Irish accent classification, but remains extremely rare. A final year project could not be sourced, so a Master's thesis was used: Nolan (2024) investigated the use of machine learning models, including logistic regression, neural networks, convolutional neural networks, and transformer-based approaches, to classify Belfast and Dublin accents. While strong performance was achieved using traditional and neural network models, the study was limited to a binary classification task and relied on existing datasets. Furthermore, it did not incorporate a user-facing system or any form of geographic visualisation.

In summary, the final year projects tend to focus either on isolated model experimentation or narrowly defined classification tasks. There is limited work that combines machine learning, custom dataset construction, and full system implementation within a single project. In particular, the integration of accent classification with geospatial visualisation and an interactive web-based interface remains largely unexplored. This project addresses this gap by developing an end-to-end system that combines experimental machine learning with user interaction and geographic interpretation.

## 2.6. Conclusions

This chapter has reviewed existing research, technologies, and systems relevant to the development of a geo-spatial accent recognition platform. The literature highlights that while machine learning approaches to speech and accent classification are well established, most work focuses on large-scale or clearly distinguishable accent groups. In contrast, relatively limited research addresses fine-grained regional variation within a single country, particularly in Ireland, where accents form a continuum with significant overlap.

The review of existing systems also revealed a separation between different areas of development. Industry tools prioritise accurate speech-to-text transcription, while research-driven approaches such as wav2vec focus on feature extraction without providing user-facing applications. At the same time, platforms that visualise speech geographically demonstrate effective interaction design but do not incorporate automated classification. Similarly, existing final year projects tend to focus on model comparison or narrowly defined tasks, without integrating machine learning into complete, interactive systems.

Across these areas, a gap was identified: the lack of systems that combine machine learning-based accent classification with geospatial visualisation and user interaction within a single application. This project plans to address this gap by developing an end-to-end system that integrates experimental machine learning, custom dataset construction, and an interactive web-based interface. The following chapter builds on this foundation by outlining the system requirements and overall architecture designed to support these objectives.

# 3. System Analysis

## 3.1 System Overview

The system is a user-facing web application that allows users to submit speech samples and receive a regional accent classification, which is then visualised on a map of Ireland. Users can provide input either by recording audio directly within the browser or by uploading an existing audio file.

Once an audio sample is submitted, it is transmitted to the backend for processing. The backend performs audio analysis and applies the selected machine learning model to generate a prediction representing the most likely region of origin of the speaker. The resulting classification is returned to the frontend and presented to the user both as textual output and as a highlighted geographic region on the map.

In addition to real-time predictions, the system maintains a history of previous interactions. Predictions are stored locally within the browser, allowing users to revisit earlier results, review associated confidence values, and view any feedback provided regarding prediction accuracy. This functionality enhances the interactivity of the system and enables users to reflect on model behaviour across multiple inputs.

## 3.2 Requirements Gathering

The requirements for this system were gathered through research, iterative development and experimentation. Due to the technical nature and nicheness of the project, a user survey didn't seem appropriate. Instead, requirements were informed by analysing existing systems, reviewing relevant academic literature, and identifying practical constraints related to data availability and machine learning performance.

### 3.2.1 Key Stakeholders

The key stakeholders of this system include:

- The General Public – The end users of the system who interact with the web-application submitting and viewing the results.
- Researchers and Linguists – Individuals with an academic interest in accent variation and geographic linguistic visualisation.
- Project Supervisor – Providing guidance, feedback and evaluation during the system development.
- The Author – Responsible for the overall design, implementation and evaluation of the system.

### 3.2.2 Requirement Collection

The requirement collection approaches were:

- **Reviewing Existing Systems**

Existing tools related to accent analysis, speech processing, and linguistic mapping were analysed to establish expected functionality and user interaction patterns. This informed decisions relating to feature extraction, model selection, and overall system design.

- **Review of Academic Research**
  Academic literature on speech processing and accent classification, including both traditional feature-based approaches and modern representation-based methods, was reviewed. This provided insight into effective modelling techniques and common challenges, informing decisions on feature extraction, model selection, and evaluation strategies.

- **Dataset Investigation and Construction**
  A significant portion of the requirements emerged from the challenge of obtaining suitably labelled training data. Publicly available datasets containing geographically labelled Irish speech data are extremely limited, making the construction of a custom dataset a necessity. This involved sourcing audio from Dáil Éireann recordings, Oireachtas proceedings, and Northern Irish political speech, followed by manual speaker isolation, quality validation, segmentation, and metadata construction.
  Additional datasets, such as the Mozilla Common Voice dataset (Mozilla, 2023), were used for comparative experimentation. The limitations encountered during dataset creation, particularly in terms of size, class imbalance, and speaker diversity, directly influenced the system requirements, especially in relation to model selection and evaluation.

- **Web-Mapping Research**
  Knowledge gained from prior study in web-mapping and GIS informed the design of the geospatial visualisation component. This included the use of GeoJSON for representing regional boundaries and the implementation of interactive mapping through technologies such as Leaflet.

- **Iterative and Experiment Driven Design**
  Requirements continuously changed and evolved throughout the project as experimental results were obtained and issues were identified. Early decisions, such as the use of MFCC-based features, were revised following results showing limited performance, leading to the transitioning to wav2vec-based embeddings. This iterative process ensured that system requirements remained aligned with both technical constraints and experimental findings.

### 3.2.3 Initial Requirements

Based on the above processes, the following requirements were identified:

1. The system must accept user-recorded or uploaded speech samples.
2. The system must process and analyse the audio data for accent classification.
3. The system must support multiple classification tasks with varying levels of regional granularity.
4. The system must provide predictions of speaker's most likely place of origin based on the result of trained machine learning models.
5. The system must present all results and findings in a clear and intuitive way.
6. The system must include a geographic visualisation of the results.

7. The system must retain a history of previous predictions, along with all associated data, and allow for users to interact with these results.
8. The system must be modular, allowing for different tasks and models to be integrated and selected dynamically.
9. The system must operate to as high a standard as possible while staying within the constraints of a limited and imperfect dataset.

## 3.3 Requirements Analysis

After identifying the systems requirements, the project was structured into a set of modular components that reflect both the function needs of the system while adhering to the constraints of the data and academic timeline.

The system has been divided into four main components:

### 3.3.1 Frontend Layer:

The frontend layer provides the user interface, through which the user can interact with the system. It allows for the user to record or upload audio samples, to view the prediction results both textually as well as geographically, as well as to be able to interact with previous outputs. The design prioritises simplicity and usability, ensuring accessibility for users with varying levels of technical expertise. This is achieved through a structured interface that clearly separates input controls, prediction outputs, and visualisation components.

### 3.3.2 Backend Layer:

The backend acts as the centre of the system, handling all API requests, processing the incoming audio data and coordinating all communication between system components. It exposes RESTful endpoints for model retrieval and prediction execution. The backend was implemented using FastAPI, a lightweight Python framework well-suited for building high-performance APIs. Its support for asynchronous processing and rapid development aligns with the requirements of handling audio input and returning prediction results efficiently.

### 3.3.3 Machine Learning Module:

The machine learning module is integrated within the backend in code, but is logically separated as an independent module. As a module it processes the audio data, extracts features from the audio, and performs the classification.

Throughout the project, two machine learning approaches were explored: a traditional MFCC-based feature extraction approach, as well as a representation-based method using embeddings from a pre-trained wav2vec 2.0 model. The final system uses the wav2vec approach as it was proven to consistently perform better during the experiments performed.

A key architectural feature is the use of a plugin-based model system. This allows multiple trained models, each corresponding to different classification tasks or feature extraction approaches, to be dynamically loaded and selected at runtime. This design supports experimentation, simplifies model comparison, and enables future extension without requiring changes to core system logic.

### 3.3.4 Data Storage Layer

A lightweight data storage approach was adopted in place of a traditional database. Training data is stored as audio files with associated metadata, while trained models are stored as artefact files. Prediction history and user feedback are stored locally within the browser using the Web Storage API (Mozilla Developer Network, 2024). This approach reduces system complexity and aligns with the prototype-driven and experimental nature of the project. It also avoids the need for backend data persistence, which was considered to be outside the scope of the system.

## 3.4 Logical Architecture of System

The logical architecture of the system describes how data flows through the application, from initial user input to final output. The system follows a sequential processing pipeline, ensuring a clear and consistent flow of data between components.

The process begins at the frontend, where the user either records an audio sample or uploads an existing audio file. This input is transmitted to the backend via a REST API POST request.

Upon receiving the audio input, the backend validates and processes the data before routing the request to the appropriate model plugin based on the selected model identifier. This routing mechanism enables dynamic selection of different classification tasks and models at runtime.

Within the selected model pipeline, the audio is first standardised into a consistent format suitable for feature extraction. Depending on the chosen approach, different feature extraction methods are applied. In the traditional pipeline, Mel-Frequency Cepstral Coefficients (MFCCs) are extracted as numerical representations of the audio signal (Davis and Mermelstein, 1980). In the alternative approach, the audio is passed through a pre-trained wav2vec 2.0 model to generate high-level embeddings that capture both acoustic and temporal characteristics of speech (Baevski et al., 2020). These feature representations are then used as input to the classification model.

Following feature extraction, the model performs inference and produces a prediction, including the predicted label and associated probability distribution. The backend formats this output into a structured response and returns it to the frontend.

The frontend then updates the user interface to display the prediction results, including the predicted region, confidence values, and probability breakdown. The predicted region is also visualised geographically on the map interface. Users are given the option to provide feedback on the prediction, which is stored locally alongside the corresponding result.

The architecture keeps the interface, backend logic, and model code separate, which made it easier to test and replace components during development. The use of a plugin-based model system further enhances flexibility, allowing new models and classification tasks to be integrated without modifying the core system structure.

**System Flow Diagram:**

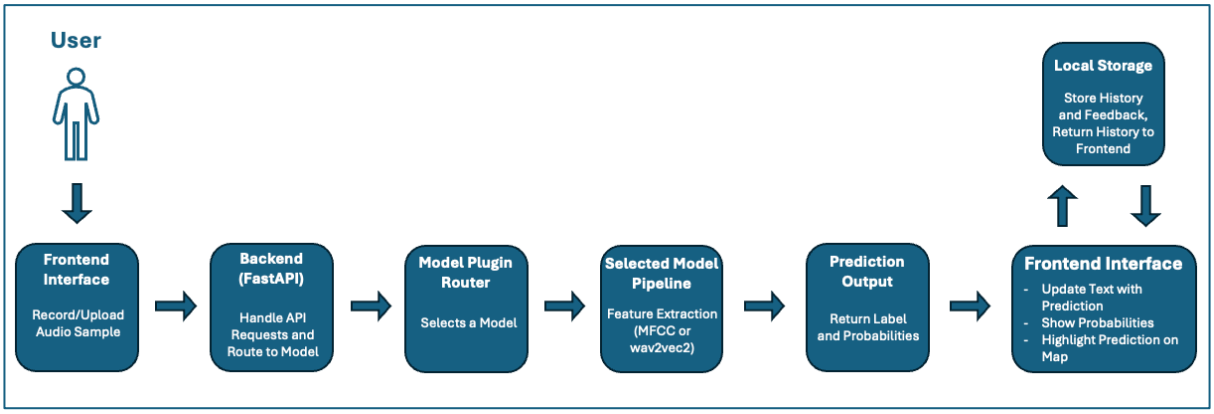


*Figure 1 - System Flow Diagram.*

## 3.5 Initial System Specification

### 3.5.1 Functional Requirements:

1. The system must provide a web-based interface, with consideration to user accessibility.
2. The system must allow users to record an audio sample within the browser, or to upload an audio sample from an existing audio file.
3. The system must send both the audio sample and the selected model identifier to the backend to start processing.
4. The system must process the audio sample and extract features to allow for machine learning classification.
5. The system must support multiple machine learning models and classification tasks, supported through the modular plugin-based architecture of the system.
6. The system must generate a prediction, and return the prediction alongside the associated confidence values and probability distributions to the frontend.
7. The system must map the prediction to a corresponding geographic region.
8. The system must display the results (including the predicted label, the probability distribution and the geographic visualisation) in a clear and understandable manner.
9. The system must retain a history of previous predictions and user feedback locally within the browser using local storage.
10. The system must allow users to revisit and interact with the previous predictions.

### 3.5.2 Non-Functional Requirements:

1. The system should provide predictions within a reasonable response time to improve user experience.
2. The system should provide feedback to the user during interaction (e.g. recording status, processing state).
3. The system should be simple, intuitive and accessible to users with varying levels of technical knowledge.
4. The system should be modular, allowing components to be easily updated or replaced.
5. The system should be robust to different audio formats and quality.
6. The system should operate within the limitations of the small and imperfect dataset.
7. The system should preserve user privacy by storing prediction history and feedback locally rather than on a remote server.

### 3.5.3 Architectural Design:

The system adopts a modular architecture, as outlined in Sections 3.3 and 3.4, separating the frontend, backend, and machine learning components. This separation of concerns ensures that each component can be developed, tested, and modified independently, improving maintainability and flexibility.

A key feature of the architecture is the use of a plugin-based model system within the backend, allowing each of the trained models to be able to be selected during runtime. This design supports the experimental nature of the project, enabling direct comparison between models and facilitating future extensions without requiring changes to the core system.

Data management follows a lightweight approach, avoiding the use of a traditional database. Instead, training data is stored as audio files with associated metadata, while trained models are stored as artefacts. Prediction history and user feedback are stored locally within the browser using local storage. This approach aligns with the prototype-driven nature of the project, reducing system complexity while remaining sufficient for the intended functionality.

## 3.6 Conclusions

This chapter outlined the analysis and design considerations for the proposed system. It consistently referred to the intended user experience while progressing through requirements gathering and system analysis. The system architecture was designed to meet the functional needs of the application while remaining within practical constraints, particularly those related to dataset availability and, consequently, model performance.

A modular, plugin-based design was adopted to support flexibility and experimentation throughout development, while maintaining a clear separation of concerns between system components. The resulting specification provides a structured foundation for the implementation and evaluation of the system presented in the following chapters.

# 4. System / Experiment Design

## 4.1 Introduction

This chapter explains how the system and experimental framework were designed. The design was shaped by the requirements identified earlier, but also evolved as the project progressed and new challenges emerged, particularly in relation to dataset limitations and model performance.

The system was developed to support two main goals: running machine learning experiments on Irish accent classification, and presenting the results through a user-facing web application. As a result, the design needed to balance flexibility for experimentation with a structure that could be integrated into a working system.

A particular focus will be on how the experimental pipeline was structured, how models were incorporated into the system, and how the overall design adapted over time. The methodology used, the planning approach, and the key design decisions that influenced the final implementation are also discussed.

## 4.2. Software Methodology

For this project a traditional Waterfall methodology was initially considered, but as development began it was found to be unsuitable due to its rigid, linear structure. The project required constant experimentation and continuous refinement of both the dataset and the machine learning models, making a more flexible approach necessary.

Instead, an iterative development methodology was adopted, which was noted by J. A. Osorio, et al. (2011) to allow “the development team to deliver value earlier and on a more frequent basis as a consequence of the iterative development approach.”, which fitted the project style. This approach allowed the system to evolve through a series of short development cycles, with each cycle improving on the previous iteration based on experimental results and identified issues.

During the early stages of development, the primary focus was on experimenting with different machine learning models using small Hiberno-English speech datasets. As results were evaluated, the system was progressively refined to align with the evolving objectives of the project.

Each development cycle consists of four short repeatable stages:

1. Planning – Define a small set of achievable goals for the cycle.
2. Implementation – Develop or modify system components in order to achieve these goals.
3. Evaluation – Test all changes made and analyse the results.
4. Revision – Refine any changes and address any identified issues for next cycle.

GitHub was used as the project’s version control system, allowing all changes to be tracked throughout development. This enabled reversion to earlier versions when required and supported the iterative workflow.

## 4.3 Project Planning

Due to the experimental nature of the project, a structured yet flexible planning approach was adopted. This approach allowed for the definition of key milestones while also enabling the project timeline to adapt in response to findings arising from experimentation.

### 4.3.1 Project Schedule

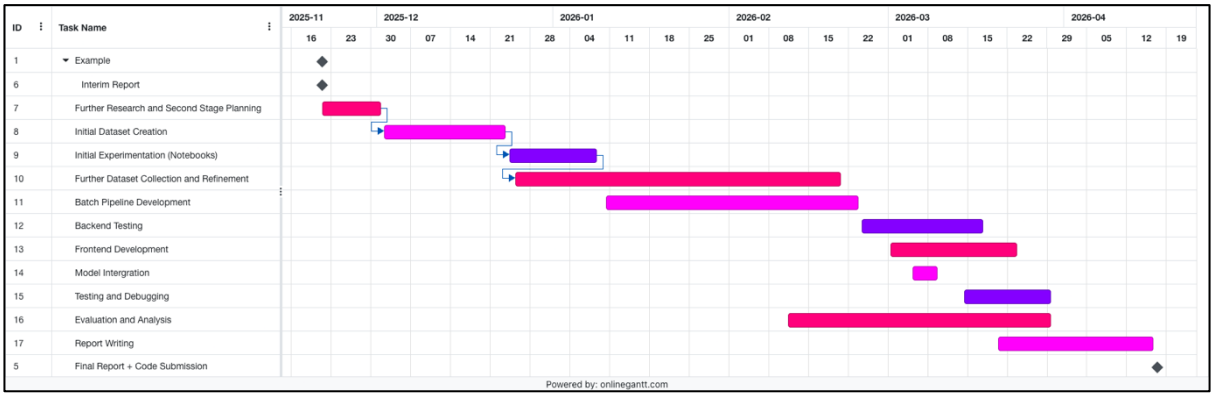


*Figure 2 - Gantt Chart of project timeline post Interim Report.*

In the Gantt chart, multiple stages continue to overlap rather than follow a waterfall, linear progression. This is due to the experimental nature of the project where findings from one stage directly influenced the previous and next stages. For example, the Evaluation and Analysis stage directly influenced the frontend and backend development, leading to them stages having to be revisited multiple times. As a result an iterative development approach was adopted, as mentioned above in Section 4.2. This made it possible to revisit earlier stages when later results showed that assumptions had to change.

## 4.4. Experimental Pipeline and System Design

The system was designed to support both machine learning experimentation and the deployment of a user-facing web application. This led to a design centred around a functional experimental pipeline that processes audio data, evaluates model performance, and integrates the most effective models into the final system.

The pipeline was implemented in a modular manner, allowing experiments to be executed systematically and results to directly inform system development. This structure ensured consistency between the experimentation environment and the deployed application, with components developed during experimentation reused within the backend processing pipeline.

### 4.4.1 Experimental Data Pipeline

The experimental pipeline formed the core of the project and was developed iteratively through continuous refinement. The primary dataset used for Irish accent classification consisted of two custom-built sources: one derived from Dáil Éireann recordings and another constructed from Northern Ireland political speech, both sourced from publicly available media.

Due to the limited availability of geographically labelled speech data, a significant portion of the project focused on dataset creation and preparation. Audio samples were collected, segmented into usable clips, and manually validated to ensure quality and speaker clarity. This involved isolating individual speakers, removing overlapping speech, and excluding any unusable segments.

Metadata construction played a critical role in the pipeline and evolved throughout the project. Initially, metadata included attributes like speaker identity, political affiliation, constituency, and

geographic origin, including town level coordinates. However, due to poor model performance at this level, the classification task was refined to operate at a provincial level. This simplification improved class balance and overall model performance, particularly by reducing the dominance of highly represented regions such as Dublin.

To complement the Irish dataset, a larger dataset based on Mozilla's Common Voice was used for comparative experimentation. This dataset contained approximately 40,000 English-language samples across multiple accent groups, allowing for evaluation on broader classification tasks. Although these models performed worse in practice and were not included in the final deployed system, they provided valuable insight into the challenges of fine-grained accent classification within Ireland.

The final experiment pipeline combined both the Dáil dataset and the Northern Ireland dataset, using speaker aware grouping to prevent data leakage between the training and testing. After all experimentation, the final metadata schema was simplified to include only the essential attributes: file path, speaker identifier, dataset source, and regional label.

The pipeline itself was designed for experimentation, where datasets, tasks, and models were automatically combined and then evaluated. The execution of the experiments was managed through the batch processing script (see D1.1 run_batch.py, Appendix D), which iterates over task and model configurations and runs each experiment in a controlled and repeatable manner.

### 4.4.2 Design Influence from Existing Systems

The design of the geospatial visualisation component was influenced in part by the Canúint project (RTÉ Archives, 2025). Canúint presents archived recordings of Gaeilge speakers as interactive markers across a map of Ireland, allowing users to explore regional speech variation through geographic interaction. While the project focuses on Gaeilge rather than Hiberno-English, it demonstrates how speech data can be effectively linked to geographic representations in an intuitive and accessible manner.

This approach informed early design considerations within the project, where initial concepts explored representing predictions using continuous geographic coordinates, allowing for precise placement of predicted speaker origins. However, due to limitations in dataset size, class imbalance, and the classification-based nature of the models used, this approach was not feasible. As a result, the system was adapted to use region-based classification, with predictions mapped to larger administrative areas such as provinces.

Even though the same level of interaction was never achieved, the influence of map-based interaction remains central to the system design, with geographic visualisation used to improve interpretability and user engagement.

### 4.4.3 Feature Extraction Approaches

During the development of the system, two different feature extraction approaches were explored: a traditional MFCC-based approach and a representations-based approach using a pre-trained wav2vec 2.0.

The MFCC-based approach involved extracting Mel-Frequency Cepstral Coefficients (MFCCs) from each audio sample, converting the raw audio signal into a time series of numerical feature vectors. These features were then summarised using statistical measures, such as the mean and standard deviation, to produce fixed-length feature vectors suitable for input into classical machine learning models. While this approach provided a structured and computationally efficient representation of the audio, it struggled to capture the subtle distinctions between regional accents. This limitation was particularly evident in more fine-grained classification tasks, where performance remained close to baseline levels (see Figures B2.1.1 and B2.1.4, Appendix B).

To handle these issues, a wav2vec embeddings-based approach was implemented. This pipeline took each audio sample and passed it through a pre-trained wav2vec model, which generated high-level feature representations. These embeddings were then aggregated into fixed-length vectors which could be used as input to classification models. This was much more effective as it was able to pick up on more complex patterns within the speech, which resulted in much more accurate and consistently better performance across the tasks.

Implementation details for both feature extraction approaches are provided in Appendix D. The MFCC feature extraction process is illustrated in Figure D2.7, which shows the implementation of the extract_mfcc_summary() function, while the wav2vec-based embedding pipeline is presented in Figure D2.8, demonstrating the conversion of audio input into fixed-length embedding representation.

### 4.4.4 Model Evaluation

The models performance was evaluated using a range of classification metrics, with results being recorded in summary datasets containing: task name, model name, number of samples total, the class distribution and the model type. The primary evaluation metrics were accuracy, balanced accuracy and the macro-average F1 score. The most important evaluation metric was the balanced accuracy due to the imbalance of samples within the dataset. The balanced accuracy ensured that the performance wasn't weighted towards the more heavily represented regions.

All experiment results were aggregated and stored in summary CSV files, which were then used to compare performances across different models and different classification tasks, by representing the results as charts created in Microsoft Excel (Microsoft Corporation, 2018). These charts allowed for clear comparisons between the two feature selection approaches and model selection.

### 4.4.5 Model Selection and System Integration

Using the results of the experiments, the most effective model and feature extraction method for each task were selected and then integrated into the final system. Rather than having a single model, the system was developed so multiple classification tasks could be supported, each with their own best-performing model.

To support this, a modular plugin-based architecture was implemented within the backend. Each trained model is encapsulated as an independent plugin, allowing models to be dynamically loaded and selected at runtime. This design enables flexibility in model deployment, as new models can be added without modifying the core system logic (see plugin_loader.py, Figure D2.2 and registry.py, Figure D2.3, Appendix D).

From a user perspective, the frontend provides a model selection interface, allowing users to choose from the available models (see Figure A1.04, Appendix A) . The selected model identifier is passed to the backend as part of each prediction request (see Figure A2.07, Appendix A) , where it is used to route the input data to the appropriate model pipeline. This ensures that multiple models can operate within a consistent system structure while maintaining separation between model-specific logic and shared processing components.

The overall physical architecture of the system is shown in the Diagram below, showing the interaction between the browser-based frontend, the FastAPI backend, and the machine learning pipeline, along with locally stored artefacts and the external service dependencies.

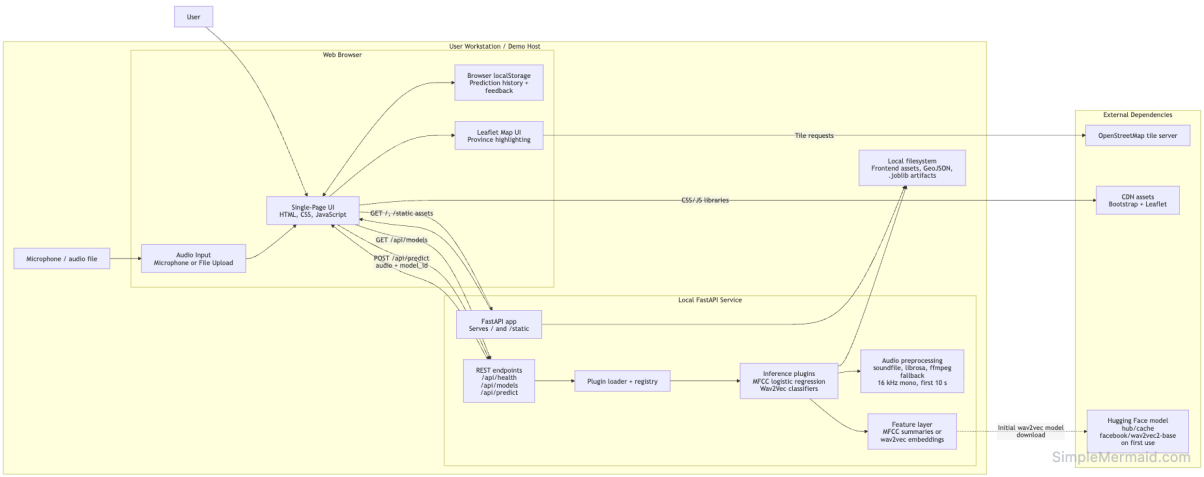


*Figure 3 - Physical architecture of the final deployed system, showing the browser-based frontend.*

## 4.5 Fulfilment of System Requirements

In this chapter the system has been shown to fulfil the functional and non-functional requirements which were identified in Section 3.5. The logical architecture of the system, as described in Section 3.4, serves as the guide for this implementation.

From a functional perspective, the system successfully enables users to submit speech data through the web application, either via file upload or in-browser recording. As outlined in Section 3.4, this input is processed by the backend and passed through the selected model pipeline, fulfilling the requirement for audio processing and classification.

Multiple classification tasks are supported through the backend plugin system, which allows task-specific models to be selected at runtime. This allows different models to be dynamically selected at runtime, which aligns with the experimental nature of the system and supports the deployment of task-specific models.

The requirement to transform raw audio into meaningful predictions is also achieved. Within the pipeline, audio samples are pre-processed, converted into numerical representations using either MFCC or wav2vec-based feature extraction, and then passed through classification models to generate predicted labels and associated probabilities.

In terms of result presentation, the system provides outputs in a clear and interpretable format. The frontend displays both the predicted label and confidence values, and enhances interpretability through geographic visualisation by highlighting the predicted region on a map of Ireland.

The requirement for a data persistence is fulfilled though the use of the browsers local storage. All previous predictions and the user feedback is stored locally and can be retrieved through the interface, allowing users to revisit and view their previous results without a traditional database needed.

Regarding non-functional requirements, the system has been designed to fulfil the previously stated requirements. Its modular structure supports maintainability and allows for future extension to the system. The iterative design methodology chosen allows the system to evolve in response to the datasets limitations, and all of the model performance issues encountered. Also, storing user data locally supports user privacy by avoiding unnecessary storage of user data on a remote server.

## 4.6 Conclusions

This chapter has outlined the design and development of both the experimental framework and the final system implementation. An iterative methodology was decided upon to support the evolving nature of the project, allowing for continuous refinement of the dataset, models, and system components in response to experimental findings.

The experimental pipeline formed the core of the system, which allowed for structured evaluation of multiple datasets, feature extraction methods, and classification models. Through this process, key design decisions were made, including the shift from fine-grained geographic prediction to region-based classification, and the transition from MFCC-based features to more effective wav2vec representations.

These experimental insights directly influenced the system architecture, resulting in a modular, plugin-based design that supports multiple classification tasks within a single application. The integration of machine learning models into a user-facing web application demonstrates how experimental results can be translated into an interactive and interpretable system.

The design stage established the experimental workflow and the final system structure. Most of the major architectural decisions, including region-based classification and wav2vec-based features, came directly from the experimental results.

# 5. System / Experiment Development

## 5.1. Introduction

This chapter will focus on the development of both the experimental pipeline and the system that emerged from the experimental results. Building on the design decisions outlined in Chapter 4, the development process centred on going from the planning of the system architecture into a working implementation, while continuously refining the machine learning pipeline through iterative experimentation.

The project didn't follow a linear progression, instead it evolved through a series of stages. Initially, development began with early experimentation using a small pre-existing Dáil dataset (DanielGall500, 2022) within Jupyter notebooks. These notebooks demonstrated how core components, such as feature extraction and machine learning models, functioned in isolation. As the project progressed, the dataset expanded alongside it, and experimentation continued within notebooks. However, as the number of models and tasks increased, this approach became difficult to manage.

To address this, a more structured experimental pipeline was introduced using Python scripts (see Section D1, Appendix D) . This enabled the automation of multiple experiments within batch processes, which significantly improved scalability, its organisation, and the reproducibility.

The system developed for the interim report represented an early prototype with limited functionality. It supported a single machine learning model, used a Django-based backend, accepted only file uploads, and displayed predictions as text without geographic visualisation. Input validation was minimal, and the system lacked the modularity required to support ongoing experimentation. As development progressed, the system evolved significantly. The backend was redesigned using FastAPI to support a lightweight, API-driven architecture, and a plugin-based model system was introduced to allow multiple models and classification tasks to be dynamically selected.

Audio input functionality was expanded to include in-browser recording, alongside improved validation for input length and format. The frontend was also enhanced to include geospatial visualisation using Leaflet, allowing predictions to be displayed both textually and as highlighted regions on a map. These changes reflect a transition from a basic proof-of-concept to a more complete and interactive system.

Following the results of the experimental phase, the system was extended into a full application, where the best-performing models were integrated into the backend and exposed through a web application. This required the implementation of API endpoints, mechanisms for dynamic model selection, and supporting frontend logic.

This chapter outlines the implementation of the experimental pipeline, the development of the system components, and the key technical decisions made throughout. Particular focus is placed on the iterative improvement of the machine learning approach, including the transition from MFCC-based feature extraction to wav2vec-based embeddings, as well as the development of a modular plugin-based architecture for model integration.

## 5.2. Experimental Pipeline Development

### 5.2.1 Dataset Creation and Collection

Collecting and constructing a suitable dataset for Hiberno-English accent classification was a major component of this project. Unlike many other machine learning tasks (Radford, A., et al, 2022), there are very few publicly available datasets containing geographically labelled Irish speech data. As a result, a significant portion of development time was dedicated to sourcing and creating a custom dataset from publicly available media that would meet the requirements of the project.

The primary dataset used for Hiberno-English accent classification was a combination of two separate custom datasets: one based on Dáil Éireann and Oireachtas recordings, and another made from Northern Irish political speech. Both datasets were created manually using publicly accessible sources.

The project initially began with a small pre-existing Dáil dataset consisting of 195 geographically labelled speech samples (DanielGall500, 2022). This dataset was used for early experimentation and served as a proof-of-concept. While it showed initial promise, it was not expanded directly into the final dataset. Instead, it influenced the decision to use political speech as a reliable source of geographically labelled audio.

The custom Dáil dataset (which includes a smaller number of Oireachtas recordings) ultimately consisted of 1,282 audio samples, each requiring resource intensive manual processing. Recordings were sourced from publicly available political videos, which were reviewed to identify the individual speakers and extract valid speaking intervals. This process involved listening to the recordings, identifying suitable segments, and then manually extracting those segments into clips. A follow up review stage was then conducted, where clips of insufficient quality were excluded.

As the Dáil does not include representation from Northern Ireland, a second dataset was created to address this gap. This Northern Irish dataset consisted of 392 clips and required a more time-consuming collection process. Unlike the Dáil dataset, there is no centralised source of Northern Irish political speech, and the availability of recordings is further limited by political factors such as certain political parties refusing to sit at parliament. As a result, each speaker had to be individually sourced from interviews, speeches, and other media appearances. These recordings were manually reviewed to isolate usable speech segments, which was often more challenging due to variations in audio quality. Extracted clips were then reviewed to ensure clarity and suitability for analysis.

For both datasets, all audio segments were standardised to 10-second clips to ensure consistency for machine learning tasks. The two datasets were then combined to form the primary Hiberno-English speech dataset used throughout the project.

To complement the experiments conducted using the custom dataset, a larger external dataset was sourced from Mozilla's Common Voice English dataset. This dataset was used for comparative experimentation and consisted of approximately 40,000 English-language speech samples across multiple accent groups, including British, North American, Oceanian, and Irish speakers. The Irish subset was later divided into samples from Northern Ireland (approximately 3,500 samples) and the Republic of Ireland (approximately 6,500 samples). While this dataset enabled broader accent classification experiments, it produced weaker results overall (see B3, Appendix B) and was therefore not included in the final deployed system.

The dataset creation was one of the most time-consuming aspects of the project, but it was essential for enabling meaningful experimentation and evaluation. This was particularly important given the limited availability of geographically labelled Irish speech data.

### 5.2.2 Audio Preprocessing and Standardisation

Following the dataset creation, all audio samples collected had to be standardised to ensure audio sample consistency throughout the dataset to make them suitable for machine learning tasks. Due to the variation of the data collected, there was a large spread in audio length and quality. To ensure consistency a preprocessing pipeline was created and implemented to normalise all audio samples before feature extraction and model training.

The first step in the process was to convert all audio in the same format; mono and a frequency of 16 kHz. This is a requirement for both MFCC feature extraction and wav2vec-based embedding approaches. Following this, the samples were segmented into fixed-length clips of 10 seconds. Clips longer than this were trimmed, while shorter clips were excluded from the dataset. This ensured uniform input length across all samples, which is important for consistent feature extraction and model performance.

Then all samples were manually validated, ensuring each sample contained clear speech from a single speaker with minimal background noise, all clips that had any issues were excluded from the dataset. While time intensive this step was essential to maintaining data quality due to the variation of sources used.

After preprocessing, the dataset was consistent in both structure and quality, making it suitable for feature extraction. These constraints helped ensure that variations within the data were primarily attributable to accent characteristics, rather than differences in recording conditions or audio quality.

### 5.2.3 Metadata Construction and Evolution

The development of the metadata was as essential to the project as the collection and curation of the dataset itself. Due to the extremely limited availability of labelled Irish speech data, a custom metadata schema had to be designed, implemented and iteratively refined throughout the project.

The metadata for the Dáil dataset was initially highly detailed. It included attributes such as speaker identity, political party affiliation, constituency, native city, and native county, as well as geographic coordinates for both the speaker's city and province of origin. In addition, several attributes were included to describe each audio segment, such as start and end times (relative to the source video), segment identifiers, clip names, unique IDs, and file paths. This resulted in a highly detailed metadata table with a large number of attributes per sample, which although time-consuming to construct, had a level of detail that was useful during early experimentation, particularly when exploring fine-grained geographic classification (e.g., predicting exact speaker origin). However, as experimentation progressed, this approach proved to be unfeasible. The limited number of samples per class, combined with the subtle nature of accent differences, resulted in poor performance for highly specific classification tasks.

As a result, the project moved to a more coarse classification level, focusing on provincial classification and binary regional classification (e.g. Northern Ireland vs the Republic of Ireland). This shift then simplified the metadata structure, reducing the amount of columns needed for training.

The metadata pipeline was refined multiple times throughout development, including the introduction of cleaned and standardised audio, clip duration tracking and loudness normalisation indicators (FFmpeg Developers, 2024). While many of these attributes were retained for

experimentation and validation purposes, the final training pipeline used a significantly reduced set of features.

The final training dataset consisted of four attributes:

- Filepath – the system's location of the processed audio file.
- Speaker – the speaker identifier, used for grouping and preventing data leakage.
- Label – the target classification label (e.g. province or binary class).
- Dataset – the source of the sample (e.g. Dáil or Northern Ireland dataset).

This simplification improved the efficiency of the training pipeline and reduced any risks of introducing irrelevant or noisy features into the model training. Retaining the speaker identifier was essential as it allowed the use of speaker aware validation techniques, which ensured samples from the same speaker didn't appear in both the training and testing data to prevent overfitting and improving model generalisation to unseen samples.

The evolution of the metadata reflects the experimental nature of the project as a whole, with a constantly transitioning schema from highly detailed to highly minimal. This shows how the system adapted to the constraints found throughout the development of the system.

### 5.2.4 Feature Extraction and Representation

A key stage in the development of the experimental pipeline was transforming the raw audio into a suitable format that could be used for machine learning models. Two different approaches were used: a traditional MFCC-based approach and a more modern representation based approach using a pre-trained wav2vec 2.0 model.

#### MFCC-Based Feature Extraction

The initial approach used Mel-Frequency Cepstral Coefficients (MFCCs), a method commonly used in speech processing (Sabuj, H.H., et al, 2023). MFCCs are designed to approximate how humans perceive sound. As MFCCs are produced in a time series (i.e., a sequence of feature vectors over time), the MFCC features were extracted from sample and then statistically aggregated to produce fixed-length representations suitable for classical machine learning models. This included calculating the summary statistics such as the mean and the standard deviation across each coefficient.

The resulting feature vectors were used as input to traditional classifiers, including Logistic Regression, k-Nearest Neighbours, and Random Forest models. While MFCCs provided structured and interpretable representations of the audio, they struggled to consistently capture the subtle variations associated with regional Hiberno-English accents. This limitation became more evident as classification tasks increased in granularity.

#### Wav2Vec-Based Feature Extraction

Since the results of the MFCC experiments were unsatisfactory, a second approach was implemented using a pre-trained wav2vec 2.0 model (Alexei Baevski et al., 2020). Unlike MFCCs which rely on manually designed features, wav2vec learns high-level representations directly from the raw audio using deep learning. The pipeline designed ensured each sample was converted each

audio sample into 16 kHz and mono format before it was passed to the wav2vec model. The model output a sequence of embedding vectors which represent the audio at a high level of abstraction. These were then aggregated into fixed-length feature vectors by averaging across time. This approach enabled the capture of more complex patterns within the speech signal, including phonetic characteristics and temporal dynamics (such as rhythm and timing), which are difficult to represent using MFCC-based methods. The resulting embeddings were then used as input to the same classifiers as in the MFCC-based approach.

**Comparison: MFCC-Based Approach vs Wav2Vec Based Approach**

The MFCC-based approach relied on manually designed audio features that are computationally inexpensive and lightweight, but seem to struggle with the subtle nature of accent distinctions within Ireland (see Figure B1.2, Appendix B).

In comparison, the Wav2Vec embeddings approach benefitted massively from using a deep learning model that has been pre-trained on large speech datasets, which enabled the capturing of much more complex and abstract speech signal patterns within the data. (see Figure B2.5, Appendix B).

Throughout experimentation, the wav2vec-based approach fared much better, consistently outperforming the MFCC-based approach in every metric tested. As a result the Wav2Vec approach was chosen for the final deployed system, while MFCC-based models were used for comparison during analysis.

### 5.2.5 Model Training and Experimental Setup

The model training process was designed to evaluate both feature extraction approaches across a range of classification tasks and machine learning models. A reusable experimental framework was implemented to ensure reproducibility and allow for fair comparison between different configurations.

**Classification Tasks**

Multiple classification tasks were created to test different levels of regional granularity:

- Ulster vs Rest
- Leinster vs Ulster
- Leinster vs Ulster vs Rest
- Four Provinces (Connacht, Leinster, Munster, Ulster)

**Models Used**

A range of various machine learning models were tested on various tasks on both MFCC and wav2vec feature representations:

- Logistic Regression (Both Weighted and Unweighted)
- Naïve Bayes
- Random Forest
- Support Vector Machines ( Both Linear and RBF Kernel)

- k-Nearest Neighbours (Multiple k Values)

These models were selected as they are suited to structured feature inputs and have been proven to perform well on smaller, restricted datasets (Sabuj, H.H., et al., 2023).

**Training Strategy**

To ensure valid evaluation, a speaker-aware training strategy was implemented. Rather than randomly splitting the dataset, samples were grouped by speaker to ensure that no speaker appeared in both the training and testing sets. This prevented the models from learning speaker-specific characteristics and ensured that evaluation reflected generalisation to unseen speakers.

Model performance was evaluated using cross-validation across multiple splits, with results averaged to provide a stable estimate of performance. The implementation of this training and validation process is detailed in the experimental pipeline functions (see D1, Appendix D).

**Feature and Model Pipeline**

For each experiment:

1. Audio samples were converted into feature representations (MFCC or wav2vec embeddings).
2. Features were passed to the selected model.
3. Predictions were generated for validation data.
4. The performance metrics were recorded.

This standardised pipeline ensured consistency across all experiments and enabled direct comparison between feature extraction methods and models.

**The Evaluation Metrics**

The metrics used for model evaluation were:

- Accuracy
- Balanced Accuracy
- Macro F1 Score

The Balanced Accuracy quickly became apparent as the most important metric due to the imbalance in the dataset, ensuring that the results weren't biased towards the more dominant classes.

Results were then stored in automatically created CSV files containing:

- Task Name (e.g. Leinster vs Rest)
- Model Name
- Number of Samples and Speakers
- Class Distribution amongst the samples
- Performance Metrics

These results were then used to create visual comparisons between the models and tasks.

**Experimental Outcome**
The experimental results demonstrated clear differences between the two feature extraction approaches. The MFCC-based approach struggled with more fine-grained classification tasks, often performing close to baseline levels. In contrast, wav2vec-based models consistently achieved higher performance across all metrics.

Both approaches showed improved performance as classification tasks became more coarse-grained (see Figure B1.1 and B2.1, Appendix B). However, wav2vec-based models maintained a clear advantage throughout. Based on these findings, wav2vec was selected as the feature extraction method for the final deployed system.

### 5.2.6 Batch Experimentation System

The initial experiments were conducted using Jupyter Notebooks, where each change in model or classification task required duplicating the base notebook, editing the code, and then re-running the entire pipeline. The outputs were then manually saved, renamed and then archived alongside the notebook. An experiment write up was also written, showing hypothesises and results' reflections, which then were used to inform later design decisions. However, as the number of experiments increased, this initial method of using notebook based workflows became evidently inefficient and difficult to manage. To fix this, a batch experimentation system was developed so that experiments were automated and consistency was maintained across all tasks, models and feature representation approaches.

**Pipeline Structure**
The experimentation system was built modularly, so that each component was reusable for different experiments:

- Task Definitions (see task_defs.py, D1.3, Appendix D)
  Here the classification tasks were defined (e.g. Ulster vs Rest), including label mapping and dataset filtering.

- Model Definitions (see model_defs.py, D1.4, Appendix D)
  Here the machine learning models and associated configurations were defined (e.g. k-Nearest Neighbour k = 7)

- Pipeline Utilities (see pipeline_utils.py, D1.2, Appendix D)
  This file contained the logic for:
  - Loading and preparing the data
  - Extracting the features
  - Training and evaluating
  - Metric Calculation

- Batch Runner (run_batch.py, D1.1, Appendix D)
  This script purpose was to iterate through all task – model combination, executing the full pipeline automatically for each configuration.

This structure allowed for all experiments to be executed systematically with no manual intervention, which ensured consistency across the runs.

**Separate Experiment Pipelines**

Each feature extraction approach was maintained within its own experiment folder. Although the overall pipeline structure remained the same, separate folders were used for MFCC-based and wav2vec-based experiments to preserve clarity and separation. This made it easier to manage results and allowed direct comparison between approaches while keeping the workflow organised.

**Results Storage and Aggregation**

Each batch produced both individual experiment outputs (e.g. confusion matrices) as well as an overall single summary csv file (see Figure B5.3, Appendix B) which contained aggregated results for all task-model combinations.

The summary csv contained the following columns:

- Task name
- Model name
- Number of samples, speakers and classes.
- Class distribution
- Average accuracy
- Average balanced accuracy
- Average macro F1 score
- Accuracy standard deviation

This summary file was the main source of data for analysis, while the individual outputs were kept for deeper analysis if needed.

**Analysis and Visualisation**

The summary results were converted from a csv file (see Figure B5.3) to a XLSX file and analysed using Microsoft Excel. A range of visualisations were created in order to compare the model performance, which supported data-driven decision making (see Appendix B).

These visualisations included:

- Bar charts comparing the balanced accuracy across models for each tasks.
- Bar charts showing the best performing model per task.
- Bar charts comparing the average performance per task.
- Bar charts of the improvement over baseline performance.
- Scatter plots comparing the accuracy and balanced accuracy (for wav2vec experiments).
- Comparisons of macro F1 scores across all models.

**Role in the Final System**
The batch experiments system was used to evaluate all models and feature combinations. Based on the results, the best-performing models for each classification task were selected and integrated into the final system. This approach ensured that the model selection was based on consistent and repeatable experimentation instead of single experiment results.

## 5.3. Backend Development

### 5.3.1 Fast API

The final backend system was implemented using FastAPI. Unlike the originally planned Django-based backend, the final system required a lightweight, API-focused architecture, where the primary responsibilities were receiving audio input, exposing available models, performing predictions, and returning structured JSON responses.

The backend was designed around a small set of core HTTP endpoints: a health check endpoint to verify system availability, an endpoint to retrieve the list of available models, and a prediction endpoint to process incoming audio and return classification results (see Figure D2.1, Appendix D). In addition to these API functions, the FastAPI application also serves the frontend interface and associated static assets (see Figure D3.2, Appendix D). This enables the system to operate as a single deployed application, while still maintaining a clear separation between frontend and backend responsibilities.

### 5.3.2 Audio Handling and Processing

One of the main responsibilities of the backend is to receive, decode and prepare the audio sample data before it is passed to the machine learning model. This ensures that all input audio is in a consistent format, regardless of how it was recorded or how it was uploaded. The audio is submitted to the system through the exposed prediction endpoint as a file upload. The backend reads the raw audio and performs some validation checks to ensure that the file has been received soundly. If the file is missing or invalid, then an error message is returned to the user.

If valid, the audio is then processed using audio.py, the dedicated audio handling module (see Figure D2.6.2, Appendix D) . This module converts the input into a standardised 16,000 Hz sample and mono format which ensures it is suitable for feature extraction and model inference.

The steps for processing are:

- Convert the audio into mono format.
- Resample to 16 kHz sampling rate.
- Handle different input file types (e.g. WAV, WebM, MP3).

This standardisation ensures that variations in recording format do not negatively impact feature extraction or model inference. By enforcing consistent input conditions, the system reduces the likelihood of performance degradation caused by any inconsistencies in the raw audio.

The audio handling layer also improved the overall robustness of the system as it allowed for multiple different input file formats, and then provided fallback mechanisms where possible (see Figures _guess_audio_suffix() from audio.py, Figure D2.6.1, Appendix D). This was particularly important due to the file uploading aspect of the system, as it gives the user more freedom to

upload any file type. After the audio is processed, it is passed to the selected model's pipeline where the features are extracted and the classification is performed.

### 5.3.3 Model Registration and Selection

A key feature of the backend is the use of a model registration and selection component. It enables multiple different models to be supported within the one system, rather than having just one fixed classifier. To do this the backend maintains a registry of available models, each one is implemented as a separate independent plugin.

At startup, all of the available model plugins are dynamically discovered and loaded using the plugin loader. Each model is instantiated and registered into an in-memory registry structure (see Figures plugin_loader.py, Figure D2.2 and registry.py, Figure D2.3, Appendix D). This registry acts as a central lookup table, mapping a model ID (model_id) to its corresponding model instance and associated prediction pipeline. By storing models in memory, the system avoids repeated loading overhead and allows for efficient access during runtime.

During a prediction request, the request contains the selected model_id alongside the audio file. The backend uses the model_id to retrieve the corresponding model from the registry and route the input data through the correct processing pipeline. This allows for multiple models, multiple feature extraction methods, and multiple classification tasks to be executed, depending on the user's selection.

### 5.3.4 Prediction Endpoint

The core functionality of the backend is implemented through the prediction endpoint, which is exposed as the RESTful API route as ` *POST /api/predict* `. This endpoint receives the submitted audio, processes through the selected model pipeline and the returns the prediction result to the frontend.

Requests to this endpoint are sent as multipart form-data (see Figure A2.08, Appendix A) containing the uploaded audio file and the model_id of the selected model. Upon receiving a request, the backend performs validation checks to ensure that both the audio input and model identifier are valid (see app.py, registry.py and audio.py, Appendix D). If either is invalid, an appropriate HTTP error response is returned. If validation succeeds, the backend retrieves the corresponding model from the registry and executes its prediction pipeline.

The uploaded audio is first pre-processed and then transformed into a suitable feature representation depending on the selected model type. For MFCC-based models, this involves extracting statistical summaries of acoustic features. For wav2vec-based models, the audio is converted into high-level embedding representations using a pre-trained neural network. Following feature extraction, the selected model performs classification and produces a predicted label along with associated confidence probabilities. These results are then formatted into a response and returned to the frontend (see Figure A2.11, Appendix A).

This endpoint acts as the central execution point of the system, linking user input, model inference, and result delivery within a single workflow. It ensures consistency across all supported models while maintaining flexibility through the model selection mechanism.

### 5.3.5 Response Structure and Error Handling

The backend was designed to consistently return structured JSON responses for all prediction requests, ensuring clean integration with the frontend. Each successful response includes the predicted label and the confidence probabilities (see Figure A2.11, Appendix A).

A successful response contains:

- The selected model's ID.
- The predicted label.
- A confidence value representing the model's certainty.
- A list of class probabilities for all possible labels.

This ensures that all results from different models remain consistent and comparable, regardless of what configuration was used.

The backend also includes basic error checking to manage invalid inputs and runtime issues. Validation checks are done to ensure that the incoming requests have the valid required fields. If any check fails, an appropriate HTTP error response is sent.
Common errors are:

- Missing or invalid audio file.
- Invalid or unregister model_id.
- Failures during audio processing or model inference.

By enforcing consistent response structures and clear error handling, the system improves robustness and ensures reliable communication between the backend and frontend components.

## 5.4. Frontend Development

### 5.4.1 User Interface Design

The frontend of the system was designed as a single page web-application. It was designed to have a clear and structured layout, supporting both usability and accessibility. The interface is divided into two main areas: a dashboard section for displaying results and receiving interaction, as well as a map section for the geographic visualisation. On larger screen, they layout follows a two-column structure, where the left side contains the main dashboard (organised as a 2x2 grid of panels) and the right side contains the map (see Figure A1.01, Appendix A). On smaller screen the layout becomes responsive, with all components stacking vertically (see Figure A1.22, Appendix A).

The user interface consists of five main components (see Figure index.html, D3.6, Appendix D):

- The Control Panel
  This provides the primary interaction mechanism for the user. It includes a model selection dropdown, audio recording controls, file upload controls and a button to start the prediction. The model selector is dynamically populated from the backend model register and it allows the users to select which machine learning model they wish to use for their prediction. Changing the selection updates the internal applications state, but a prediction isn't started until the button is clicked, ensuring that the users actions remain controlled.

- The Current Prediction Panel

This displays the results returned from the backend. It included the selected model, the predicted label, a confidence score and a breakdown of the class probabilities. The probability distribution is presented as a list, making it easier for the users to interpret.

- The Prediction History Panel
  This panel provides a record of the previous predictions made during the session. Each record includes metadata such as the predicted label, the confidence score, a timestamp and the model used. The history is designed to be interactive, allowing users to select previous predictions and restore the selected prediction to the map and results section.

- The Feedback Panel
  This panel allows the users to evaluate the prediction accuracy. Users can indicate whether a prediction is correct using “Yes” or “No” options. If the prediction was marked as incorrect, a new input in displayed and the user can select the correct province. This design supports basic evaluation of the models performance from a user perspective and the data is stored locally within the browser.

- The Map Panel
  This panel provides a geographic visualisation of the predictions by using a map of Ireland. When an accent sample is classified, the associated region’s polygon is highlighted on the map. The map also includes controls such as a reset view button allowing users to return the map to the default position.

The interface has been designed to clearly guide the user through the system, from the audio input to the prediction being visualised, and then through inputting feedback. The modular panel based design ensures that each sub-component is clearly separated, while still maintaining a smooth and intuitive user experience.

### 5.4.2 Audio Recording and Upload (MediaRecorder)

The system has been designed to support two methods of audio input: direct recording within the browser and uploading pre-existing audio files. This dual approach allows users to either record audio directly or upload an existing file, making the system more flexible to use (see Figure A1.04 – A1.08 and A1.09 – A1.12, Appendix A).

For the in-browser recording, the MediaRecorder API was used. This API allows the application to access the user’s microphone and capture audio directly without requiring external software. When a recording starts the application requests permission to access the microphone and begins to capture audio in real time. The recording process is controlled through two buttons: one to start and one to stop the recording. The interface provides feedback through status messages and a countdown timer. To ensure consistency with the experimental pipeline, the countdown timer enforces a 10-second clip. If the user stops the recording before 10 seconds, the recording is discarded and prediction remains disabled.

When a recording is successfully completed, the audio becomes available for playback in the interface so that users can review their input before submitting (see Figure A1.07, Appendix A). The recorded audio is encoded in WebM, a format most browsers support, and is stored temporarily in memory until it is sent to the backend.

In addition to recording, users can upload existing audio files through a file input system. The system accepts common formats including WAV, MP3, M4A, OGG, and FLAC (see Figure audio.py _guess_audio_suffix, D2.6.1, Appendix D) . Basic client-side validation ensures that a file has been selected before prediction is enabled. This is supported by deeper validation in the backend, where uploaded files are decoded to confirm they are valid audio and checked to ensure they meet the minimum duration requirement of 10 seconds. If the audio is longer than 10 seconds, it is trimmed to the first 10 seconds (see Figure audio.py load_audio_from_bytes(), D2.6, Appendix D).

This ensures that uploaded audio matches the same constraints as the training data and prevents unsuitable inputs from entering the model pipeline.

Both input methods produce an audio file that is submitted to the backend through a multipart HTTP request. Alongside the audio file, the selected model_id is sent, allowing the backend to route the request to the appropriate model pipeline. This approach keeps the system flexible for users while maintaining consistency with the machine learning pipeline.

### 5.4.3 Displaying the Prediction and Probabilities

After a prediction is processed by the backend, the results are displayed to the user in the 'Current Prediction' panel in the interface (see Figure A1.08, Appendix A). The frontend takes the returned JSON response and formats it into a clear layout. The response includes the selected model, the predicted label, and the associated probabilities.

These elements are presented in a structured format, making the results easy to interpret. The most prominent element is the predicted label, accompanied by a confidence percentage representing the model's highest probability output.

A full breakdown of class probabilities is also displayed, showing how the model distributes likelihood across each region. This is particularly useful in cases where the prediction is less certain, as it highlights overlap between regions. This supports the experimental nature of the project by helping users better understand the strengths and limitations of different models and classification tasks.

The frontend dynamically updates this panel whenever new results are generated or when a previous result is selected from the history panel. When a result is selected, the same display is restored, ensuring consistency between current and past outputs (see Figure A1.15, Appendix A).

It was challenging to ensure that the prediction display, the feedback system, and the map visualisation all remained synchronised throughout the process.

### 5.4.4 Map Visualisation

Geospatial visualisation is a core component of the frontend. To implement this, the system uses the Leaflet library to display an interactive map of Ireland and present model predictions visually. The map is rendered within a dedicated panel in the interface and initialises with a default centre and zoom level focused on Ireland (see Figure map.js D3.4 and Map Div D3.6.3, Appendix D).

Instead of using point markers, the system transitioned to using a polygon-based approach, where each region's boundary is loaded from a GeoJSON file and rendered as map layers (see Figure getRegionsToHighlight() in map.js, D3.4, Appendix D). Each province is initially shown with a simple grey outline (see Figure A1.01, Appendix A) , while the predicted region is highlighted using a blue

polygon fill. This approach was chosen because the final classification tasks operate at broader regional levels, making polygon highlighting clearer than coordinate-based plotting.

After a prediction is returned from the backend, the frontend attempts to match the predicted label to a region in the GeoJSON data. If a match is found, the corresponding polygon is highlighted on the map. A reset view button is also provided, allowing users to return the map to its default centre and zoom without clearing the current prediction, improving overall usability.

A key design decision was to stop automatically zooming to the highlighted region when a prediction is made. Instead, the map remains stable unless the user interacts with it. This provides a smoother and more predictable experience, especially when performing multiple predictions in sequence.

The map component is linked with the prediction history system, allowing previous results to be displayed when selected by the user. This requires the frontend to keep the map, prediction panel, and feedback state synchronised.

### 5.4.5 Prediction History and Feedback System

To improve user experience and support the experimental nature of the system, a prediction history and feedback system was implemented. This allows users to revisit past predictions and optionally provide feedback on the results.

All predictions are stored locally in the browser using the localStorage API (Mozilla Developer Network, 2024). Each entry contains information about a prediction, including a unique ID, a timestamp indicating when the prediction was made, the selected model ID, the predicted label, the confidence value, the full distribution of class probabilities, and any optional user feedback (see store.js Figure D3.5, Appendix D). This data is stored locally to avoid the need for a backend database, reducing system complexity while preserving user privacy.

The prediction history is displayed as a selectable list within the frontend interface. Each entry corresponds to a previously saved prediction. When selected, the system updates the current prediction panel with the stored data, including both the textual output and the highlighted map region (see Figure A1.19, Appendix A).

Users are also given the option to provide feedback on the accuracy of each result (see Figure A1.13, Appendix A). The interface is kept simple, allowing users to indicate whether a prediction is correct or incorrect. If marked incorrect, the user is prompted to select the correct region from a list (see Figure A1.19, Appendix A). The corrected label is then stored alongside the original prediction in local storage. This provides a lightweight way to collect user-evaluated feedback without adding unnecessary complexity to the backend.

The prediction history and the feedback system enhances the usability of the application by allowing users to interact with past results, while also supporting the experiment-driven approach to the project's development by exposing model evaluation to the user.

## 5.5. Model Integration and Plugin Architecture

A key architectural feature of the system is its modular plugin design, which allows multiple trained machine learning models, each using different feature extraction methods and classification tasks, to be integrated without modifying the core application logic.

The system is divided into three main components: a backend with shared services for audio processing(see Figure D2.6, Appendix D) and feature extraction(see Figure D2.7, Appendix D), a model registry (see Figure D2.3, Appendix D) for managing available models, and plugins that encapsulate model-specific prediction logic. This structure ensures consistency across all models while supporting experimentation and future expansion.

### 5.5.1 Plugin Structure

The system has been designed to use a plugin-based approach to represent trained learning models as independent self-contained components. Each plugin has a common plugin interface, and contains model specific prediction logic, while the preprocessing functionality is shared through reusable backend services (as an example see features.py, D2.7, Appendix D).

Plugins are implemented as Python classes, all following a consistent structure (see wav2vec_ulster_vs_rest_rf.py, Figure D2.4, Appendix D). This consistency ensures that models can be used interchangeably within the system. For example, each plugin exposes a standard prediction function, which accepts processed audio input and returns an output containing the predicted label and associated probabilities.

While each plugin encapsulates the inference logic for a specific model, not all stages of the pipeline are handled independently. Core functionality such as audio decoding and standardisation is handled by shared backend services. Where possible, feature extraction steps are also shared, for example MFCC-based models reuse the same MFCC computation pipeline. This avoids duplication and ensures consistency across all models.

Despite differences in internal implementation, all plugins conform to the same external interface. This allows the backend to treat each model identically, regardless of the task, classification type, or whether it uses MFCC-based features or wav2vec embeddings. This design promotes modularity and scalability, allowing new models to be integrated easily by adhering to the same interface, without requiring changes to the frontend.

### 5.5.2 Model Loading and Registration

The backend's dynamic model loading and registration mechanism was designed to support the plugin-based architecture described in Section 5.5.1. It is responsible for discovering available model plugins and making them available for prediction at runtime.

When the application initialises, the loader module scans for available plugins and instantiates them (see load_plugins() Figure D2.2 and app.py, Figure D2.1.1, Appendix D). These instances are then registered in an in-memory registry, which acts as the central lookup structure for all models. Each registered plugin is referenced at runtime using a unique ID (model_id).

The registry (see Figure D2.3, Appendix D) allows the backend to manage a set of available models without hardcoding model-specific logic into the API layer. Once registered, the models are exposed

through the /api/models endpoint, allowing the frontend to dynamically retrieve and display them. Using an in-memory registry also avoids repeatedly reloading models, improving system efficiency.

#### 5.5.3 Runtime Model Selection

During a prediction request, the frontend sends the selected model_id alongside the audio file. Once the request is received, the backend retrieves the corresponding plugin instance from the in-memory registry using the provided model_id.

The audio input is processed using shared backend services, such as decoding and validation. Depending on the model, the appropriate feature extraction is performed using shared modules. After preprocessing, the prepared input is passed to the selected plugin, which performs the classification and returns the prediction.

This mechanism allows the system to support multiple models simultaneously, allowing users to switch between different classification tasks and models at runtime (see A.3, Appendix A). It also ensures that all models operate under the same input constraints and API structure, so that evaluation and comparison are consistent.

## 5.6. Challenges Faced During Development and Iterations

#### 5.6.1 Dataset and Problem Constraints

A major challenge encountered during development was the limited availability of geographically labelled Irish speech data. Unlike other widely studied speech classification tasks (Radford, A., et al, 2022), there are very few publicly available datasets that provide regional labels within Ireland. As a result, a significant portion of the project was dedicated to constructing a custom dataset from publicly available sources such as Dáil recordings and Northern Irish political speech.

This process introduced several constraints that impacted the final system. The dataset required very time-consuming manual effort, including sourcing audio, isolating individual speakers, researching speaker origins, and validating audio quality. In addition, there was no centralised source for Northern Irish speech, which made data collection more time-consuming and resulted in uneven representation across regions.

The limited size and imbalance of the dataset also affected model performance throughout experimentation. Some regions were significantly more represented than others (see B5.2.1, Appendix B), which risked bias during training and evaluation. The limited number of samples per class also made it difficult to perform fine-grained classification tasks, such as predicting constituencies or cities. As a result, the classification tasks were adjusted to operate at a coarser regional level (e.g. province classification), which improved class balance and led to more stable results.

In addition to dataset limitations, the nature of Hiberno-English itself presented further challenges. Regional accents exist within a relatively small geographic area, meaning that neighbouring regions often share overlapping linguistic and phonetic features, as can be seen in the diagram below, Figure 4. This made them more difficult to distinguish using machine learning techniques, reducing separation between classes and negatively impacting accuracy in more detailed classification tasks.

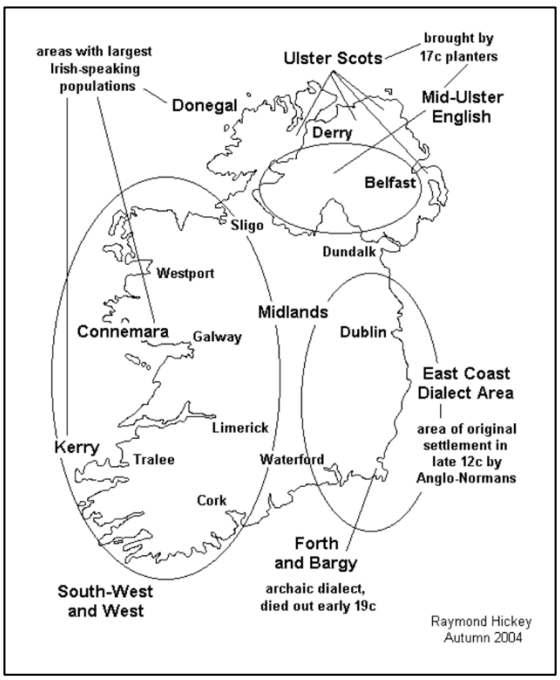


*Figure 4 - Map of Hiberno-English accent grouping, sourced from Raymond Hickey (2004).*

These constraints heavily influenced the final system design, causing the project aim to shift from highly granular prediction to more feasible, coarse-grained classification. This ensured that the system aligned with realistic data limitations while still supporting meaningful experimentation and analysis.

### 5.6.2 Model Development and Feature Extraction Challenges

A second key challenge in system development was identifying an effective feature extraction approach for capturing the subtle differences between regional accents in Ireland. The project initially used a traditional MFCC-based approach, as this is commonly applied in speech processing and is generally well suited to smaller datasets. MFCC features were extracted and then aggregated into fixed-length vectors, which were used as input for machine learning models.

Although this approach provided a structured and computationally efficient representation of the audio, it struggled to capture the accent-defining speech patterns required for regional classification within Ireland. As experimentation progressed, results showed that MFCC-based models only performed slightly better than baseline (see Figure B1.1.5, Appendix B) and degraded as the number of classes increased (see Figure B1.1.2, Appendix B). This indicated that the chosen acoustic features were not sufficient for modelling the complexity of speech patterns in the dataset.

To address these limitations, the project transitioned to a representation-based approach using a pre-trained wav2vec 2.0 model. Unlike MFCCs, which rely on manually designed features, wav2vec learns high-level representations directly from raw audio using deep learning (Baevski, A. et al., 2020). This allowed the system to capture more abstract patterns within the speech signal that are difficult to represent using traditional methods. This led to immediate improvements in performance across all tasks, producing more consistent results (see Figure B1.2.4, Appendix B). This transition was a key development point in the project, shifting the focus from manually designed feature extraction to leveraging pre-trained deep learning representations.

However, this approach also introduced additional challenges. The wav2vec pipeline required additional computational resources and increased system complexity, particularly in terms of dependency management and backend integration. Although this required extra time and effort, the improved performance justified the change, and wav2vec-based models were selected for the final system.

The progression from MFCC-based features to wav2vec embeddings highlights the experimental and iterative nature of the project, demonstrating how initial assumptions were tested and revised based on experimental results, ultimately leading to a more effective solution for regional accent classification in Ireland.

### 5.6.3 System Integration and Frontend Challenges

Integrating the trained machine learning models into a functional end-to-end system introduced a new set of challenges across both the frontend and backend. While individual elements, such as data processing and model training, were developed and tested independently, combining them into a single application required careful coordination between the system's layers.

Within the backend, one of the main difficulties was ensuring consistent integration of multiple models through the plugin-based architecture. Issues arose during development, particularly in model registration and instantiation when dynamically loading plugins at runtime. For example, incorrectly registered plugin classes led to runtime errors during prediction requests. Resolving these issues required refining the loading mechanism to ensure that all plugins were correctly initialised and accessible through the registry.

Dependency management also proved challenging, particularly for wav2vec-based models, which introduced additional external libraries for deep learning and transformer-based components. Ensuring all dependencies were correctly configured and installed was essential for allowing the models to execute reliably.

The frontend introduced its own set of complexities. Maintaining synchronisation between interface components was a significant difficulty, as the system consisted of multiple interconnected elements, including the prediction display, map visualisation, history panel, and feedback system. Ensuring that these components remained consistent when displaying both new and historical predictions required explicit state management within the application.

The map visualisation introduced further complexity. The system needed to match predicted labels to geographic regions defined using a simplified GeoJSON file created in QGIS (QGIS, 2026). It also had to handle cases where the predicted label did not directly correspond to a region. Additionally, several design adjustments were made to improve usability, such as disabling automatic zooming and introducing a reset view button.

Another related challenge, affecting both the map and the frontend more broadly, was ensuring robustness in cases where components failed to load or resources were unavailable. For example, the system was designed to continue functioning even if the map failed to load, preventing a single component from disrupting the entire application.

These challenges highlight the increased complexity involved in transitioning from isolated machine learning experiments to a fully integrated application. Addressing these issues required the iterative, short-cycle approach described in Section 4.2, with continuous refinement to achieve a system that is both robust and provides a smooth user experience.

## 5.7 Conclusions

This chapter has outlined the development of both the experimental pipeline and the final system implementation. Building on the design decisions presented in Chapter 4, the system evolved from early prototype stages into a fully integrated application through an iterative and experiment-driven process.

A significant portion of the development effort was dedicated to constructing and preparing a suitable dataset, which formed the foundation for all subsequent experimentation. The challenges associated with limited and imbalanced data directly influenced both the experimental design and the final system, leading to adjustments in classification tasks and feature extraction approaches. The experimental pipeline played a central role in the development process, enabling systematic evaluation of different models and feature representations. The transition from MFCC-based features to wav2vec embeddings marked a key turning point, resulting in improved performance and shaping the final system architecture. The introduction of a batch experimentation framework further improved consistency, scalability, and reproducibility across experiments.

From a system perspective, the backend was redesigned to support a lightweight, API-driven architecture using FastAPI, while the frontend was extended to provide a more interactive and user-friendly experience. Key features, including in-browser audio recording, prediction visualisation, and a history and feedback system, were implemented to support both usability and experimentation.

The adoption of a modular, plugin-based architecture allowed multiple models and classification tasks to be integrated within a single system, supporting flexibility and future expansion. Despite the technical challenges encountered during integration—particularly in relation to dependency management, model loading, and frontend synchronisation—the final system demonstrates a cohesive pipeline from user input to geographic prediction output.

This  chapter demonstrates how iterative development, structured experimentation, and careful system integration enabled the successful implementation of a complete machine learning application. The resulting system provides a practical platform for exploring regional accent classification within Ireland, while also highlighting the limitations and challenges inherent in working with fine-grained linguistic data.

# 6. Testing and Evaluation

## 6.1. Introduction

This chapter presents an evaluation of the project in terms of both the technical performance of the machine learning models and the overall behaviour of the system.

The evaluation focuses on two main aspects:

1. How well the models perform across different classification tasks, based on different configurations.
2. How the system operates as a complete application, from its audio input to the geographic visualisation of the prediction result.

Evaluation was conducted using controlled experiments carried out during development, with speaker-aware validation applied to ensure that performance reflects generalisation to unseen speakers. The results presented in this chapter are primarily drawn from the batch experimentation described in Section 5.2.6.

## 6.2. System Testing

### 6.2.1 Functional Testing

Functional testing (see Appendix C: System Tests) was carried out to ensure that the system's core functions worked as intended and aligned with the functional requirements defined in Section 3.5.1.

The following functionalities were tested:

- Audio was submitted by file upload (including WAV, MP3, WebM, M4A and FLAC formats) and by in-browser recording using the MediaRecorder API.
- Input into the system was validated, including the handling of non-audio file formats.
- A selection of machine learning models were selected via the frontend interface.
- Prediction requests were sent to the backend.
- The displayed prediction results, including the predicted label, the confidence score and the class probability distribution.
- The predicted regions were visualised on a map of Ireland by highlighting the correct polygon.
- Previous predictions were able to be stored and retrieved from in-browser local storage.
- User feedback was submitted, and the corrected labels were stored alongside the incorrect predictions.

Each component was tested individually before being evaluated as part of the complete end-to-end system. These tests confirmed that audio input could be successfully submitted, classified, and displayed with the correct visual and textual output.

Additionally, robustness testing was conducted to ensure that the system handled error scenarios appropriately. For example, when invalid file types were submitted, the system returned appropriate error messages without failing entirely.

A structured set of system tests was used to validate each major component of the application, as shown in Appendix C.

### 6.2.2 Backend Testing

Backend testing was carried out to validate the behaviour of the system's API endpoints, ensuring correct handling of requests, model execution, and error management across a range of conditions. The primary endpoints tested were */api/models, /api/predict*, and */api/health*.

The */api/models* endpoint was tested to ensure that all available model plugins were correctly loaded and exposed to the frontend. Successful responses returned JSON that listed all available models, including their ID, name, and description (see Figures A2.04 and A2.05, Appendix A). This confirmed that the plugin registration (see Figure D2.3, Appendix D) and in-memory model registry functioned as expected (see app.py, D2.1.1, Appendix D).

The */api/predict* endpoint provides the core functionality of the application. As a POST request, it was tested rigorously under both valid and invalid conditions. For valid requests, audio files were submitted alongside the model_id, and the system returned a JSON response containing the predicted label, confidence score, and class probability distribution (see Figures A2.07 and A2.11, Appendix A). These results showed that, with valid input, the full pipeline—including audio processing, feature extraction, model inference, and response formatting—operated correctly.

Error handling was tested by submitting invalid inputs to the API. This included uploading unsupported file formats and submitting valid audio with an invalid model_id. In both cases, the system returned appropriate HTTP 400 responses with error messages explaining the cause of the error (see Figures A2.08 and A2.09, Appendix A). This demonstrated that input validation and error handling mechanisms were functioning correctly and prevented invalid data from entering the system (see app.py, D2.1.2, Appendix D).

The */api/health* endpoint was tested to confirm that the backend service was running and accessible. Successful responses returned a simple status message indicating that the system was operational (see Figure A2.03, Appendix A) (see app.py, D2.1.1, Appendix D).

Additional backend testing was conducted using Swagger UI, which confirmed that system behaviour remained consistent across different interaction methods (see Figures A2.01 to A2.10, Appendix A).

These tests demonstrate that the backend APIs behave reliably, correctly handling valid requests while appropriately managing invalid inputs, and supporting the end-to-end workflow of the system.

### 6.2.3 Frontend Testing

Frontend testing was carried out to ensure that the user interface functioned correctly, responded appropriately to user interactions, and displayed accurate, up-to-date data reflecting the system's current state throughout a user session.

When the application initialised, it was verified that the loading state was correctly replaced with the available model options. The model selection interface was tested to ensure that models were populated from the backend and that selecting a model updated the application state correctly. It was also confirmed that the selected model was maintained consistently throughout the prediction process and overall application usage (see Figures A1.02 and A1.03, Appendix A).

The audio input interface was tested for both in-browser recording and file uploads. For recording, the frontend enforced a minimum duration of 10 seconds by rejecting shorter recordings. A countdown mechanism was implemented to control recording length, stopping the recording slightly after 10 seconds and trimming it to exactly 10 seconds to ensure consistency with the training data

(see count down functions, recording.js, D3.3.4, Appendix D). This approach was introduced to resolve timing inconsistencies observed during development. Prediction functionality was only enabled once a valid recording was completed.

For file uploads, the interface validated user input and updated to reflect the selected file before enabling predictions (see Figures A1.04 to A1.12, Appendix A). The frontend also enforced minimum duration requirements for uploaded audio, rejecting shorter samples and providing clear error messages to the user (see Figure A1.13, Appendix A).

The prediction panel was tested to ensure that results returned from the backend were displayed clearly and consistently. The interface correctly presented the selected model, predicted label, confidence score, and probability distribution. When new predictions were generated, the display updated dynamically. It was also verified that selecting a previous prediction restored the corresponding data in the interface (see Figures A1.08, A1.12, and A1.19, Appendix A).

The map visualisation component was tested to confirm that predicted regions were correctly highlighted. It was verified that the textual prediction corresponded to the highlighted polygon, and that only the current prediction was displayed while previous highlights were cleared (see Figures A1.08 and A1.15 to A1.18, Appendix A).

The prediction history and feedback system was tested to confirm that predictions were successfully stored in local storage(store.js, D3.5, Appendix D) (see Figure A1.21, Appendix A) and could be retrieved and redisplayed. Feedback functionality was also tested, confirming that user responses were stored alongside the original prediction (see Figures A1.13 to A1.17, Appendix A).

The frontend testing confirmed that all interface components functioned correctly across a range of user scenarios. The system provided clear and consistent feedback, maintained synchronisation between components, and delivered a smooth and reliable user experience.

### 6.2.4 Audio Processing Testing

Audio processing testing was carried out as this stage was identified as a potential failure point within the system pipeline. To validate the audio processing pipeline, tests were conducted to assess the system's ability to correctly handle, standardise, and prepare audio input for model inference. This included evaluating compatibility with a range of input file formats, enforcement of input constraints, and consistency with the training pipeline.

A variety of audio formats were tested, including WAV, MP3, M4A, FLAC, OGG, WebM, AIFF, and AMR (see ST-03, Appendix C). The backend successfully decoded and processed all supported formats except one, converting them into a consistent representation (16 kHz, mono, 10 seconds) suitable for feature extraction. WMA files were not accepted by the frontend but were successfully processed by the backend, highlighting an inconsistency in validation. This issue was resolved by adding WMA to the list of accepted file types (see _guess_audio_suffix() , audio.py, Figure D2.6, Appendix D). Non-audio file formats were also tested, ensuring that such inputs were rejected and appropriate error messages were returned (see Figure A2.08, Appendix A).

The system enforced a minimum audio duration of 10 seconds to maintain consistency with the preprocessing pipeline used during dataset preparation. Audio samples shorter than this threshold were rejected, with validation applied at both the frontend and backend. This ensured that only valid inputs were passed to the pipeline and reduced inconsistencies between training and inference data (see Figures A2.12 and A2.13, Appendix A).

All valid audio inputs were standardised prior to feature extraction. Each sample was converted to mono format and resampled to 16,000 Hz (see audio.py, D2.6, Appendix D), ensuring compatibility with both MFCC and wav2vec pipelines. Testing confirmed that this standardisation process was consistently applied across all inputs.

Some variation in prediction outputs was observed when the same audio sample was submitted in different formats, particularly for highly compressed formats such as AMR. This suggests that encoding and compression can influence extracted features and, consequently, model predictions. This represents a limitation of the system and highlights the sensitivity of speech-based models to input quality and format.

## 6.3. System Evaluation

### 6.3.1 Evaluation Methodology

The system was evaluated using a structured experimental approach designed to assess the performance of different machine learning models across a range of classification tasks (see D1, Appendix D). The primary aim was to determine how effectively regional Hiberno-English accents could be classified using different modelling approaches and feature extraction methods.

All experiments were conducted using the batch experimentation pipeline described in Section 5.2.6, and found in D1, Appendix D. This pipeline ensured that each model and task was evaluated under consistent conditions, allowing for fair comparison and reproducible results.

Overfitting was identified as a significant issue in earlier stages of development. To address this, a speaker-aware cross-validation strategy was implemented. Samples were grouped by speaker, ensuring that no individual appeared in both the training and testing sets. This approach was essential in preventing models from learning speaker-specific characteristics, instead encouraging the learning of generalisable accent features.

Multiple classification tasks were defined to evaluate model performance at different levels of regional granularity (see task_defs.py, D1.3, Appendix D). These included binary classification tasks (e.g. Ulster vs Rest and Dáil dataset vs Northern Ireland dataset) as well as multi-class classification tasks (e.g. Four Provinces) (see Figure B5.1.1, Appendix B).

Each task was evaluated using a range of machine learning models, applied to both MFCC-based features and wav2vec-based embeddings (see model_defs.py, D1.4, Appendix D). Performance metrics were calculated across multiple validation folds and averaged to provide stable estimates of model performance. The results were stored in structured CSV files (see Figure B5.3, Appendix B) and visualised using charts presented in Appendix B.

Additional experiments were conducted using a dataset derived from the Mozilla Common Voice dataset (see Section 5.2.1). These experiments were used primarily for exploratory analysis; however, due to relatively poor performance (see Figures B3.1 and B3.2, Appendix B), they were not included in the final system. As a result, the evaluation presented in this section focuses solely on the Hiberno-English dataset used in the deployed application.

### 6.3.2 Evaluation Metrics

The performance of the classification models was evaluated using accuracy, balanced accuracy, and the macro-averaged F1 score. These metrics were selected to provide a comprehensive view of model performance, particularly given the class imbalance within the dataset.

Accuracy measures the proportion of correctly classified samples relative to the total number of predictions. While it provides a general indication of performance, it can be misleading in imbalanced datasets, as models may achieve high accuracy by favouring more frequently occurring classes. This can result in artificially inflated performance that does not reflect true classification ability.

To address this limitation, balanced accuracy was used as the primary evaluation metric. Balanced accuracy computes the average recall across all classes, ensuring that each class contributes equally to the final score regardless of its size. This was particularly important for this project, as certain regions, such as Ulster and Leinster, were more heavily represented than others (see Figure B5.2.1, Appendix B).

The macro-averaged F1 score was also used to evaluate the balance between precision and recall across all classes. By treating each class equally, it provides additional insight into how well the model performs on less represented classes (see Figure B2.2, Appendix B).

Together, these metrics provide a well-rounded evaluation of model performance, capturing overall accuracy while also assessing how effectively models generalise across all classes. Among these, balanced accuracy proved to be the most informative metric, as it reduces bias toward dominant classes and provides a more reliable measure of performance in the context of imbalanced data (see Figures B1.1 and B2.1, Appendix B).

### 6.3.3 Comparison of MFCC and Wav2Vec Performance

A key development in this project was the identification of limitations in traditional MFCC-based feature extraction, which led to a transition toward a wav2vec-based approach.

MFCC-based models produced relatively limited performance, with results decreasing as task complexity increased. For example, in the four-province classification task, MFCC-based models achieved a mean balanced accuracy of 36.12% (see Figure B1.1.1, Appendix B), while wav2vec-based models achieved 42.61% (see Figure B2.1.1, Appendix B), representing an improvement of 6.49 percentage points. Although the overall performance of both approaches remained constrained, this improvement is notable given the difficulty of the task.

Irish regional accents exist within a relatively small geographic area and share many overlapping phonetic features, making class separation challenging. As a result, even modest improvements in performance reflect a meaningful increase in the model's ability to capture subtle patterns within the speech data.

The difference between the two approaches is more apparent when analysing individual classification tasks. As shown in Table 1, wav2vec-based models consistently outperform MFCC-based models across all evaluated tasks.

| TASK | MFCC BA | MFCC F1 | Wav2Vec BA | Wav2Vec F1 | BA Improvement |
|---|---|---|---|---|---|
| 4 Provinces (dail only) | 26.07 | 23.65 | | | |
| 4 Provinces (dail and ni) | 36.12 | 34.78 | 42.62 | 41.16 | 6.5 |
| dail vs ni | 70.06 | 71.85 | | | |
| ulster leinster vs rest | 47.73 | 47.87 | 55.15 | 55.02 | 7.42 |
| leinster vs rest | 56.08 | 54.49 | 61.82 | 60.32 | 5.74 |
| ulster vs rest | 69.98 | 69.84 | 75.41 | 76.48 | 5.43 |

*Table 1 - Comparison of wav2vec against MFCC across various performance metrics.*

For example, in the Ulster vs Rest task, balanced accuracy increased from 69.98% (MFCC) to 75.41% (wav2vec), while the macro-averaged F1 score improved from 69.84% to 76.48%. Similar improvements were observed across all tasks, with an average balanced accuracy increase of 6.27%. The largest improvement was observed in the Ulster vs Leinster vs Rest task, where performance increased by 7.42 percentage points.

These results indicate that wav2vec-based representations are more effective at capturing subtle distinctions between Irish accent groups. This is likely due to the model's ability to learn high-level acoustic and temporal features through large-scale pre-training, compared to the manually designed features used in MFCC-based approaches.

Despite these improvements, the overall performance gap remains moderate, suggesting that feature representation alone is not the primary limiting factor. Instead, performance is likely constrained by dataset limitations, including class imbalance, limited sample size, and the inherently subtle nature of regional accent variation within Ireland.

### 6.3.4 Task Difficulty and Class Structure

The results from the previous section show clear variation in model performance across different classification tasks. This variation can largely be explained by differences in task complexity, class structure, and dataset distribution.

Binary classification tasks achieved the highest performance overall. For example, the Ulster vs Leinster task reached a balanced accuracy of 75.41% using wav2vec-based models, whereas the four-province task achieved a substantially lower balanced accuracy of 42.61%. This difference reflects the increased difficulty of distinguishing between multiple closely related accent groups.

A key factor influencing performance is the number of classes within each task. As the number of classes increases, classification becomes more complex and the likelihood of misclassification rises. The four-province task highlights this challenge, as the model must distinguish between Leinster, Ulster, Connacht, and Munster, all of which share overlapping linguistic features. This is evident in the confusion matrix for MFCC-based four-province classification (see Figure B4.1, Appendix B), where frequent misclassification occurs between regions.

Analysis of the confusion matrix also suggests that Ulster is more separable than other regions. This is reflected in the balanced accuracy results, where the Ulster vs Rest task achieved 78.44%, compared to 68.10% for the Leinster vs Rest task. This indicates that clearer decision boundaries exist for certain regions, suggesting that some accents are inherently more distinguishable than others.

Class distribution also plays a significant role in task difficulty. There is a clear imbalance in both sample counts and speaker representation across classes (see Figure B5.2, Appendix B). For example, in the Ulster vs Rest task, the Ulster class contains 541 samples from 30 speakers, while the Rest category contains 1,134 samples from 176 speakers. Although balanced accuracy and macro-F1 scores help mitigate class imbalance, unequal speaker representation can still negatively affect the model's ability to generalise.

Task design further influences performance, particularly when dataset composition varies. In the MFCC-based experiments, the "Four Provinces (Dáil Only)" task demonstrates this clearly. In this version, the Ulster class contains only 165 samples from 14 speakers (see Figure B5.2.4, Appendix B), compared to the combined "Dáil and NI" dataset (see Figure B5.2.3, Appendix B). This reduced representation limits the model's ability to learn distinguishing features, resulting in a significantly lower balanced accuracy of 27.21%, compared to 39.39% for the combined dataset (see Figures B1.1.2 and B1.1.4, Appendix B).

### 6.3.5 Impact of Dataset Limitations

A major influencing factor of the project was the limitation of the dataset that was used for both training and evaluation. Throughout all configurations, the overall performance was always limited by the size, imbalance and the quality of the available data.

The dataset only contained  1675 audio samples across 212 speakers; significantly smaller than typical datasets used in speech classification studies (Radford, A., et al, 2022)). This restricts the model's ability to learn generalisable patterns, becoming increasingly apparent as tasks with more classes are introduced.

On top of the limited size, class imbalance is another challenge. Certain regions are much more heavily represented than others (see Figure B5.2.1, Appendix B). Both Ulster and Leinster are much more represented than Connacht. And Munster. This is directly tied to a population imbalance on the island of Ireland, as seen in the table below:

| Feature | Leinster | Connacht | Munster | Ulster (Exc. NI) | Ulster (NI) | Total |
|---|---|---|---|---|---|---|
| Amount | 2870000 | 591000 | 1373000 | 314000 | 1927000 | 7075000 |
| Percent | 40.565371 | 8.35335689 | 19.4063604 | 4.438162544 | 27.23674912 | 100 |

*Table 2 - Population of Ireland by Province. (sourced from: Central Statistics Office (2022) and Northern Ireland Statistics and Research Agency(2022) )*

| Class | Samples | Percent |
|---|---|---|
| Leinster | 481 | 29.728059 |
| Ulster | 541 | 33.436341 |
| Connacht | 249 | 15.38937 |
| Munster | 347 | 21.44623 |
| | 1618 | 100 |

*Table 3 - Percentage of Samples by Province.*

Even with class balancing implemented, this population distribution appears clearly in the data sample collection.

Even more influential is the speaker imbalance within the dataset. While sample counts may appear moderately balanced, the unique speaker count is very low. This is most apparent in Ulster, where

541 clips are sourced from 30 speakers. This limits the diversity of speech patterns analysed, and therefore

### 6.3.6 Model Selection for Deployment

Following the evaluation of multiple models, feature extraction methods, and classification tasks, a subset of models was selected for integration into the final system. This selection was based primarily on performance metrics, while also considering the exploratory and user-facing nature of the application.

As demonstrated in Section 6.3.3, wav2vec-based models consistently outperformed MFCC-based approaches across all tasks. Although the performance improvement was moderate, wav2vec models achieved higher balanced accuracy and macro F1 scores across all comparable configurations (see Table 1, Section 6.3.3). As a result, wav2vec-based models form the core of the deployed system.

However, model selection was not limited to choosing a single best-performing model overall. Instead, the highest-performing model for each classification task was selected individually (see Figure B2.5, Appendix B). This ensures that each task is supported by the most appropriate model configuration.

In addition to performance, variety was also considered. Given the experimental nature of the project, it was important to allow users to explore different modelling approaches. For this reason, a small number of MFCC-based models were retained alongside higher-performing wav2vec models. This enables users to observe differences in behaviour and better understand the impact of feature extraction methods on classification performance.

The tasks selected for final deployment are:

1. Four Provinces task, MFCC-based Logistic Regression model.
2. Ulster vs Rest task, wav2vec-based Random Forest Model.
3. Leinster vs Rest task, wav2vec-based Logistic Regression Model.
4. Ulster, Leinster vs Rest task, wav2vec-based Logistic Regression Model.
5. Four Provinces, wav2vec-based Logistic Regression.

See Figure C1.1, Appendix C. See plugin_loader.py, D2.2, Appendix D.

Each selected model was integrated into the system using the plugin architecture described in Section 5.5. This allows models to be dynamically loaded and selected at runtime through the user interface, supporting flexibility and reinforcing the experimental nature of the system.

### 6.3.7 Summary of Findings

The key findings from the evaluation are as follows:

- Wav2vec-based models consistently outperformed MFCC-based models across all classification tasks, although gains remained small.

- The performance of models was strongly linked to class complexity, with binary tasks achieving higher accuracy than multi-class tasks.
- Certain geographic regions, such as Ulster, were more distinguishable, while other regions, such as Connacht and Munster (see Figure 4, Section 5.6.1), have more shared features making classification more difficult, showing greater overlap and confusion.
- Dataset limitations, including small size, class imbalance, and limited speaker diversity, were the primary factors that limited the model performance.
- Improvements in feature extraction were insufficient to overcome these limitations, indicating the dataset quality was the most influential aspect of the accent classification performance.
- The final system design reflects these findings by prioritising wav2vec-based models while also including a range of alternative approaches to support exploration and comparison

## 6.4. Evaluate Project Management

The aim of this section was to evaluate how well the project was planned and then executed. Due to the experimental nature of the project, traditional linear planning wasn't suitable and therefore a more flexible, iterative approach was used. This allowed the project to continuously adapt in response to the findings, dataset limitations and any technical challenges I faced.

### 6.4.1 Evaluate Project Plan

The project plan had a structured plan with key development stages outlined, such as:

- Dataset creation
- Experimentation
- System implementation
- Evaluation

These stages provided clear progression points and ensured that the overall objectives of the project were defined from the start.

However, as development progressed, it became apparent that a linear plan could not be followed. Much more time was spent on dataset construction than was expected, due to tasks such as sourcing suitable audio samples, isolating the speakers, gathering metadata based on these selected speakers, and validating clip quality all took up significant amount of time due to their manual nature, which impacted the original timeline.

The divergence from the approach defined in the interim report show this clearly. The plans create in the interim report were made with the assumption that the creation of sufficiently large and high quality datasets would be manageable, and that MFCC-based feature extraction would be effective in accent classification. As experimentation progressed, these assumptions were proven to be inaccurate, with the limitations on the dataset, the class imbalance and the subtle nature of regional Irish accents making fine-grained accent classification much more difficult than expected. The project direction ended up shifting at this point, with classifications being made more coarse larger regions, and the feature extraction approach shifting from MFCC-based representations to wav2vec-based embeddings (see Section 5.2.4), improving performance results.

These changes, although splitting from the original plan, were essential to keeping the system aligned with realistic constraints and allowing for the system to produce improved results.

Despite the changes made, the project plan was still used as a high level guide, with its milestones helping to keep me on track and ensured that all the major components of the system were successfully developed. The ability to adapt the plan in response to the experiment results shows the iterative and evidence driven approach to project management.

### 6.4.2 Evaluate Project Execution

The execution of the project followed an iterative and experiment-driven approach, which proved essential given the uncertainty surrounding dataset availability and model performance. Rather than following a fixed sequence of stages, development progressed through cycles of experimentation, evaluation, and refinement.

One of the key strengths of the project execution was the structured experimental pipeline (see D1, Appendix D). The transition from notebook-based experimentation to a batch processing system significantly improved the scalability and organisation of experiments. This allowed multiple models and tasks to be evaluated consistently, supporting more reliable decision-making. Additionally, the modular system design, particularly the plugin-based architecture, enabled efficient integration of models into the final system without requiring major structural changes.

However, several challenges impacted execution. Dataset construction required significantly more time than anticipated, which delayed progress in other areas. In addition, the shift from MFCC-based features to wav2vec embeddings introduced additional complexity, particularly in terms of dependency management and system integration. Integrating machine learning components into a full end-to-end application also required careful coordination between frontend and backend systems.

Despite these challenges, the project was executed successfully, with all major components completed. The iterative approach allowed the system to evolve in response to emerging constraints and experimental findings. Overall, the execution of the project was effective, demonstrating adaptability and the ability to respond to technical challenges as they arose.

## 6.6. Conclusions

This chapter presented a comprehensive evaluation of both the system and the machine learning components developed in this project. Testing (see Appendix C) confirmed that the system functions correctly as an end-to-end application, supporting audio input, model selection, prediction generation, and geospatial visualisation in a consistent and reliable manner.

From a machine learning perspective, the evaluation demonstrated that wav2vec-based models consistently outperformed MFCC-based approaches across all classification tasks. However, the overall performance remained constrained, particularly for more complex multi-class tasks. This reflects the inherent difficulty of fine-grained accent classification within Ireland, where regional accents exist along a continuum and share many overlapping features.

A key finding throughout the evaluation was the significant impact of dataset limitations. The relatively small dataset size, class imbalance, and limited speaker diversity were identified as the primary factors restricting model performance. While improvements in feature extraction provided measurable gains, they were not sufficient to overcome these underlying data constraints.

Task design was also shown to play a critical role in performance. Simpler binary classification tasks achieved substantially higher accuracy than more complex multi-class tasks, highlighting the importance of aligning model design with the characteristics of the available data.

The final system reflects these findings, with wav2vec-based models forming the core of the deployed solution, alongside a selection of alternative models to support exploration and comparison (see plugin_loader.py, D2.3, Appendix D). This approach balances performance with the experimental and user-facing goals of the project.

In addition to technical evaluation, this chapter also reviewed project management and execution. The adoption of an iterative development approach proved effective, allowing the project to adapt to challenges and evolve based on experimental results.

This evaluation demonstrates that while accurate fine-grained accent classification within Ireland remains challenging, the system successfully achieves its primary aim of exploring this problem and presenting the results through an interactive geospatial interface. The findings provide a clear foundation for future work, particularly in relation to dataset expansion and model refinement.

# 7. Conclusions and Future Work

## 7.1. Introduction

This chapter concludes the project by reflecting on the work carried out and outlining the potential direction the project could go for future development. It summarises how the project achieved its primary objectives, highlights key limitations identified during evaluation, and proposes areas where the system could be extended or improved.

While the system demonstrates the feasibility of regional accent classification within Ireland and provides an interactive platform for visualising results, there remain several opportunities to enhance both the technical performance and the overall user experience. These are discussed in the following section.

## 7.2. Future Work

The frontend interface could be further developed through user-centred evaluation. Although the system was well tested, it lacked multi-user testing, so usability testing with users from different demographics, regions, and levels of technical experience would provide valuable insight into how effectively the system communicates predictions and supports interaction. These findings could inform improvements to accessibility, clarity of feedback, and overall user experience, ensuring the system is intuitive for a wider audience.

In addition to interface improvements, future work could expand the range of classification tasks supported by the system. While the current implementation focuses on regional Irish accent classification, there is potential to introduce broader tasks using datasets such as Mozilla Common Voice (Mozilla, 2023). For example, models could be developed to distinguish between Irish and non-Irish speech, or to compare Irish accents with other English-speaking regions. This would extend the system beyond a national focus and support more generalised accent classification.

To support this expansion, the geospatial visualisation component could also be extended beyond Ireland. Currently, predictions are displayed using province-level polygons on a map of Ireland. Future iterations could incorporate country-level or global maps, enabling predictions from broader classification tasks to be visualised geographically using international boundary data.

Another area for improvement is the system's testing strategy. While functional and system testing were conducted during development (see Appendix C and A.2, Appendix A), a more comprehensive automated testing framework could be introduced. This could include unit tests for audio processing, feature extraction, plugin registration, and model inference, as well as integration tests for key API endpoints such as */api/health*, */api/models*, and */api/predict*. Expanding testing in this way would improve system reliability and maintainability.

From an architectural perspective, the current implementation relies on browser-based local storage (see store.js, D3.5, Appendix D) for prediction history and feedback. While this approach simplifies development and preserves user privacy, future work could explore the use of a backend database to enable persistent storage, multi-user support, and more advanced analysis of user interactions. This could be extended further through the introduction of authentication and user management.

Further improvements could also be made to the machine learning pipeline. Although wav2vec-based models demonstrated improved performance over MFCC-based approaches, overall gains remained limited (see B1, B2 and B3, Appendix B). Future work could include more extensive parameter tuning, experimentation with alternative model architectures, or the combination of multiple feature representations. Additionally, the existing feedback system could be extended into a human-in-the-loop framework, where corrected predictions are collected and used to support model retraining.

Finally, future work could focus on deploying the system as a production-ready application. This would involve improving backend performance to handle concurrent requests, implementing logging and monitoring, and introducing model versioning to track changes over time. Deploying the system publicly would allow for real-world usage and provide valuable data for ongoing evaluation and improvement.

## 7.3. Conclusions

This project set out to investigate the feasibility of classifying regional Hiberno-English accents using machine learning techniques and to present the results through a geospatial interface. The system successfully integrated audio processing, machine learning models, and interactive visualisation into a cohesive, user-facing application.

The experimental evaluation demonstrated that while accent classification is achievable to a degree, performance is strongly constrained by the nature of the problem and the available data. Wav2vec-based feature extraction provided consistent improvements over traditional MFCC-based methods, highlighting the value of modern representation learning approaches. However, the results also showed that dataset limitations, particularly size, class imbalance, and limited speaker diversity, had the greatest impact on the overall performance.

The project also highlighted the challenges of fine-grained accent classification within Ireland, where regional accents overlap greatly and share many linguistic features. As a result, simpler classification tasks achieved higher performance than more complex multi-class tasks, reinforcing the importance of aligning model design with the characteristics of the data.

Despite these challenges, the system achieved its primary aim as an exploratory and experimental platform. The integration of geospatial visualisation enhances interpretability, allowing users to engage with model outputs in a meaningful way. In addition, the modular, plugin-based architecture supports flexibility and future extension, demonstrating a scalable system design.

This project demonstrates both the potential and the limitations of machine learning approaches to regional accent classification. While high-accuracy classification remains difficult under realistic data constraints, the system provides a strong foundation for further research and development in this area.

# References


Anglophonia (2021) *Is creaky voice a valley girl feature? Stancetaking and evolution of a linguistic stereotype*. Available at: https://journals.openedition.org/anglophonia/4104 (Accessed: 16 April 2026).

Amazon Web Services (2023) *Amazon Transcribe*. Available at: https://aws.amazon.com/transcribe/ (Accessed: 16 April 2026).

Baevski, A., Zhou, H., Mohamed, A. and Auli, M. (2020) *wav2vec 2.0: A framework for self-supervised learning of speech representations*. Available at: https://www.proquest.com/working-papers/wav2vec-2-0-framework-self-supervised-learning/docview/2416047903/se-2 (Accessed: 16 April 2026).

Barr, C. (2025) *Classification trees & logistic regression models*. Available at: https://library.tudublin.ie/articles/5806444.6718/1.PDF (Accessed: 16 April 2026).

Bauer, M.K. (2013) *Twang and slang: Regional origin and perceptual dialectology in Ohio*. Available at: https://kb.osu.edu/items/de500e73-7856-5948-89e5-69cd8c5530a5 (Accessed: 16 April 2026).

Bolger, F. (2022) *Comparing classification methods*. Available at: https://library.tudublin.ie/articles/5582110.5915/1.PDF (Accessed: 16 April 2026).

Central Statistics Office (2022) *Census of Population 2022*. Available at: https://www.cso.ie/en/statistics/population/censusofpopulation2022/ (Accessed: 16 April 2026).

DanielGall500 (2022) *dail-eireann-audio-dataset*. Available at: https://github.com/DanielGall500/dail-eireann-audio-dataset/tree/master/clips (Accessed: 16 April 2026).

Davis, S. and Mermelstein, P. (1980) 'Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences', *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), pp. 357–366. doi: 10.1109/TASSP.1980.1163420.

Department of Public Expenditure, Infrastructure, Public Service (2026) *Ireland's Open Data Portal*. Available at: https://data.gov.ie/ (Accessed: 16 April 2026).

Etymonline (no date) *Hibernia*. Available at: https://www.etymonline.com/word/Hibernia (Accessed: 16 April 2026).

FastAPI (2018) *FastAPI documentation*. Available at: https://fastapi.tiangolo.com/ (Accessed: 16 April 2026).

FFmpeg Developers (2024) *FFmpeg*. Available at: https://ffmpeg.org/ (Accessed: 16 April 2026).

Google Cloud (2023) *Speech-to-Text documentation*. Available at: https://cloud.google.com/speech-to-text (Accessed: 16 April 2026).

Gourisaria, M.K. et al. (2024) *Comparative analysis of audio classification with MFCC and STFT features using machine learning techniques*. Available at: https://doaj.org/article/a5230b33e86949eeb59d6b5e666f5a82 (Accessed: 16 April 2026).

Hickey, R. (2004) *A sound atlas of Irish English*. Berlin: Walter de Gruyter.

Hickey, R. (2007) *Irish English: History and present-day forms*. Available at: https://www.academia.edu/66954079/Irish_English_history_and_present_day_forms (Accessed: 16 April 2026).

Internet Engineering Task Force (2015) *The GeoJSON specification*. Available at: https://datatracker.ietf.org/wg/geojson/charter/ (Accessed: 16 April 2026).

J. A. Osorio et al. (2011) 'Moving from waterfall to iterative development: An empirical evaluation of advantages, disadvantages and risks of RUP', in *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications*. Oulu, Finland: IEEE, pp. 453–460. doi: 10.1109/SEAA.2011.69.

Leaflet (2011) *Leaflet API reference*. Available at: https://leafletjs.com/reference.html (Accessed: 16 April 2026).

Matasović, R. (2017) *Addenda et corrigenda to Etymological Dictionary of Proto-Celtic*. Available at: https://www.academia.edu/1489376/Addenda_et_Corrigenda_to_Etymological_Dictionary_of_Proto_Celtic_Brill_Leiden_2009_ (Accessed: 16 April 2026).

McFee, B. et al. (2015) *librosa: Audio and music signal analysis in Python*. Available at: https://brianmcfee.net/papers/scipy2015_librosa.pdf (Accessed: 16 April 2026).

Microsoft Corporation (2018) *Microsoft Excel*. Available at: https://office.microsoft.com/excel (Accessed: 16 April 2026).

Mozilla (2023) *Common Voice dataset*. Available at: https://commonvoice.mozilla.org (Accessed: 16 April 2026).

Mozilla Developer Network (2024) *Storage API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Storage_API (Accessed: 16 April 2026).

Nolan, P. (2024) *Machine learning for feature extraction and classification of English-language accents in Ireland*. MSc thesis. National College of Ireland. Available at: https://norma.ncirl.ie/8600/1/peternolan.pdf (Accessed: 16 April 2026).

Northern Ireland Statistics and Research Agency (2022) *Population statistics*. Available at: https://www.nisra.gov.uk/statistics/people-and-communities/population (Accessed: 16 April 2026).

OpenStreetMap (2004) *OpenStreetMap*. Available at: https://www.openstreetmap.org/ (Accessed: 16 April 2026).

QGIS (2002) *QGIS resource hub*. Available at: https://qgis.org/resources/hub/ (Accessed: 16 April 2026).

Radford, A. et al. (2022) *Robust speech recognition via large-scale weak supervision*. Available at: https://www.proquest.com/working-papers/robust-speech-recognition-via-large-scale-weak/docview/2748631235/se-2 (Accessed: 16 April 2026).

RTÉ Archives (2025) *Canúint*. Available at: https://www.rte.ie/archives/category/archives/2025/0303/1499960-canuint/ (Accessed: 16 April 2026).

Sabuj, H.H. et al. (2023) 'A comparative study of machine learning classifiers for speaker's accent recognition', in *2023 IEEE World AI IoT Congress (AIIoT)*. Seattle, WA: IEEE, pp. 627–632. doi: 10.1109/AIIoT58121.2023.10174511.

The Guardian (2016) *Can Google Translate understand a Liverpool accent?* Available at: https://www.theguardian.com/technology/2016/feb/01/can-google-translate-understand-a-liverpool-accent (Accessed: 16 April 2026).

The Guardian (2022) *Bias against working-class and regional accents has not gone away*. Available at: https://www.theguardian.com/inequality/2022/nov/03/bias-against-working-class-and-regional-accents-has-not-gone-away-report-finds (Accessed: 16 April 2026).

Vocal Image (2023) *Vocal Image: Voice and accent training*. Available at: https://vocalimage.ai (Accessed: 16 April 2026).

# Appendix A: System Testing Evidence

This appendix provides visual evidence of system testing, with a full system interaction, backend API validation, and verification of the plugin architecture.

## A.1 – Full System Run Through

### Initial Interaction

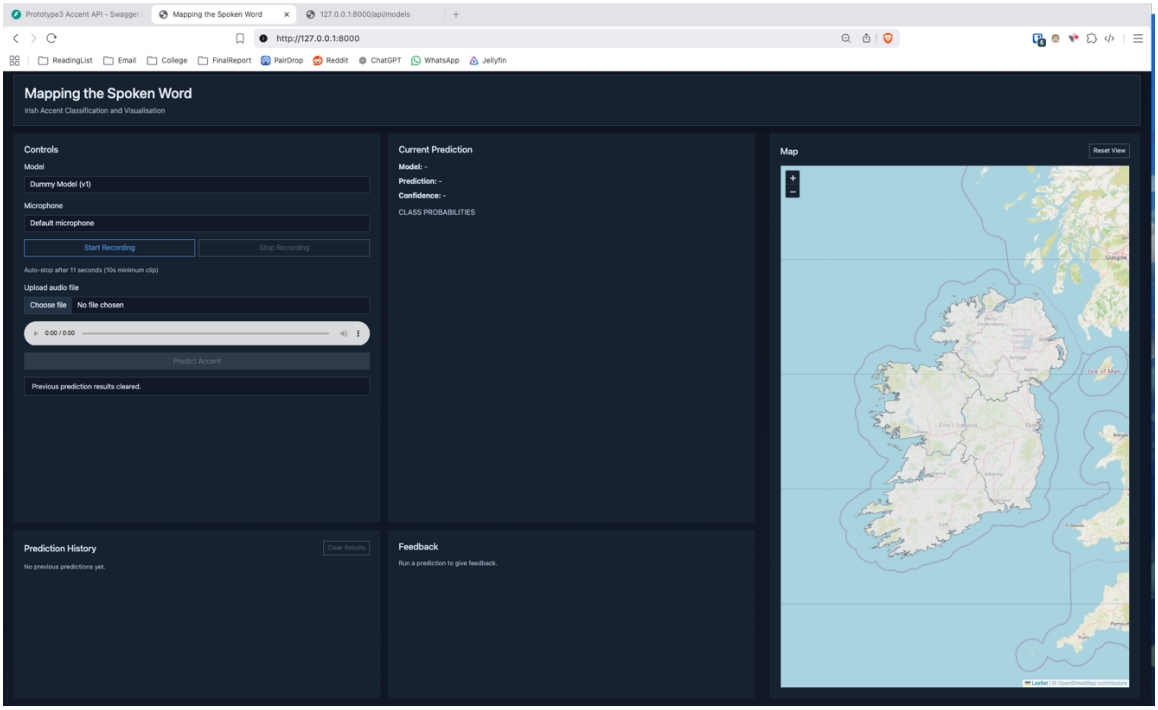


*Figure 5 A1.01 System's initial state upon loading the application's interface.*

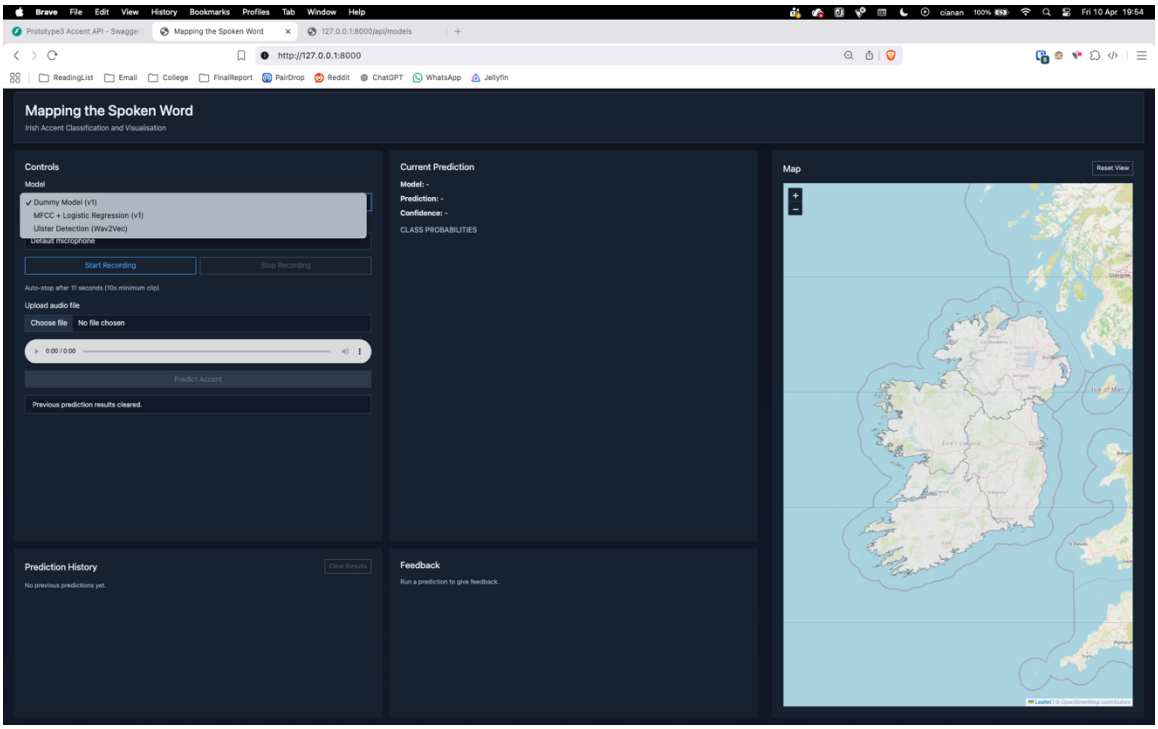


*Figure 6 - A1.02 Model selection dropdown expanded showing available models.*

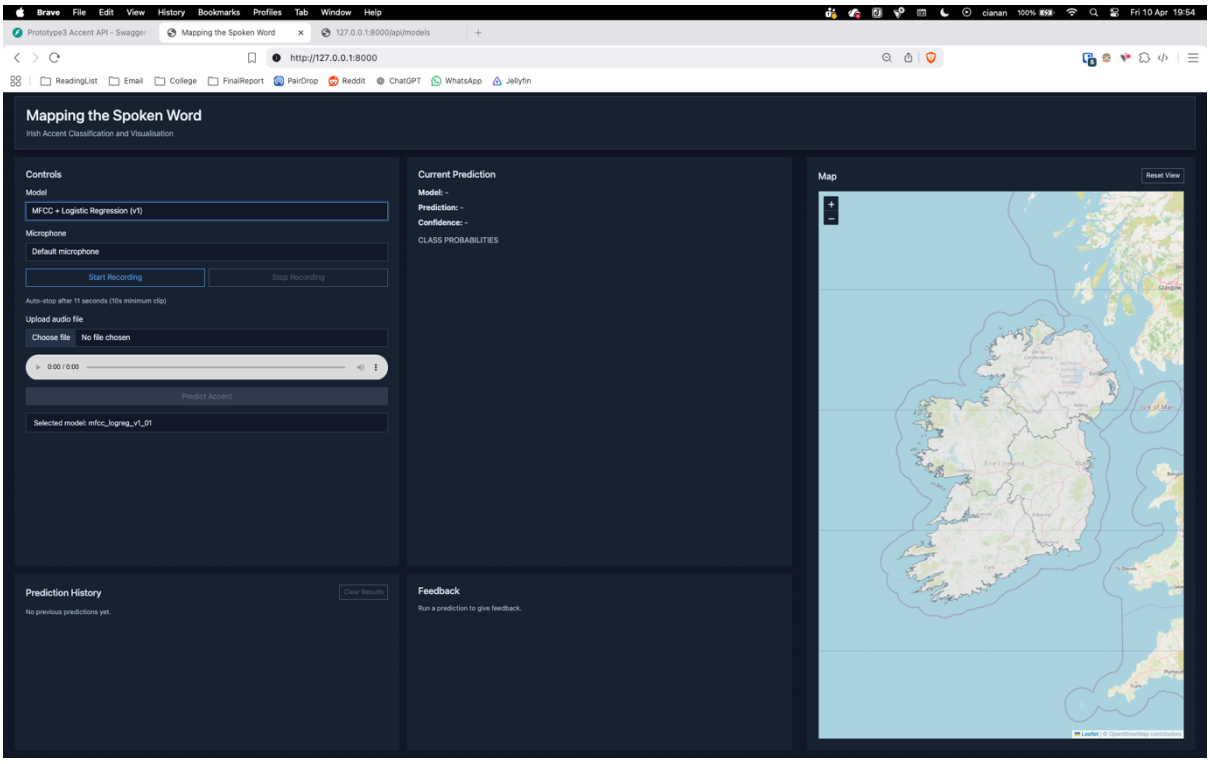


*Figure 7 - A1.03 Selection of a model from the dropdown list.*

## Recording Workflow

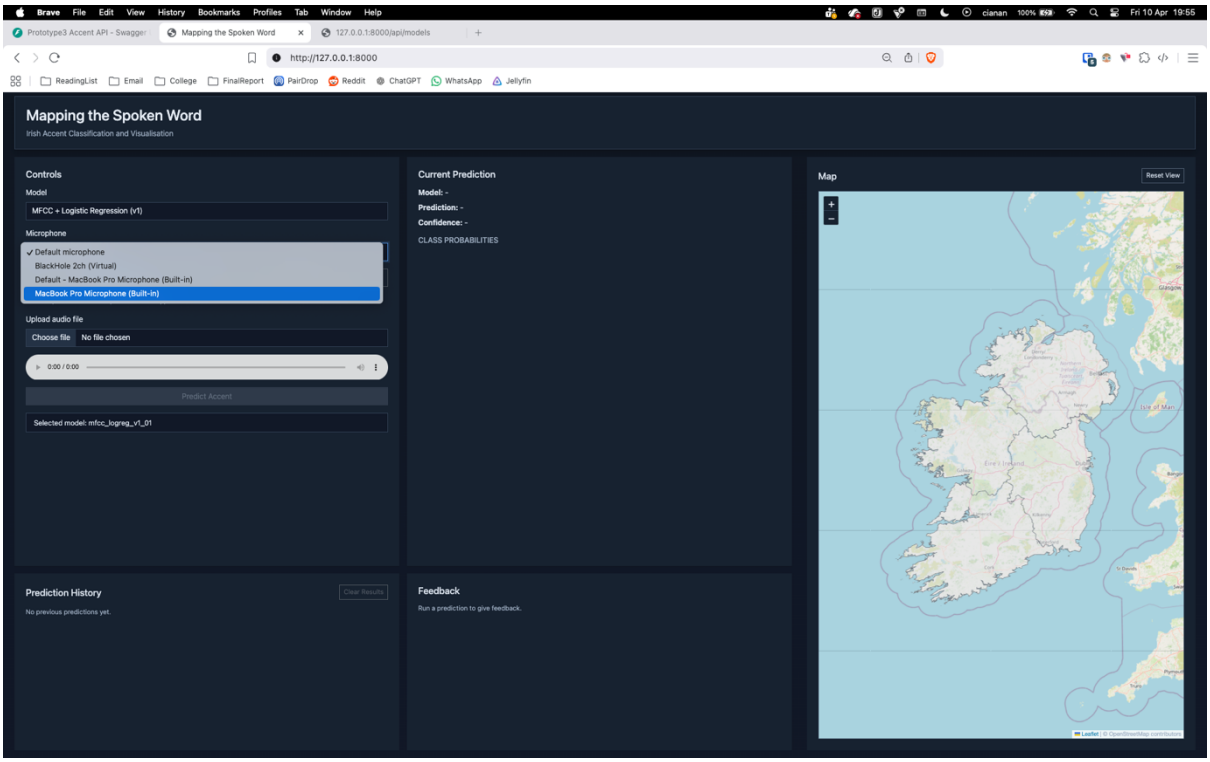


*Figure 8 - A1.04 Selection of microphone from the microphone dropdown list.*

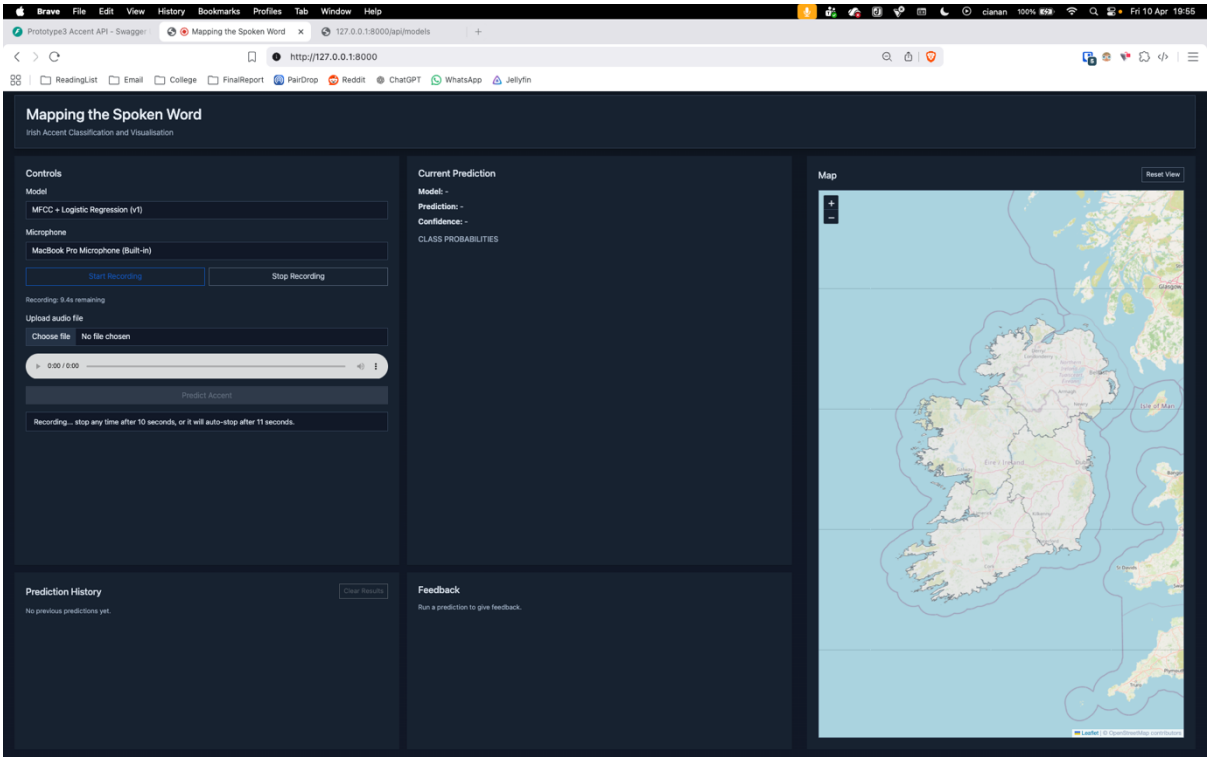


*Figure 9 - A1.05 Recording process initiated by the user.*

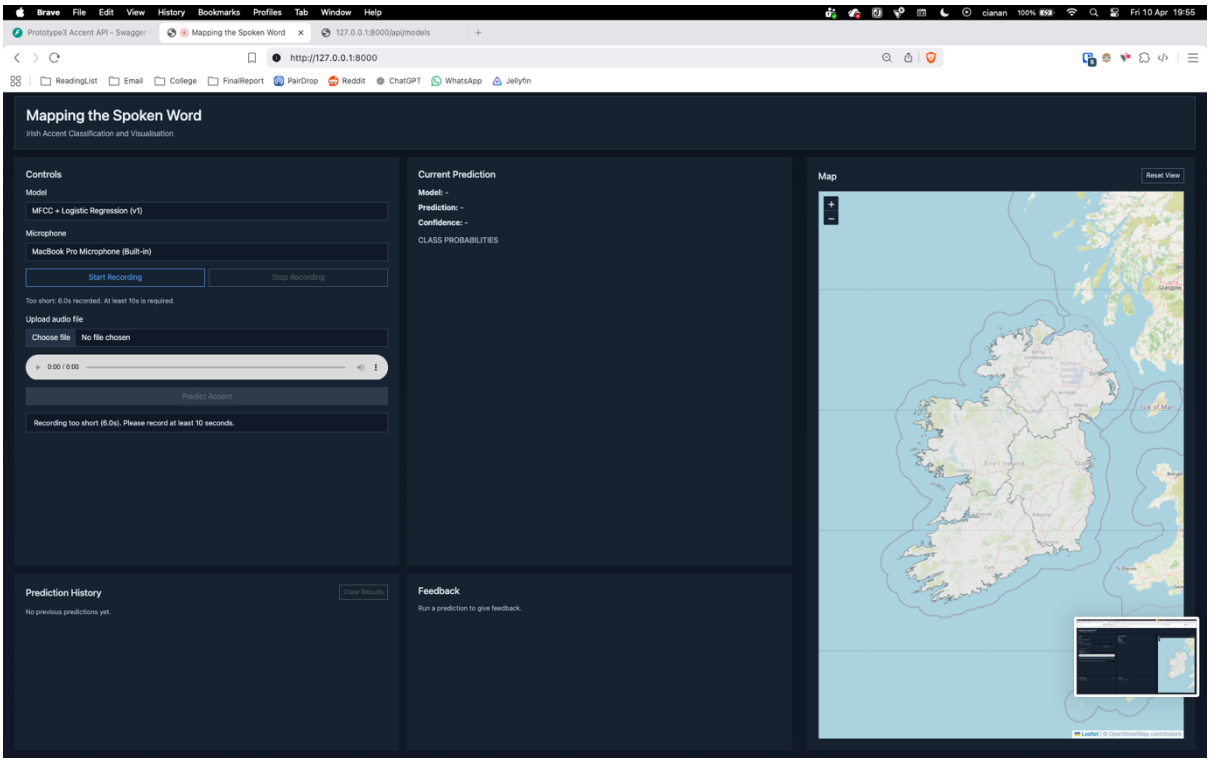


*Figure 10 - A1.06a Recording stopped before minimum duration requirement.*

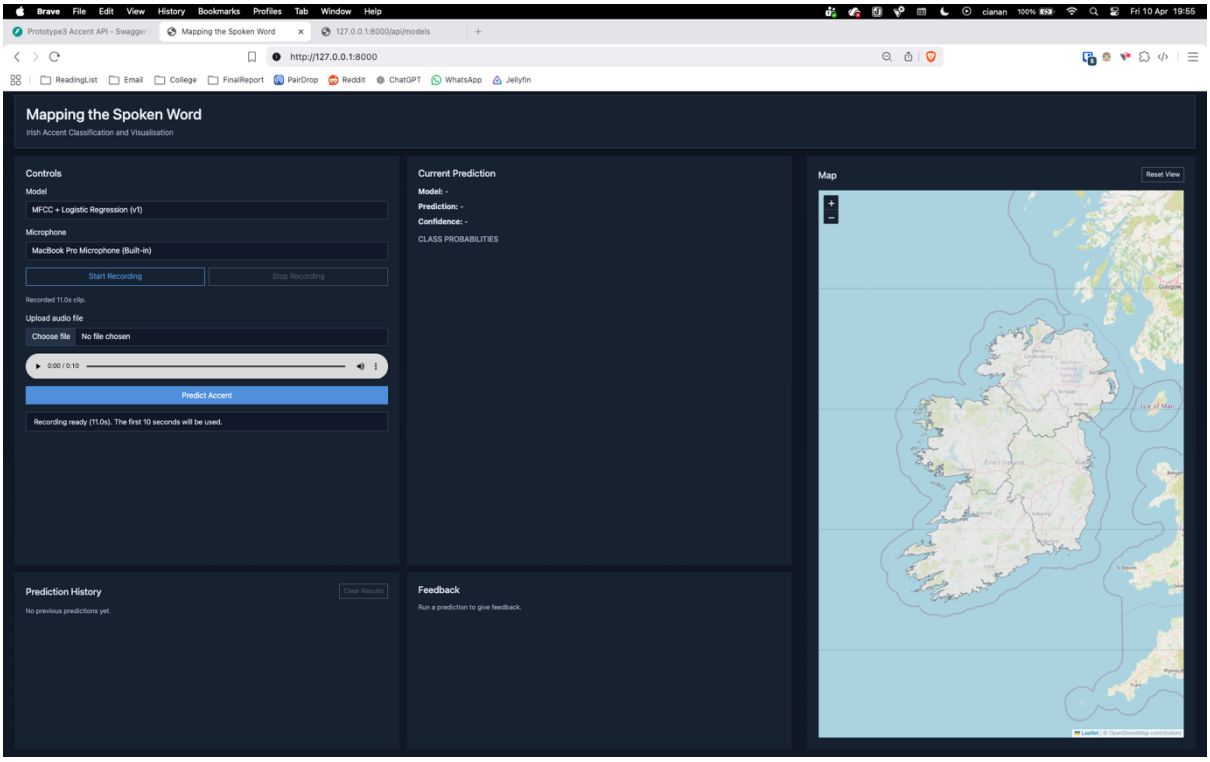


*Figure 11 - A1.06b Recording successfully completed after required duration.*

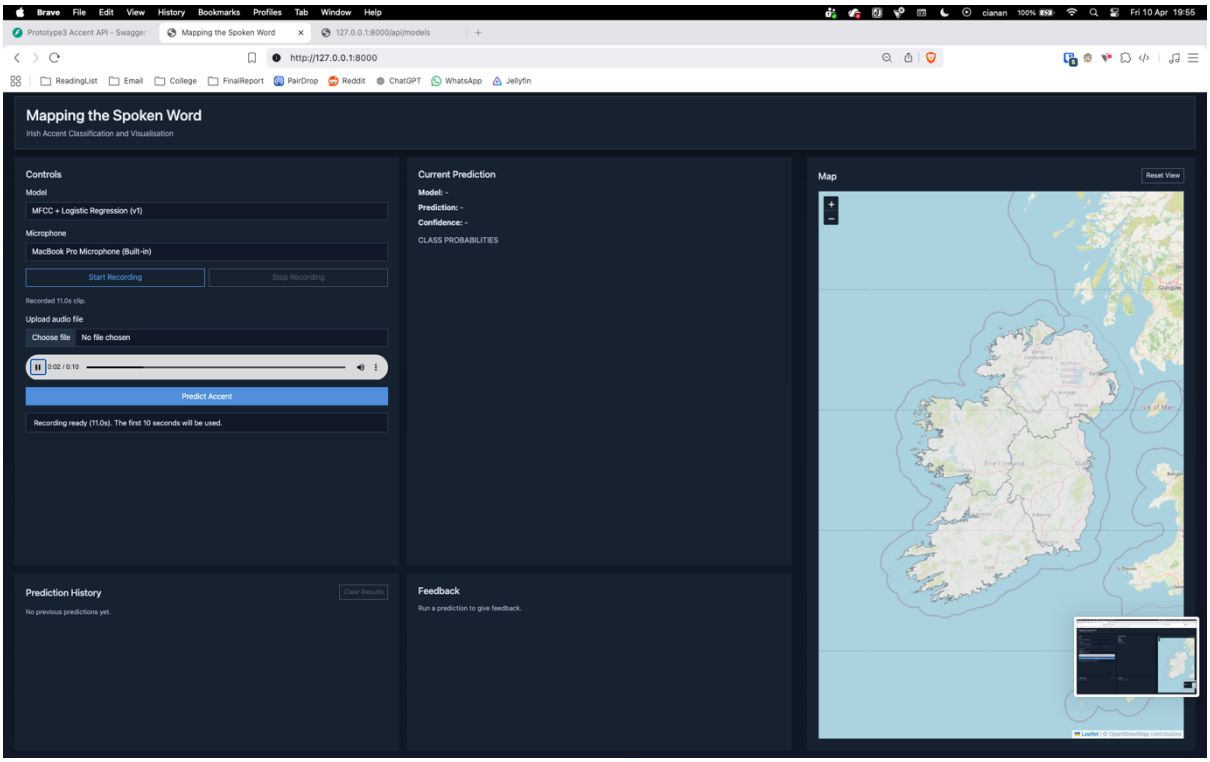


*Figure 12 - A1.07 Playback of recorded audio sample before submission.*

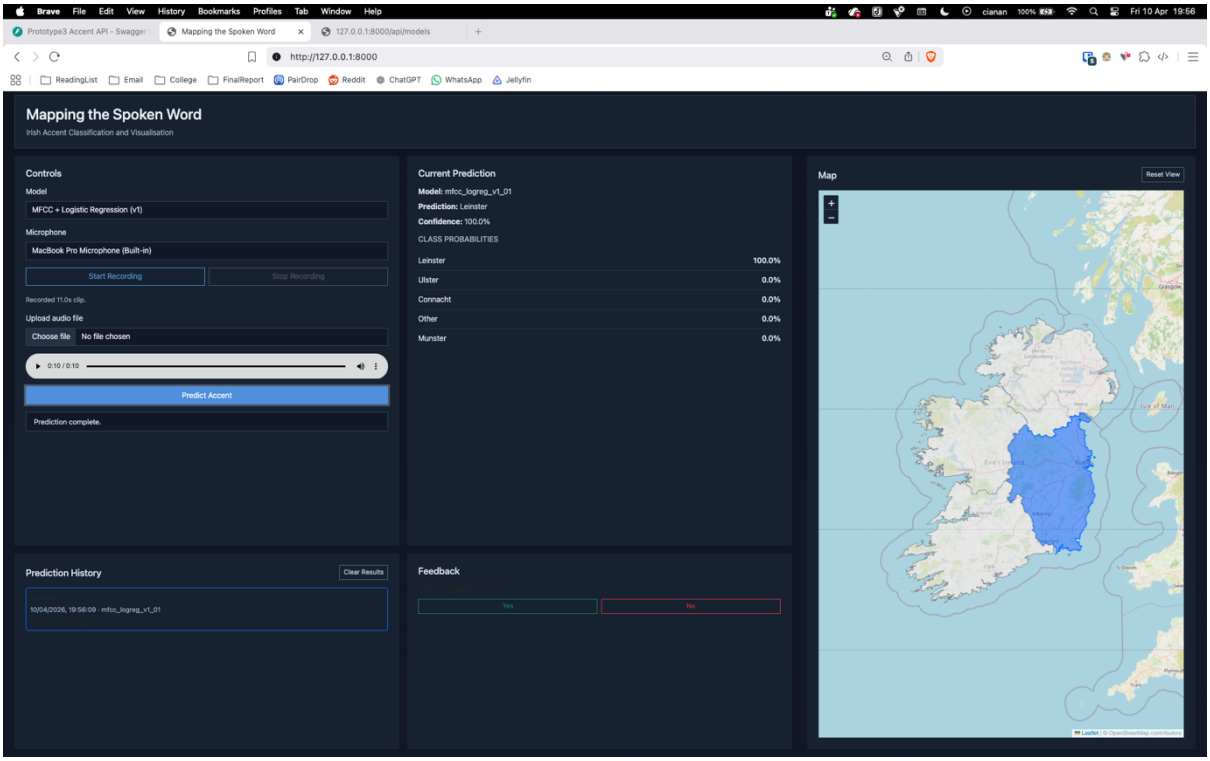


*Figure 13 - A1.08 Prediction made from recorded audio sample with result displayed.*

## File Upload Workflow

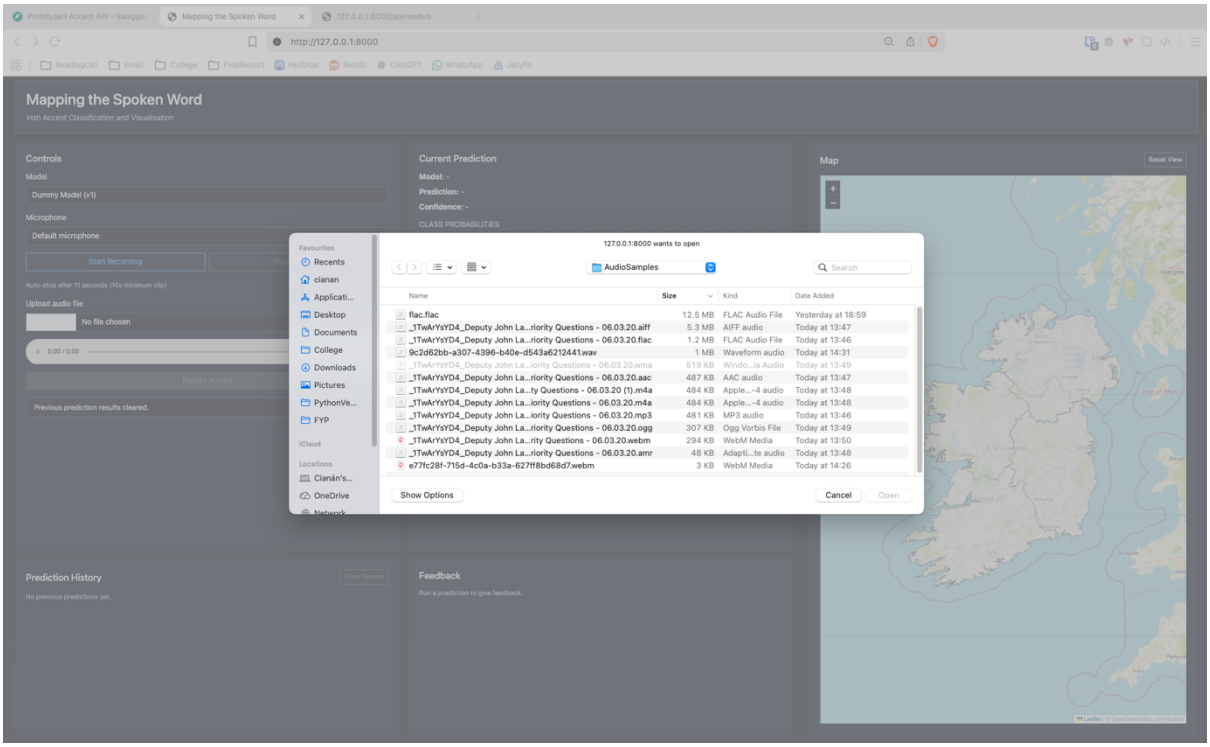


*Figure 14 - A1.09 File upload interface. Valid file types remain usable while invalid file types are greyed out.*

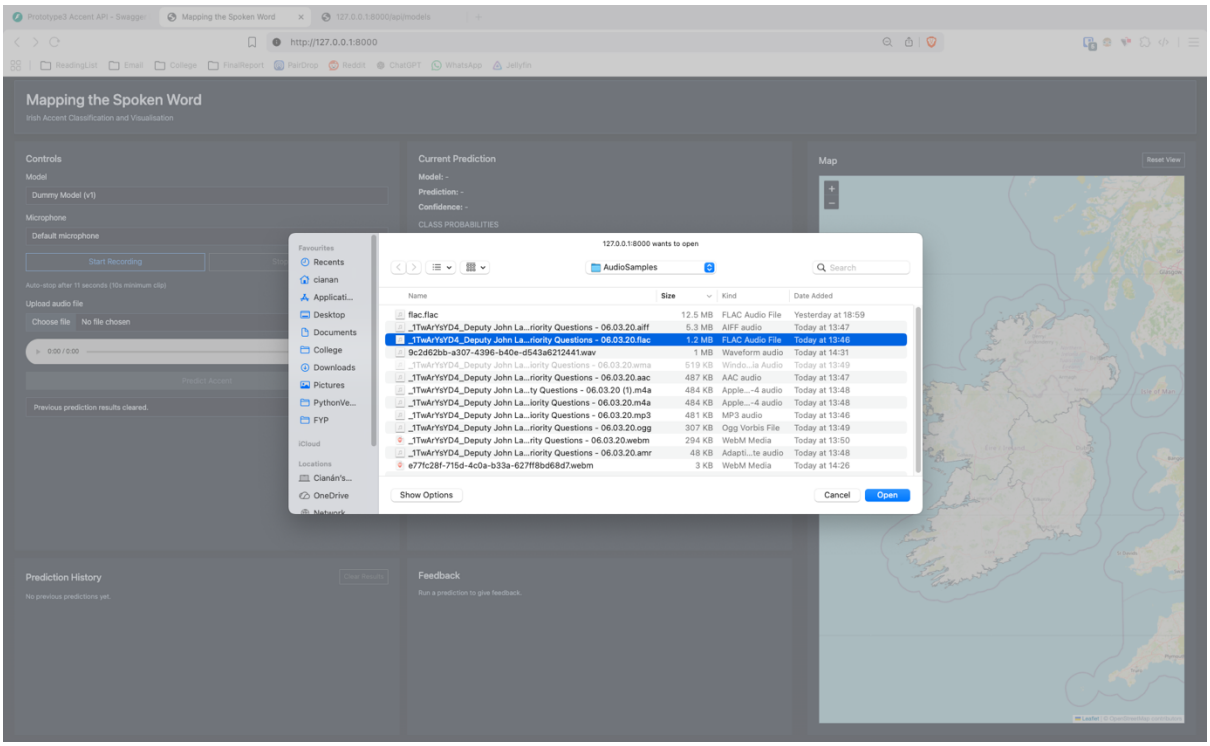


*Figure 15 - A1.10 Selection of an audio file for upload.*

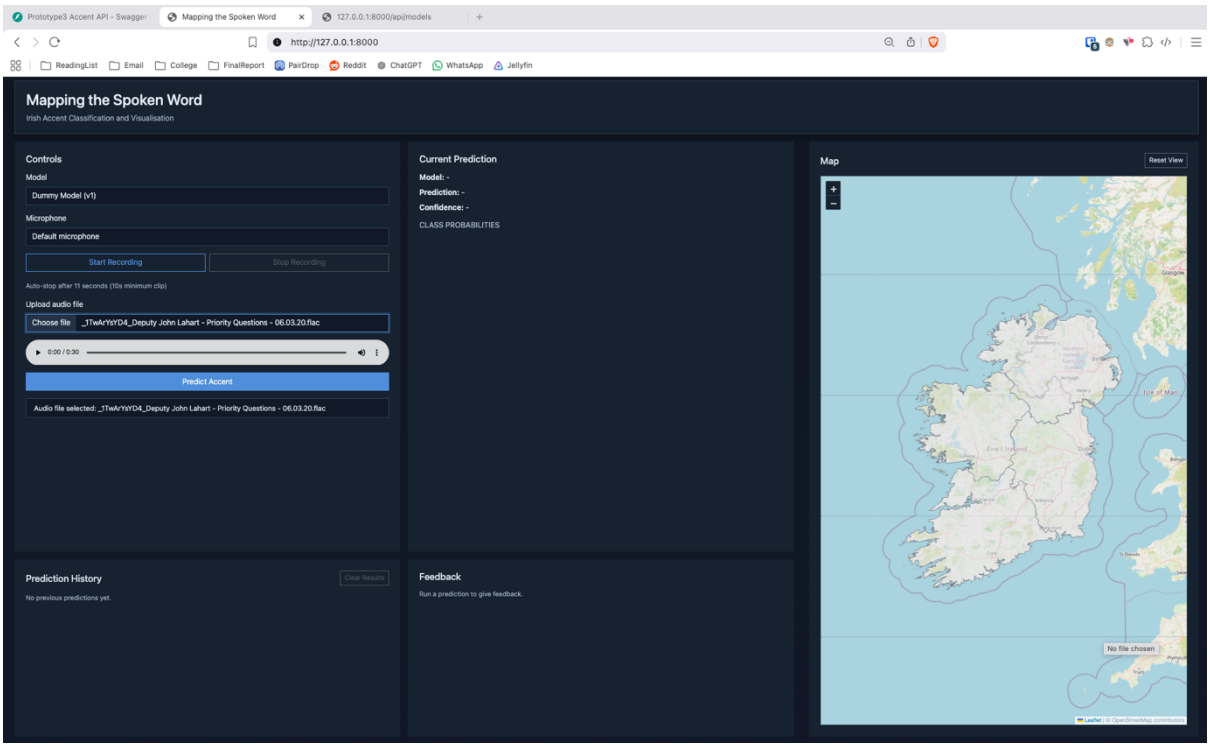


*Figure 16 - A1.11 Interface updated with selected file ready for prediction.*

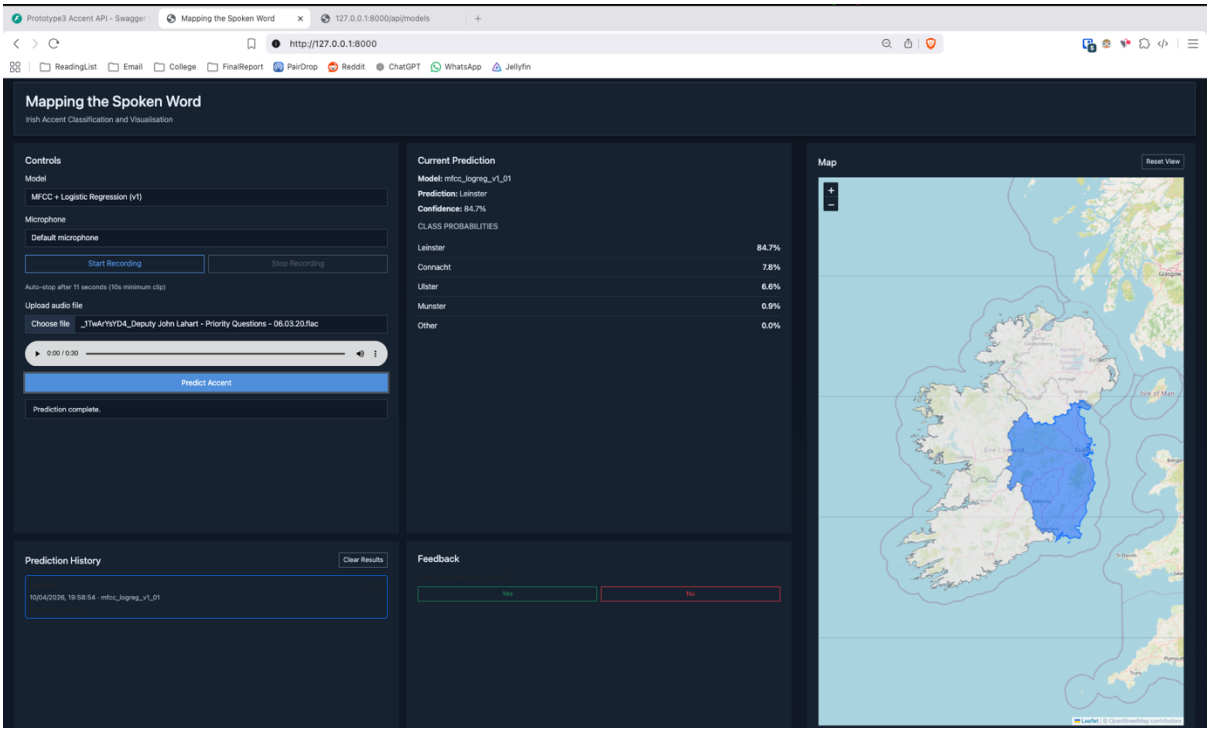


*Figure 17 - A1.12 Prediction generated from uploaded file with result displayed.*

## Model Comparison and Feedback

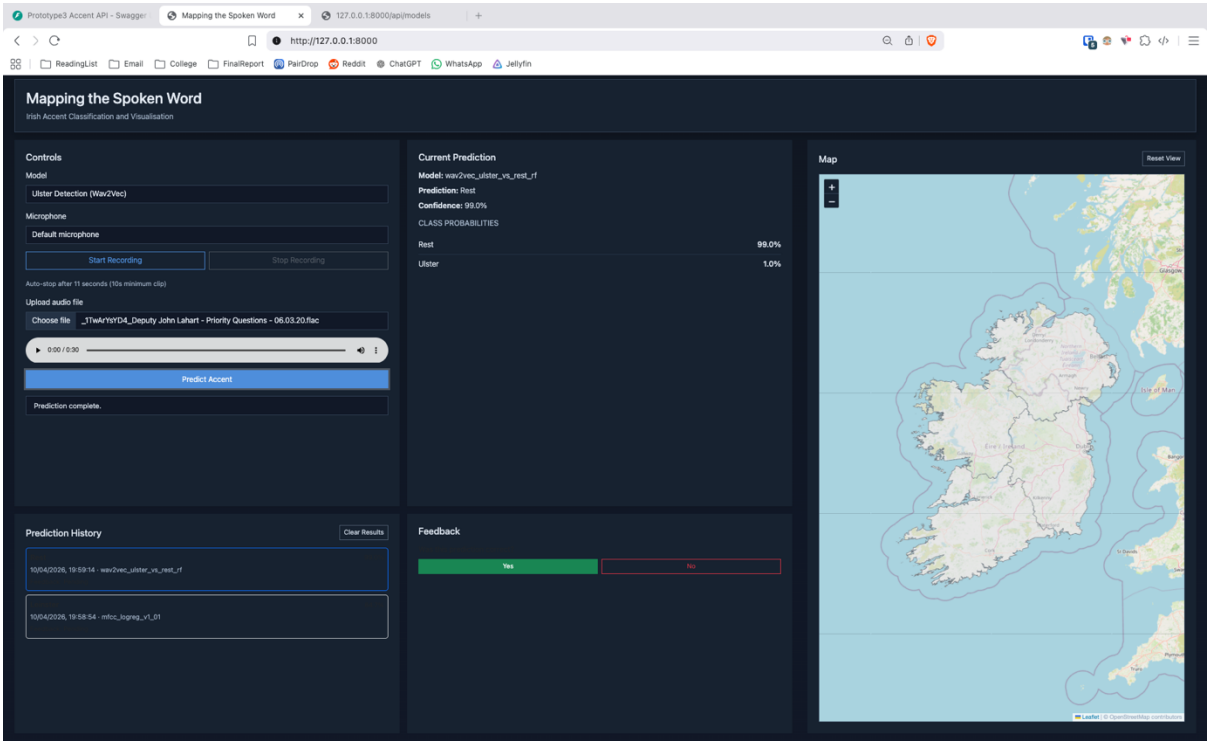


*Figure 18 - A1.13 Prediction performed using a different model with user marking result as correct.*

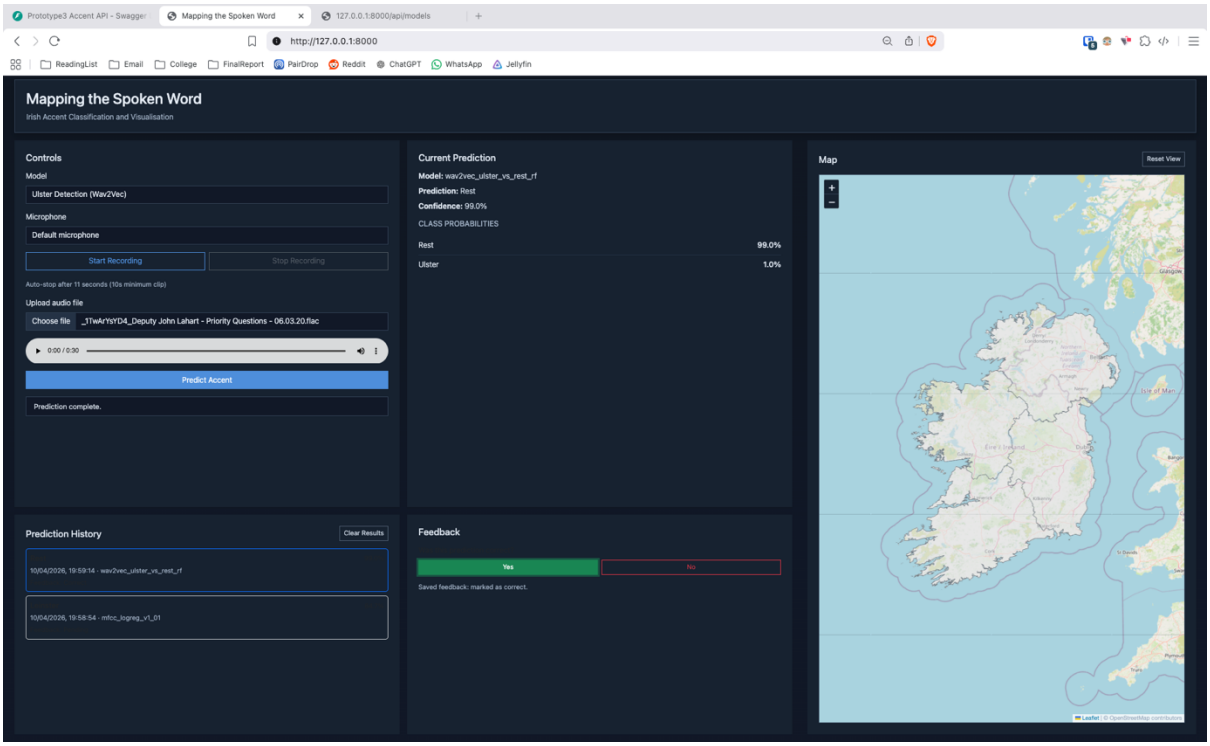


*Figure 19 - A1.14 Confirmation message indicating feedback has been successfully saved.*

## Prediction History and Interaction

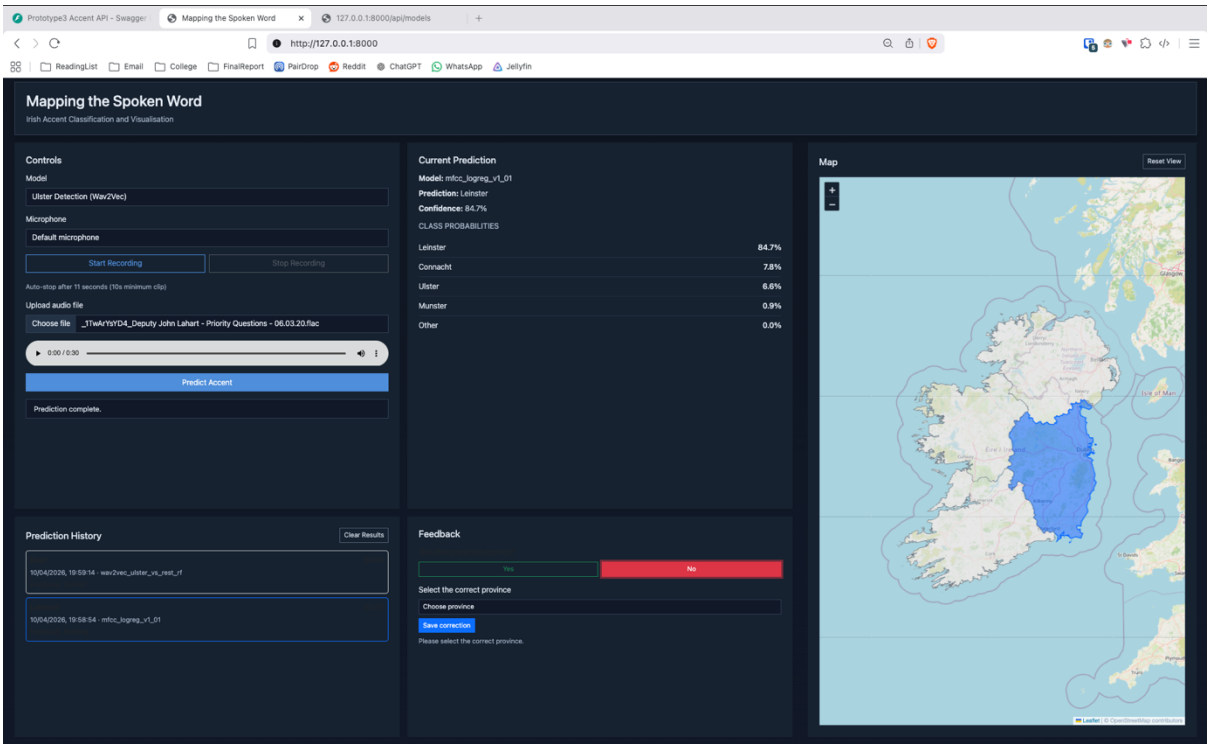


*Figure 20 - A1.15 Selection of a previous prediction from history with user marking it as incorrect.*

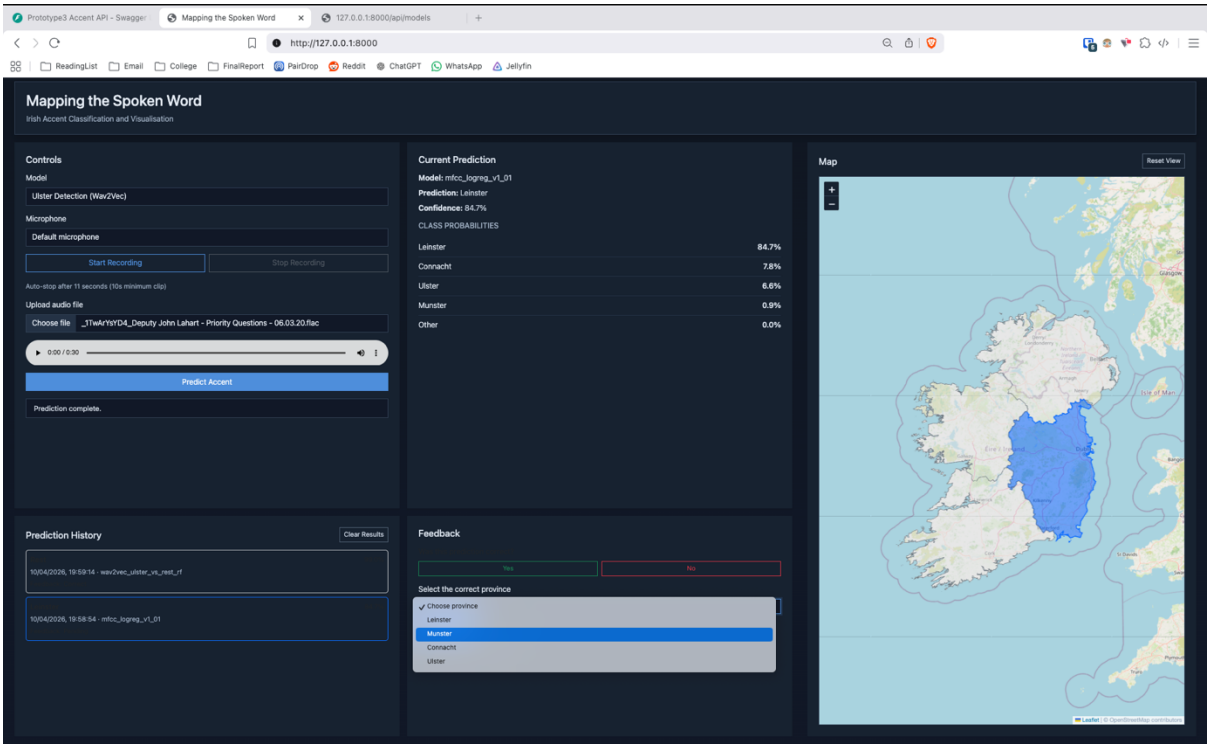


*Figure 21 - A1.16 Updated prediction history reflecting incorrect feedback entry.*

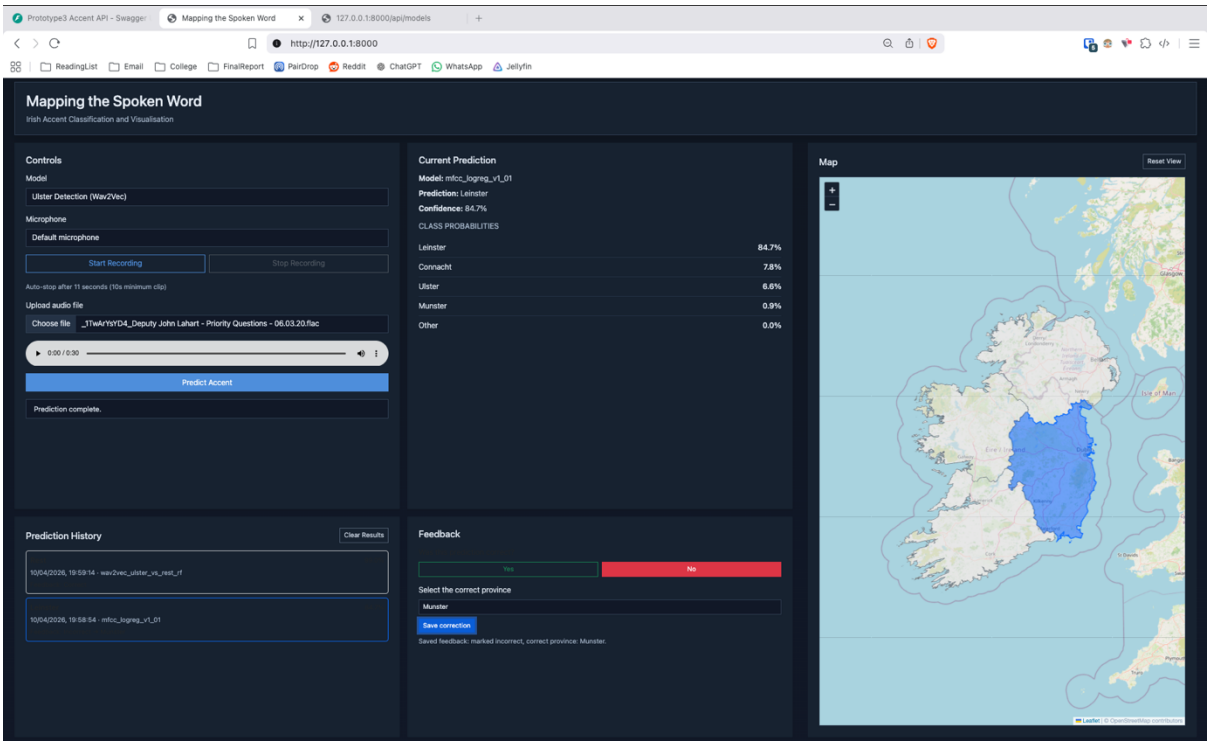


*Figure 22 - A1.17 Selection of corrected prediction result from available options.*

## System Interaction and State Management

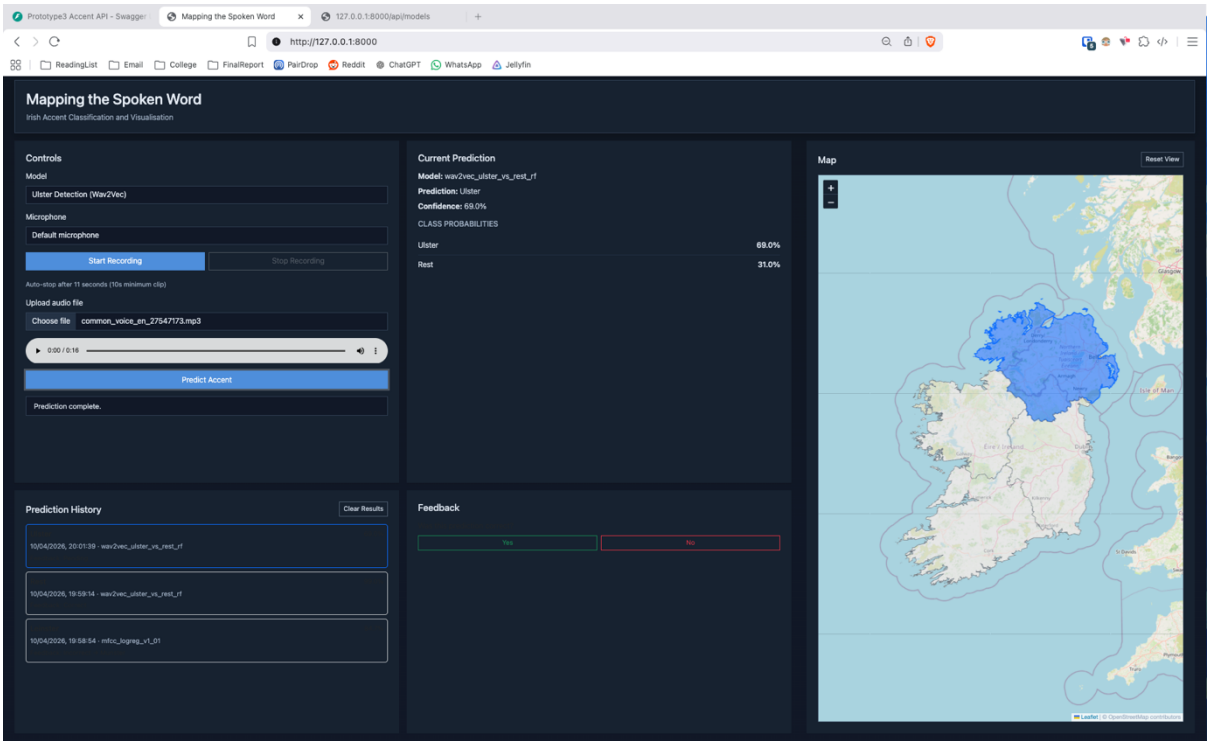


*Figure 23 - A1.18 Execution of a new prediction and display of updated result.*

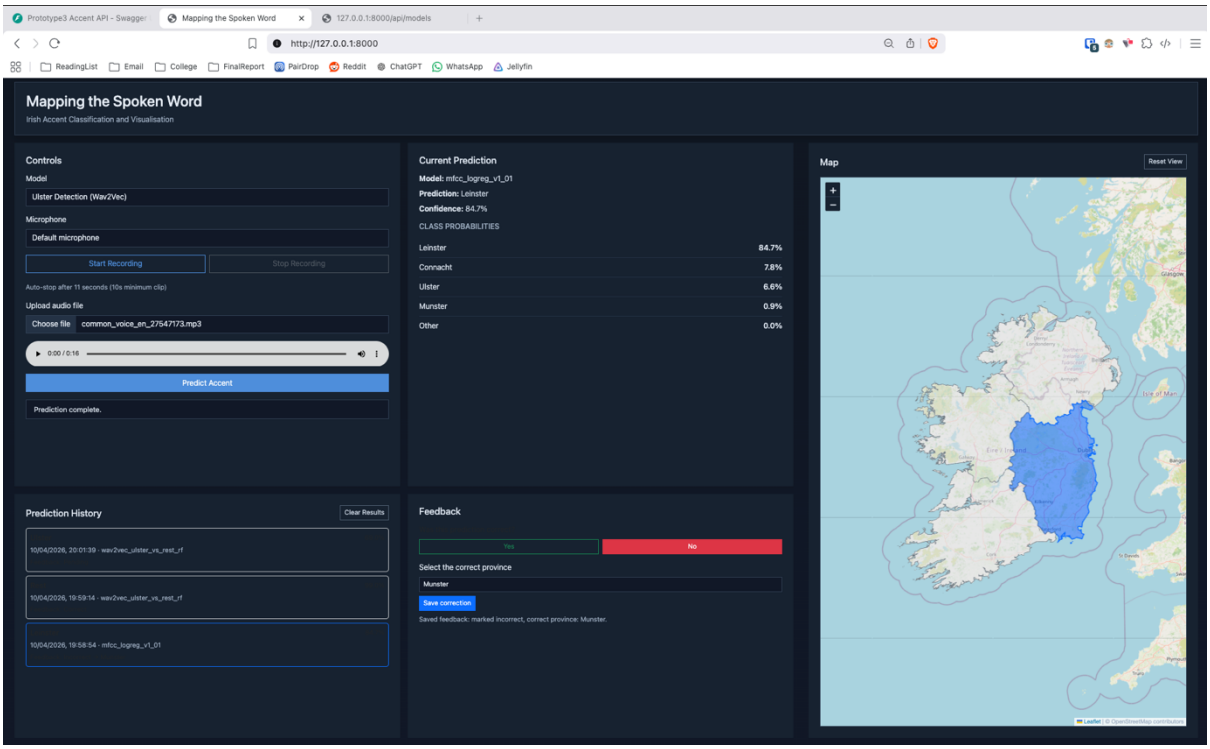


*Figure 24 - A1.19 Re-selection of a previous prediction with corresponding result restored.*

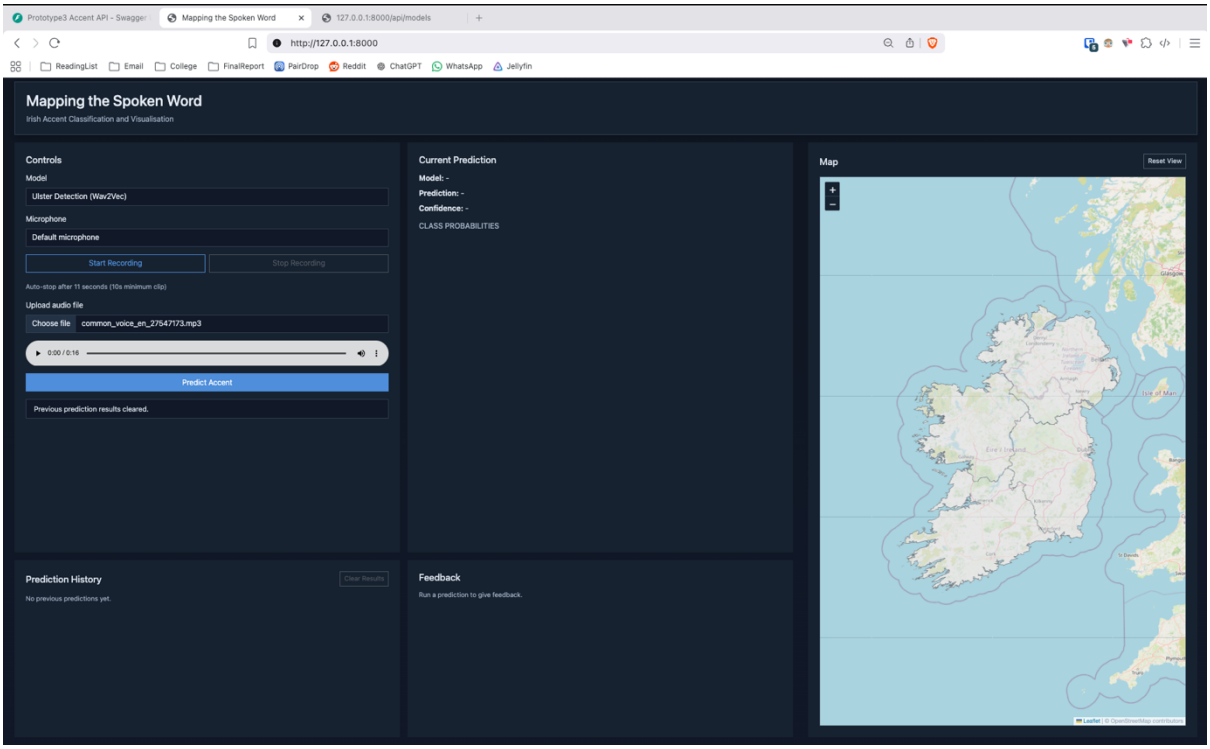


*Figure 25 - A1.20 Clearing of stored prediction history via user action.*

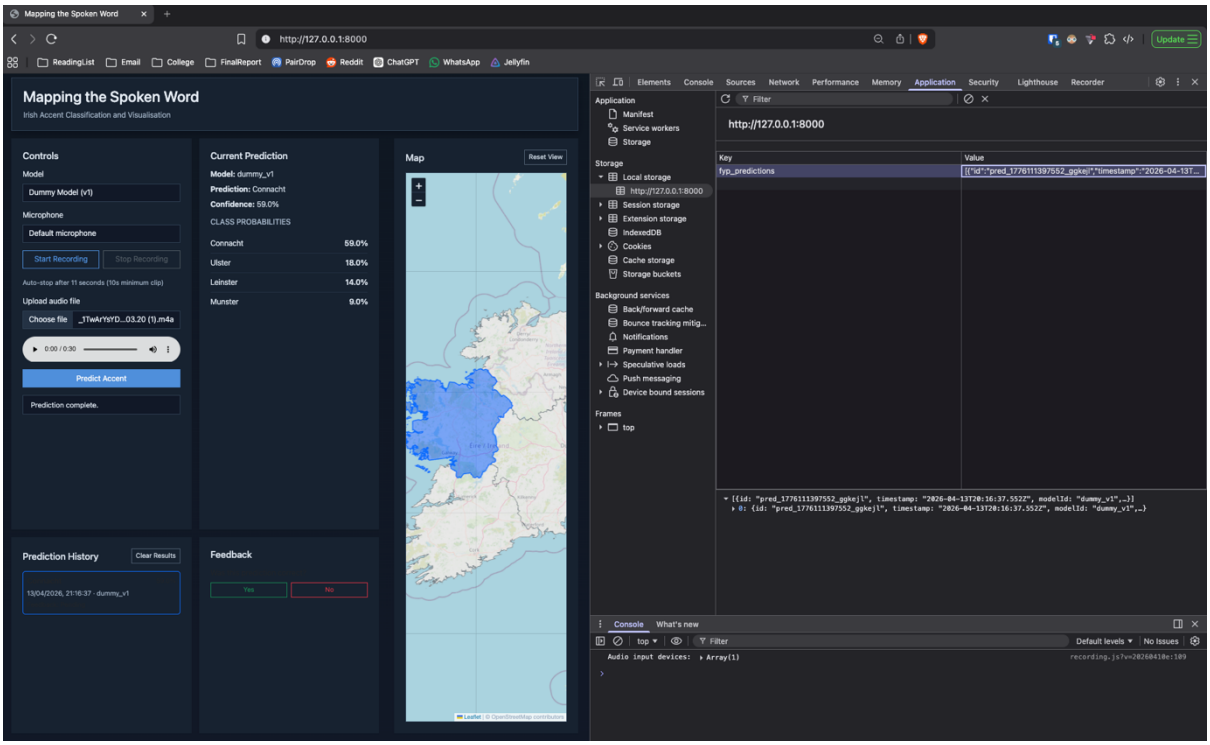


*Figure 26 - A1.21 Example of prediction data stored in browser local storage.*

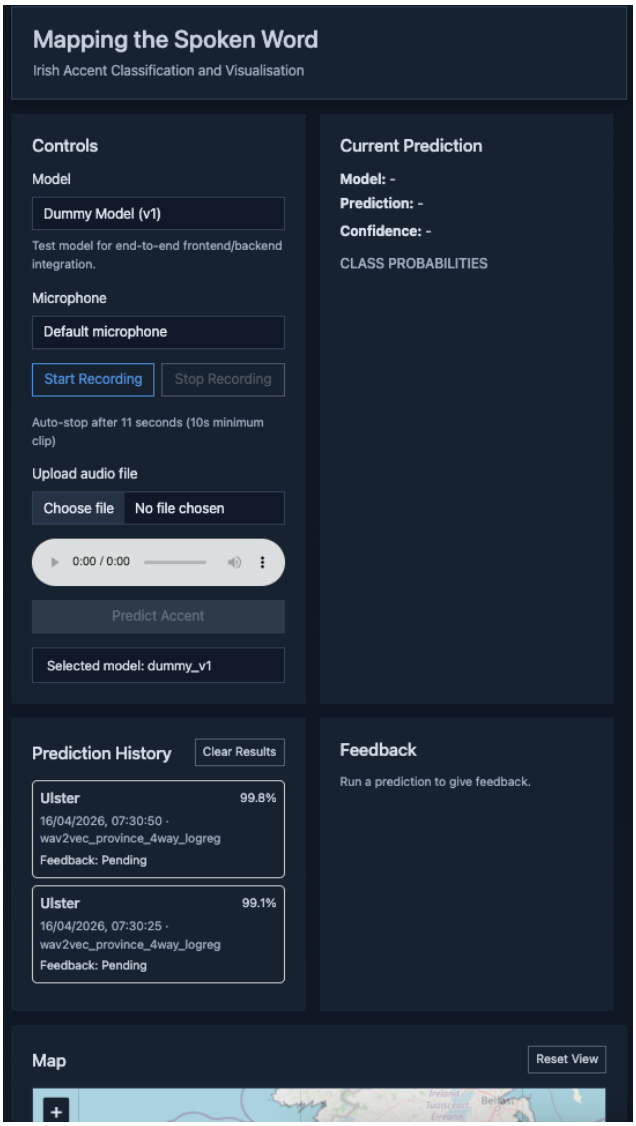


*Figure 27 - A1.22 Initial state on small screen.*

## A.2 – Backend API Testing

This section shows evidence of my backend API testing, showing behaviour of the system endpoints under both valid and invalid input conditions. Testing was performed using Swagger UI and direct HTTP requests for `/api/health`, `/api/models`, and `/api/predict`.

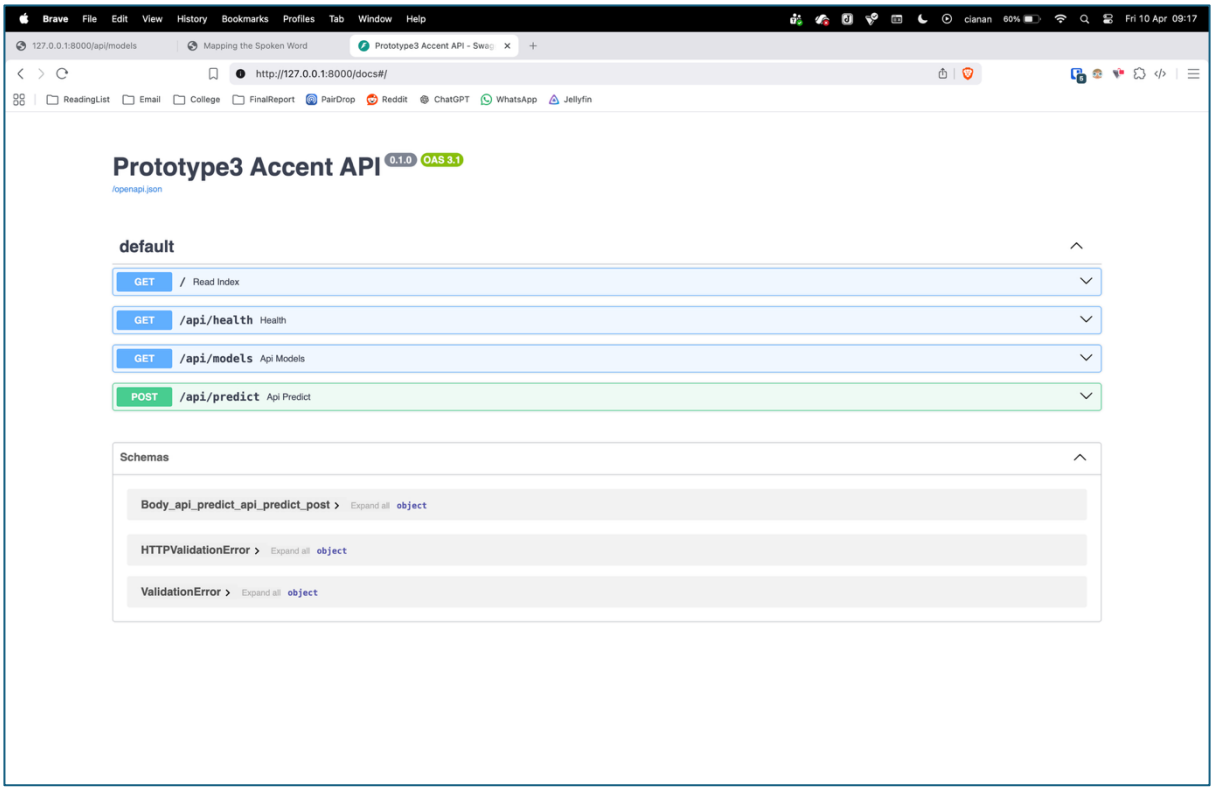


*Figure 28 - A2.01 Swagger API docs interface.*

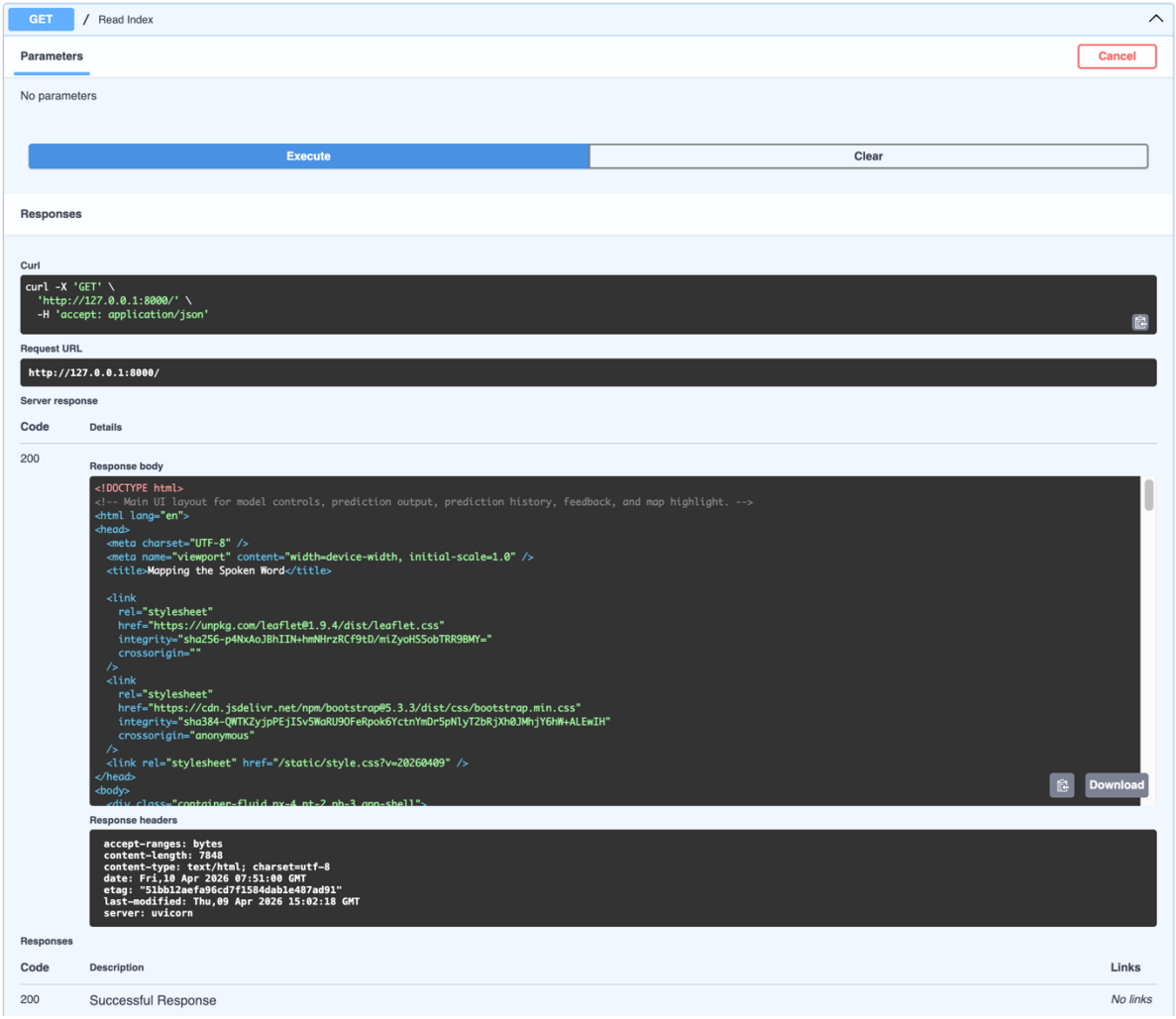


*Figure 29 - A2.02 Successful loading of frontend index route via backend server.*

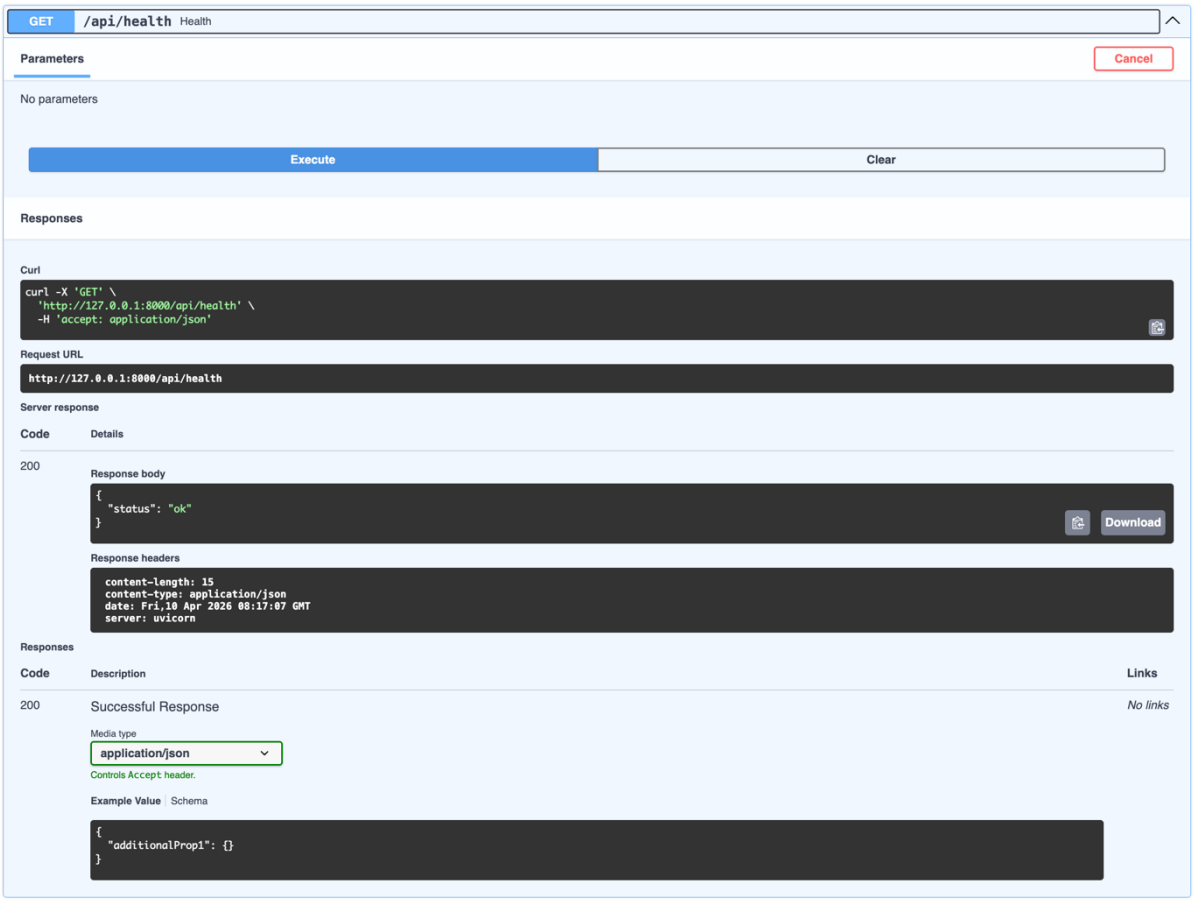


*Figure 30 - A2.03 Successful /api/health endpoint response confirming system availability.*

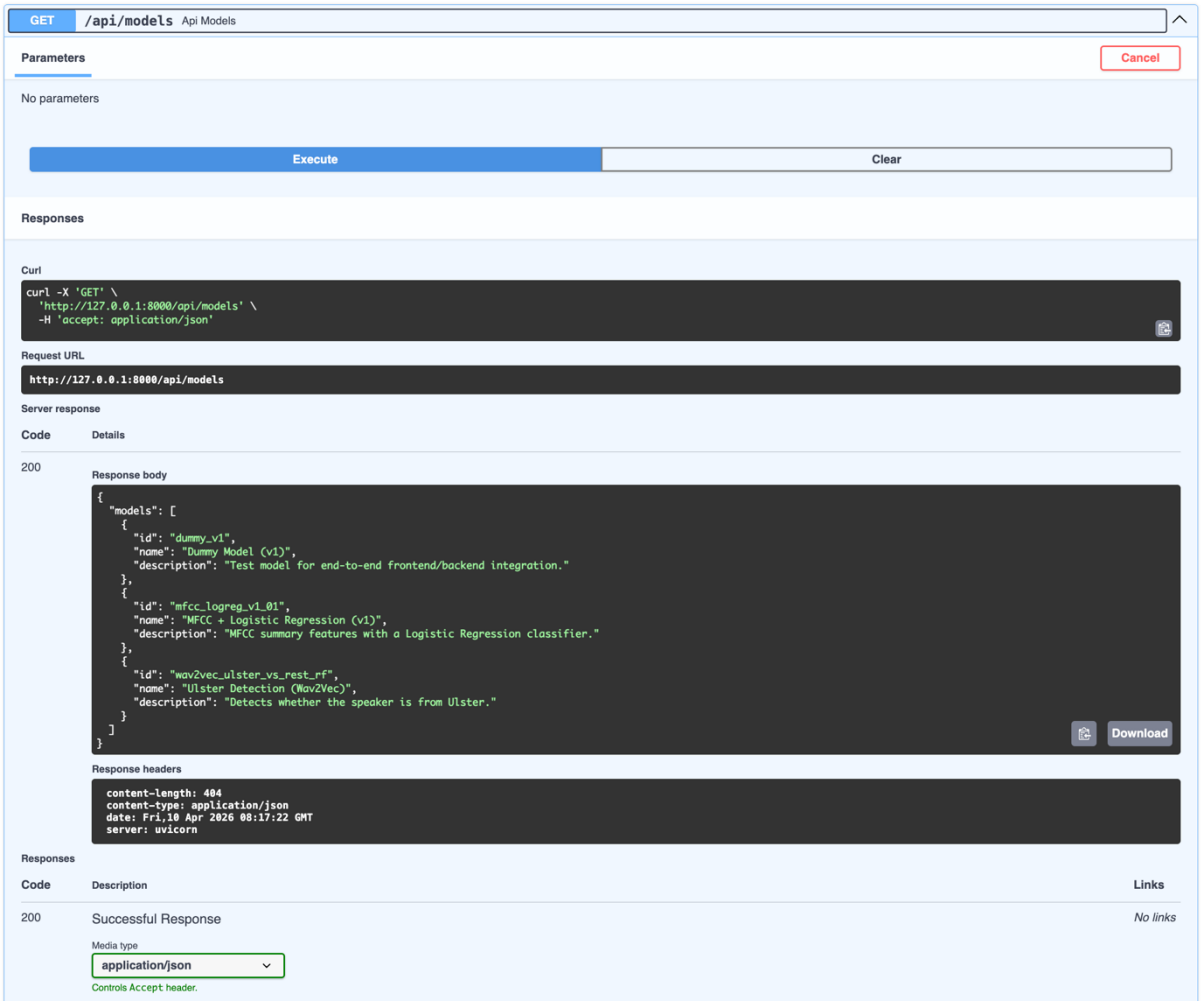


*Figure 31 - A2.04 /api/models endpoint accessed through Swagger UI showing the available model plugins.*


```
{
  "models": [
    {
      "id": "dummy_v1",
      "name": "Dummy Model (v1)",
      "description": "Test model for end-to-end frontend/backend integration."
    },
    {
      "id": "mfcc_logreg_v1_01",
      "name": "MFCC + Logistic Regression (v1)",
      "description": "MFCC summary features with a Logistic Regression classifier."
    },
    {
      "id": "wav2vec_ulster_vs_rest_rf",
      "name": "Ulster Detection (Wav2Vec)",
      "description": "Detects whether the speaker is from Ulster."
    }
  ]
}
```


*Figure 32 - A2.05 /api/models endpoint JSON response listing the registered models*

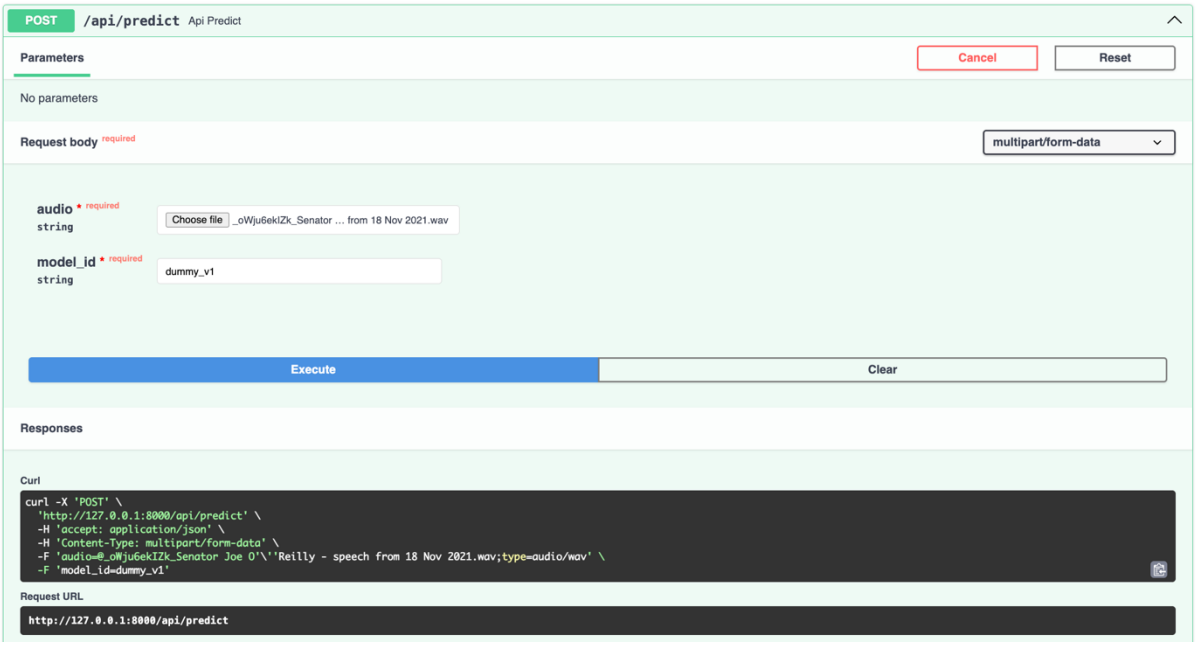


*Figure 33 - A2.06 Swagger interface used to submit a /api/predict POST request with audio input.*

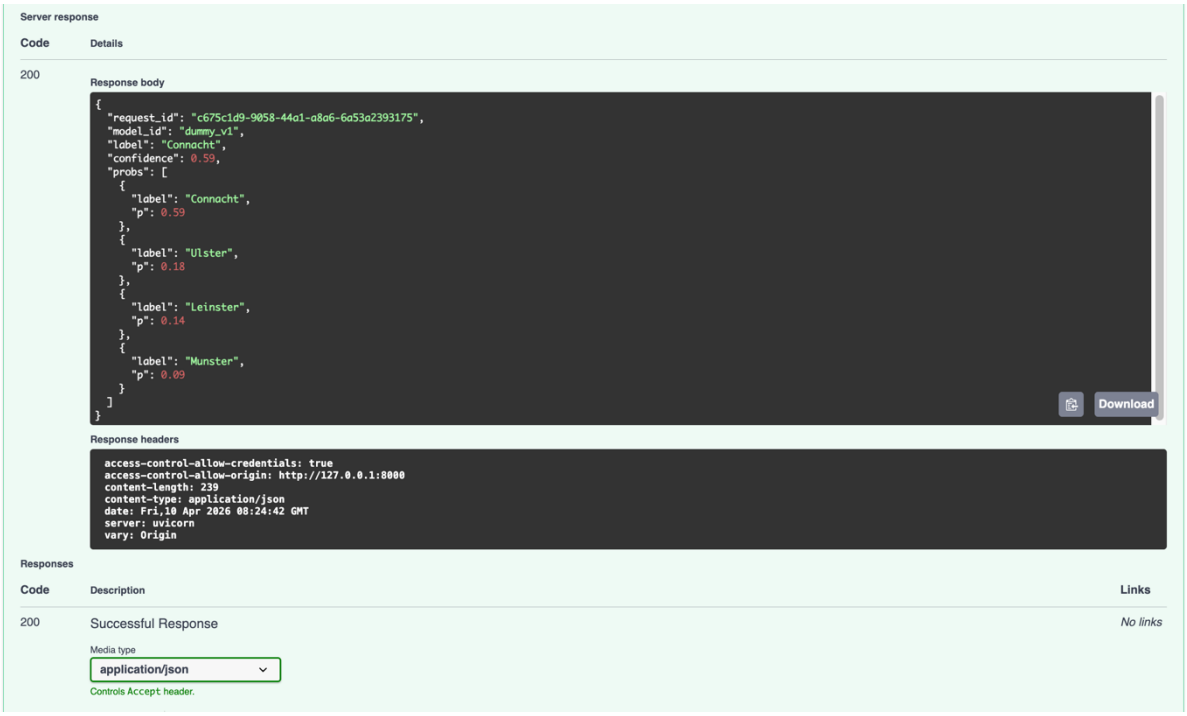


*Figure 34 - A2.07 Successful /api/predict response including predicted label and probability distribution (HTTP 200 OK).*

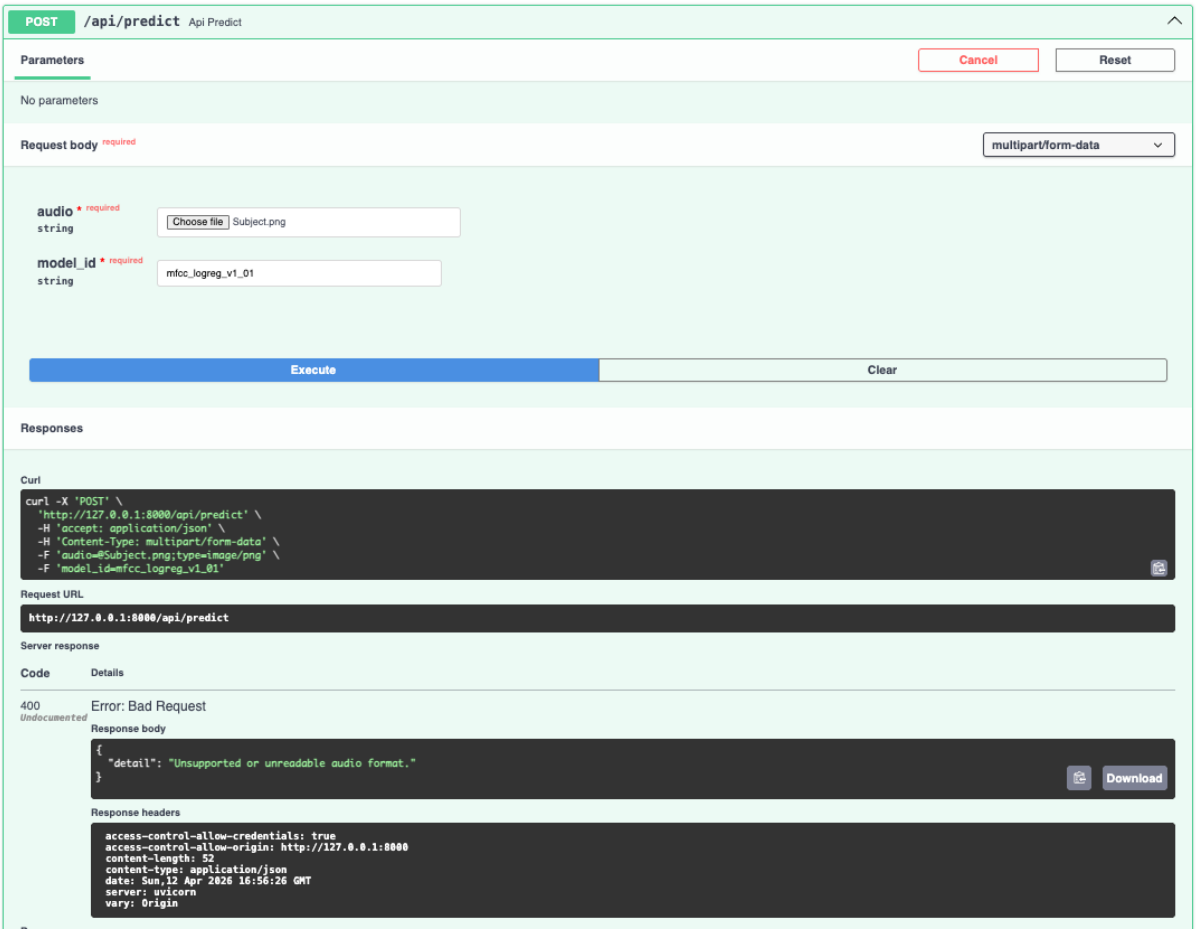


*Figure 35 - A2.08 /api/predict response for invalid or unsupported file type (HTTP 400 Bad Request).*

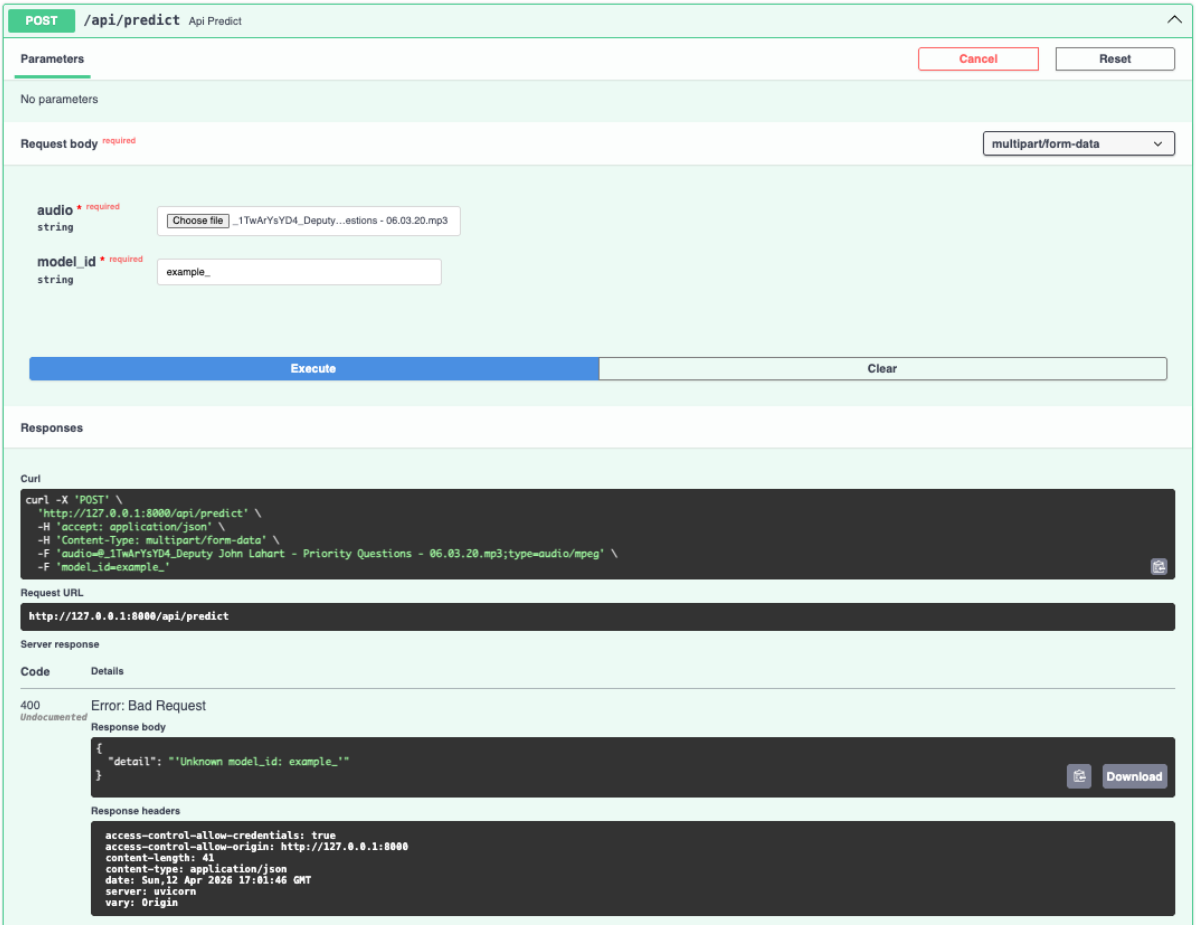


*Figure 36 - A2.09 /api/predict response for invalid or unregistered model_id (HTTP 400 Bad Request).*


*Figure 37 – A2.10* Terminal-based HTTP request demonstrating successful API response.

```
{
  "request_id": "70667842-f353-4417-86be-bc1f43884a34",
  "model_id": "wav2vec_ulster_vs_rest_rf",
  "label": "Rest",
  "confidence": 0.99,
  "probs": [
    {
      "label": "Rest",
      "p": 0.99
    },
    {
      "label": "Ulster",
      "p": 0.01
    }
  ]
}
```


*Figure 38 - A2.11 Successful prediction JSON Response.*

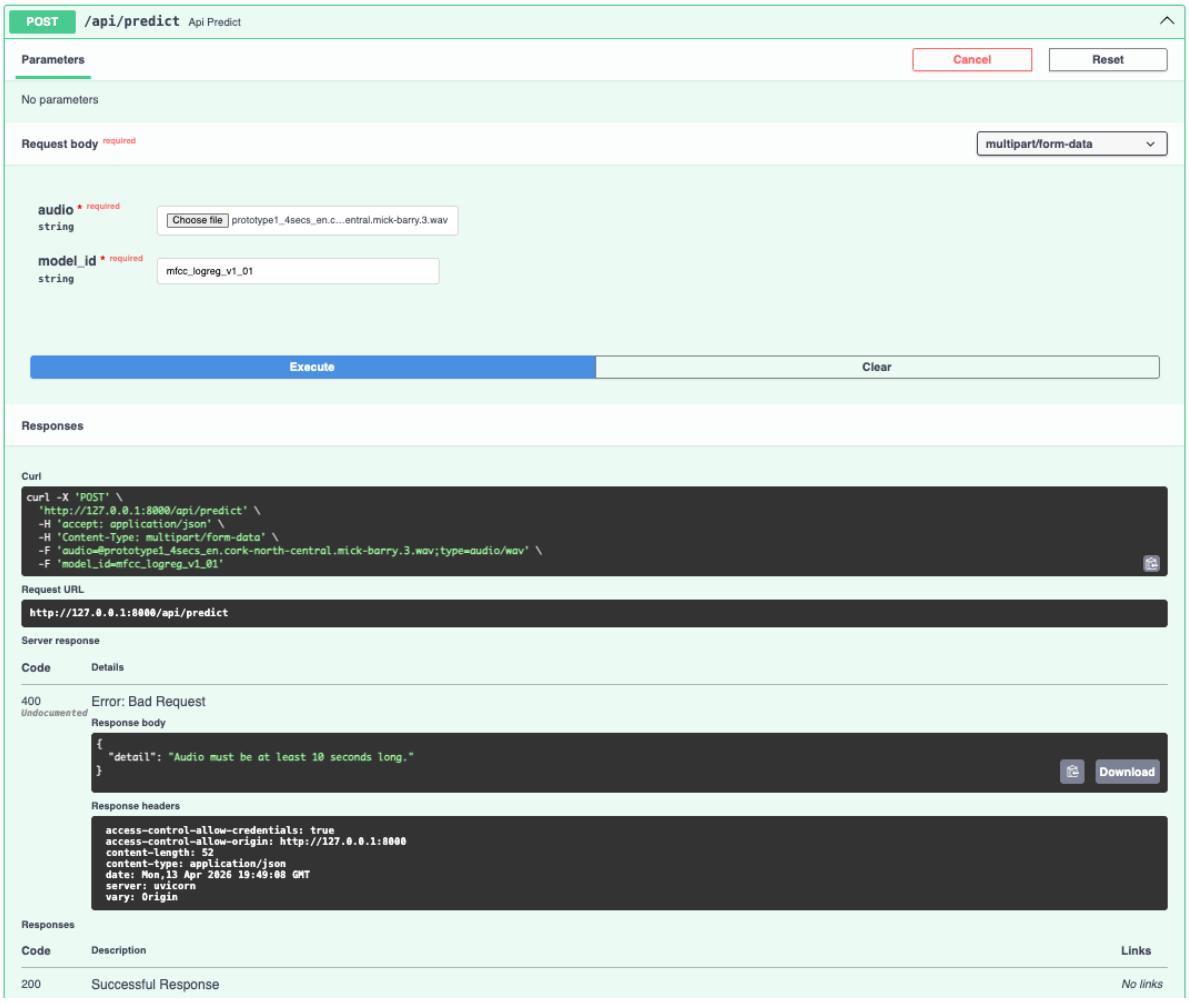


*Figure 39 - A2.12 Swagger /api/predict response rejecting audio shorter than 10 seconds.*

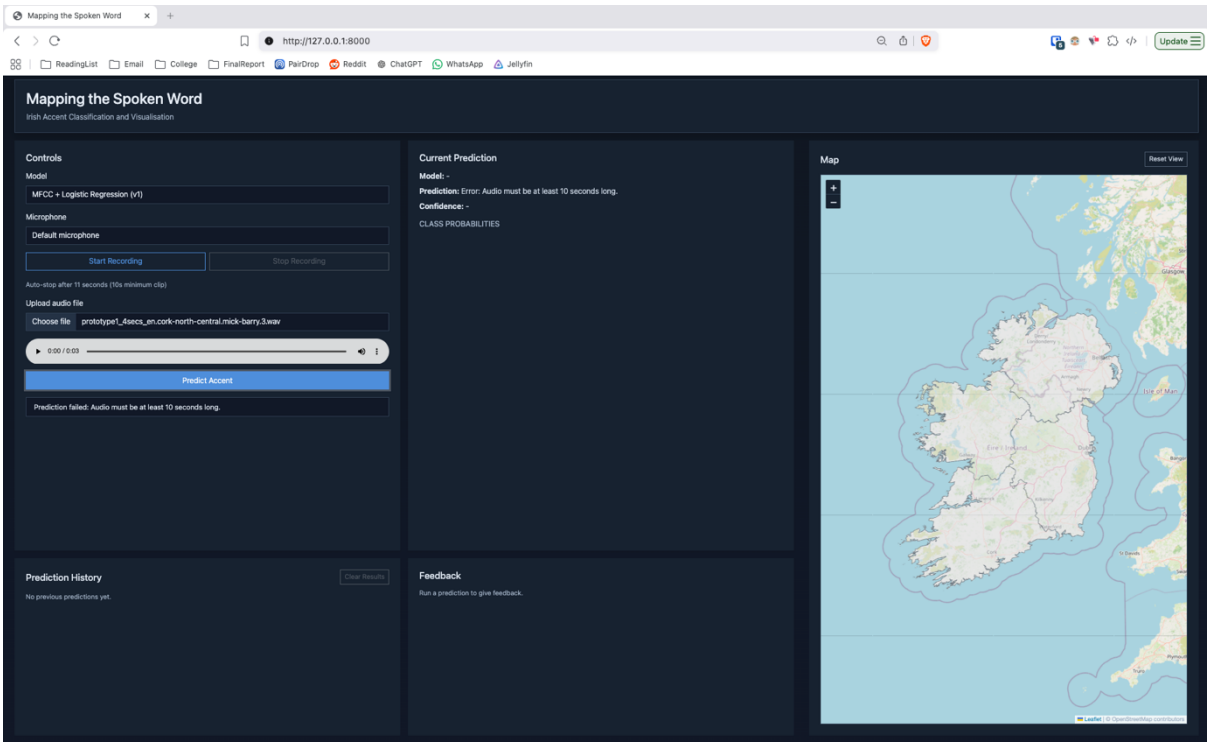


*Figure 40 - A2.13 Frontend validation rejecting uploaded audio shorter than 10 seconds.*

## A.3 - Plugin Architecture Validation (ST-11)

The purpose of this test was to validate the system's plugin-based architecture by demonstrating that models can be added or removed at the backend level without requiring any modifications to the frontend.

This test involved removing a model plugin from the backend by modifying the plugin loading configuration. The system was then restarted to observe the effect across the backend API and frontend interface.

**Before removal of Dummy Model Plugin:**

backend > plugin_loader.py > ...

```
1   from registry import register
2   from plugins.dummy_model import plugin as dummy_plugin
3   from plugins.mfcc_logreg_v1_01 import plugin as mfcc_logreg_plugin
4   from plugins.wav2vec_ulster_vs_rest_rf import Wav2VecUlsterVsRestRF
5
6
7   def load_plugins() -> None:
8       register(dummy_plugin)
9       register(mfcc_logreg_plugin)
10      register(Wav2VecUlsterVsRestRF())
11
```

*Figure 41 – A3.1 backend/plugin_loader.py with Dummy Model Plugin.*

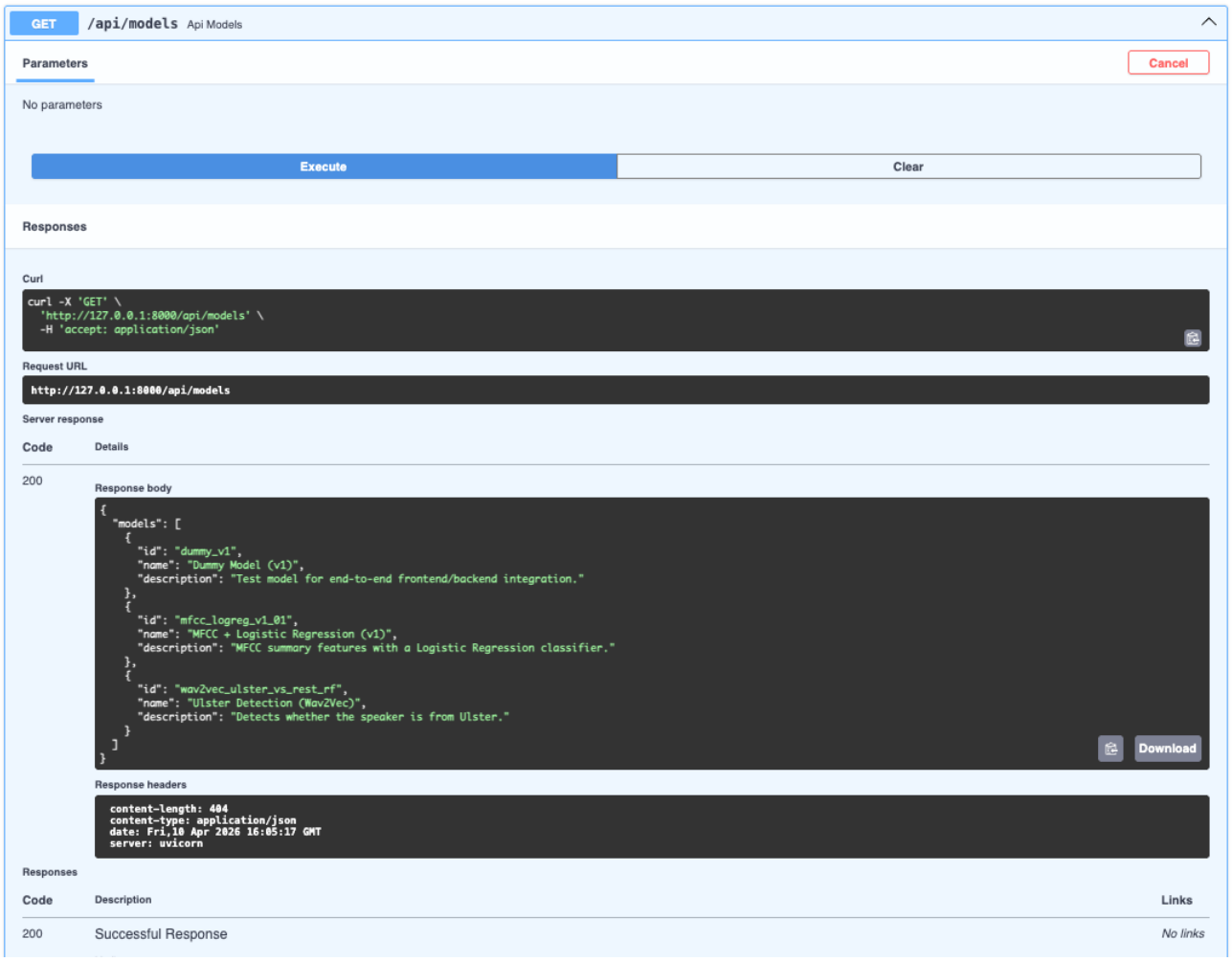


*Figure 42 – A3.2 /api/models with Dummy Model Plugin.*

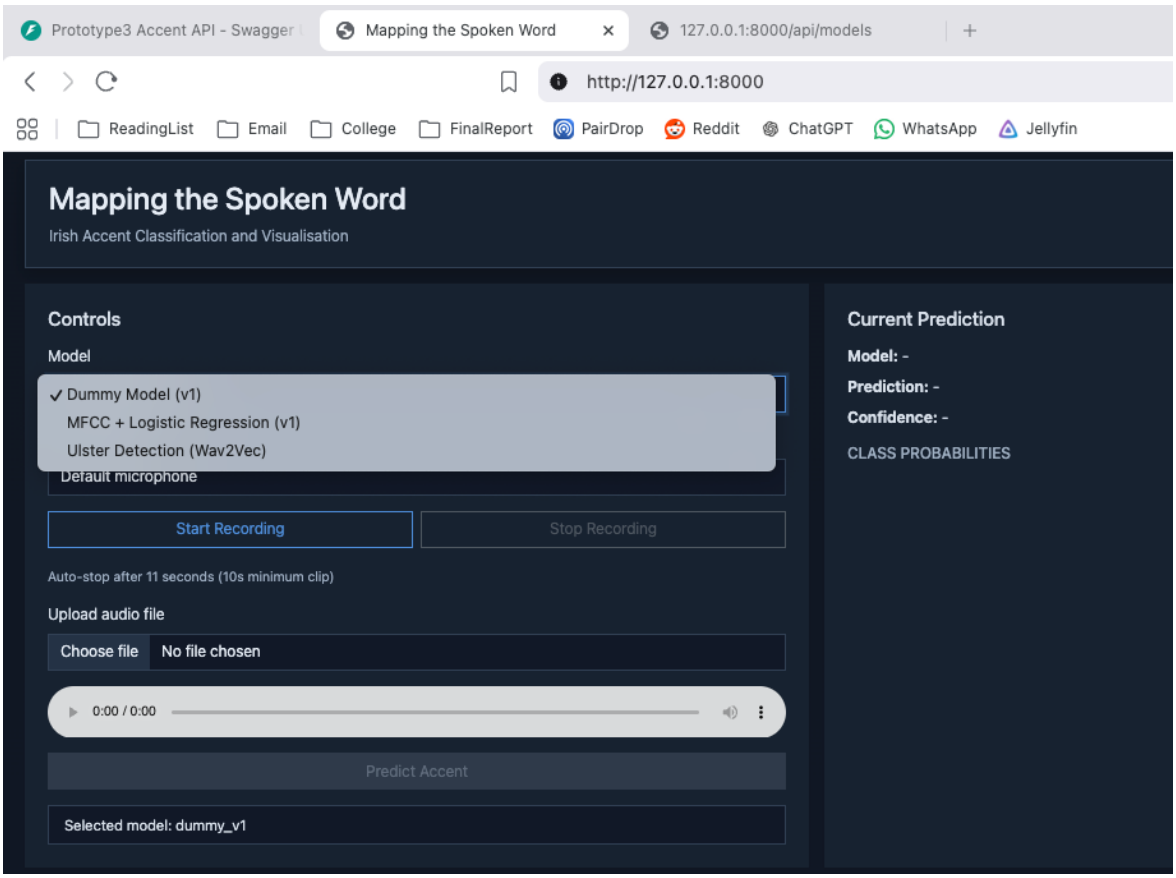


*Figure 43 – A3.3 Frontend with Dummy Model Plugin.*

**After removal of Dummy Model Plugin:**

```
backend > plugin_loader.py > ...
1   from registry import register
2   # from plugins.dummy_model import plugin as dummy_plugin
3   from plugins.mfcc_logreg_v1_01 import plugin as mfcc_logreg_plugin
4   from plugins.wav2vec_ulster_vs_rest_rf import Wav2VecUlsterVsRestRF
5
6
7   def load_plugins() -> None:
8   #    register(dummy_plugin)
9       register(mfcc_logreg_plugin)
10      register(Wav2VecUlsterVsRestRF())
11
```

*Figure 44 – A3.4 backend/plugin_loader.py with Dummy Model Plugin removed.*

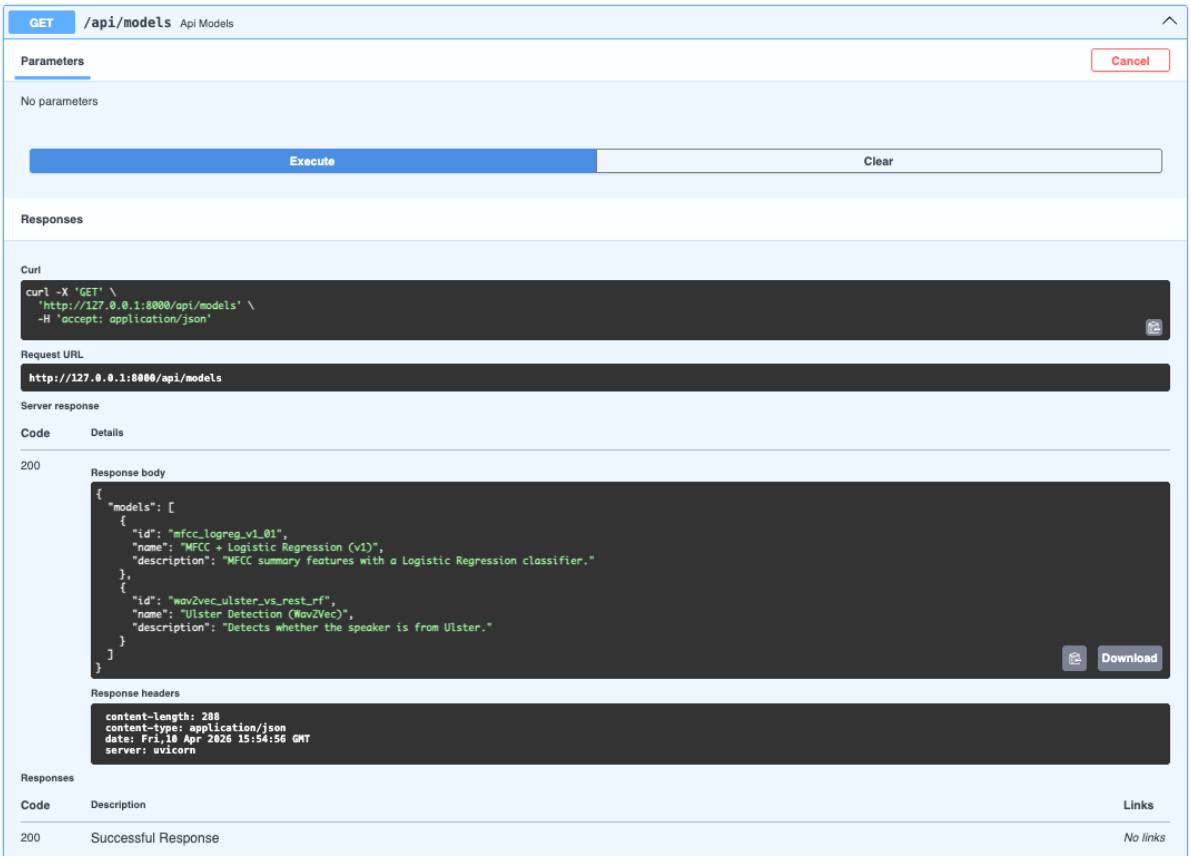


*Figure 45 – A3.5 /api/models with Dummy Model Plugin removed.*

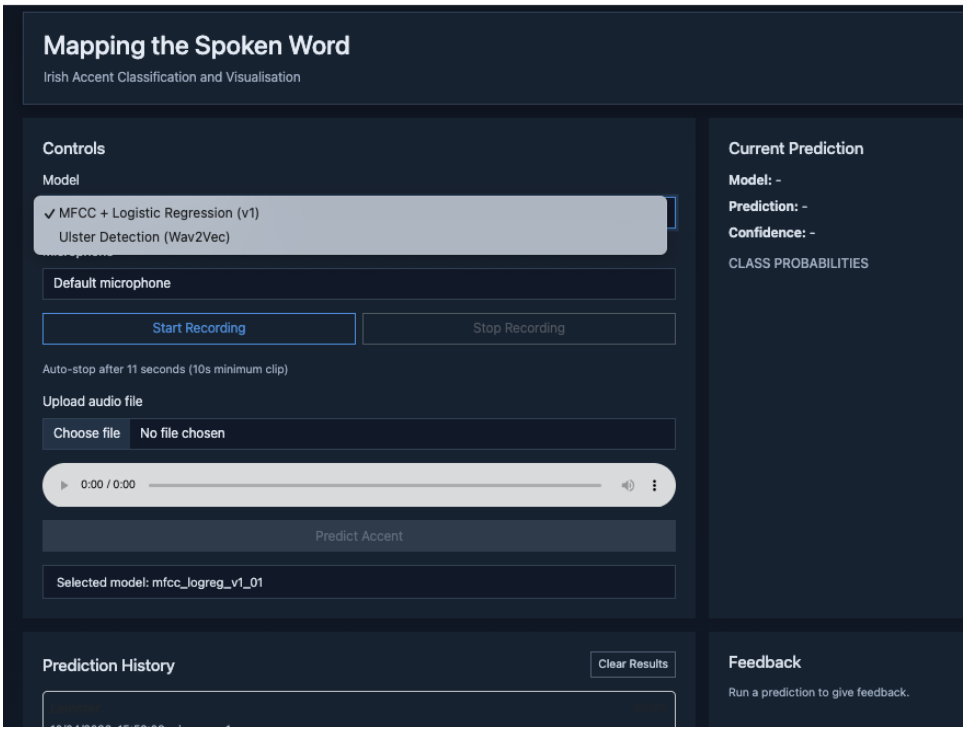


*Figure 46 – A3.6 Frontend with Dummy Model Plugin removed.*

# Appendix B: Experiment Results and Evaluation Charts

This appendix contains the experimental results used to evaluate model performance across different feature extraction approaches and classification tasks.

## B1. Irish MFCC Results

### B1.1 Balanced Accuracy

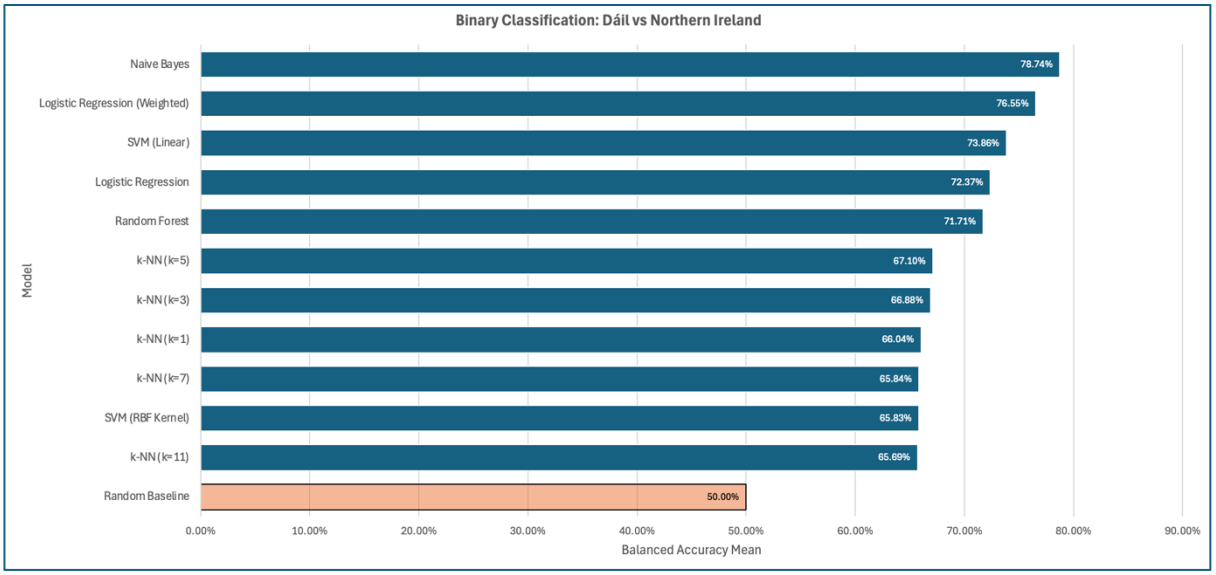


*Figure 47 – B1.1.1 Balanced Accuracy: Dáil vs NI Datasets.*

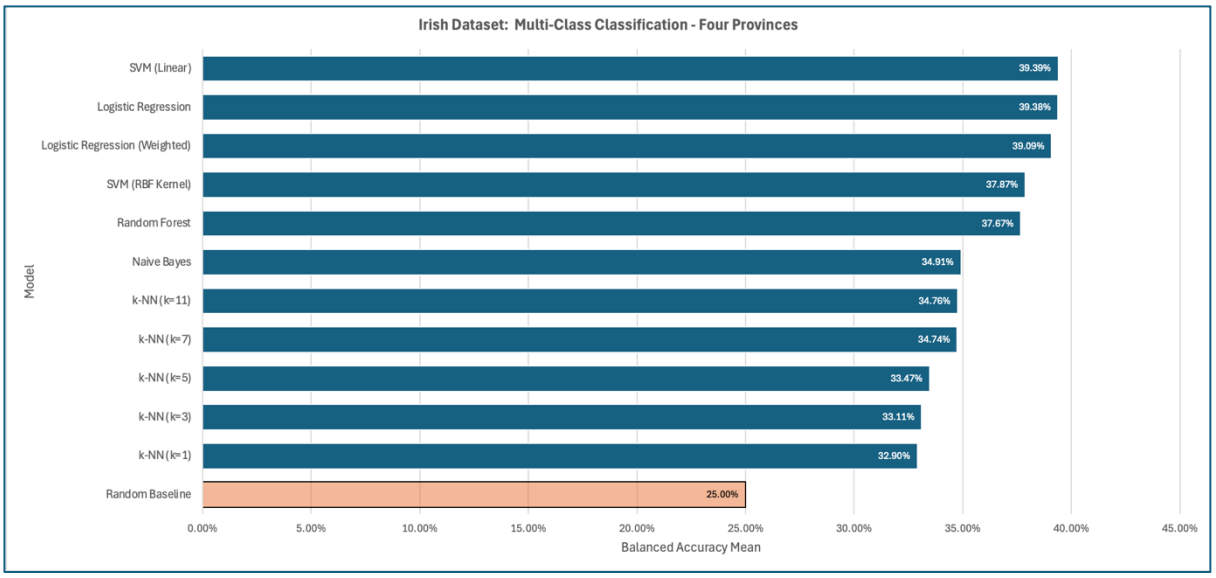


*Figure 48 – B1.1.2 Balanced Accuracy: Four Provinces (Dáil and NI datasets).*

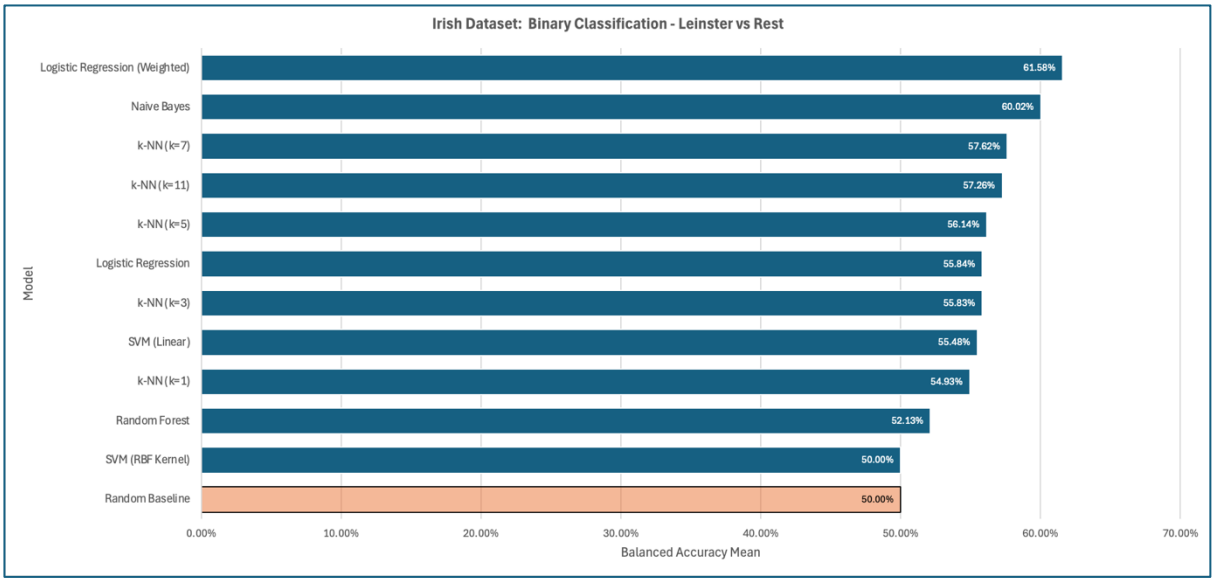


*Figure 49 – B1.1.3 Balanced Accuracy: Leinster vs Rest.*

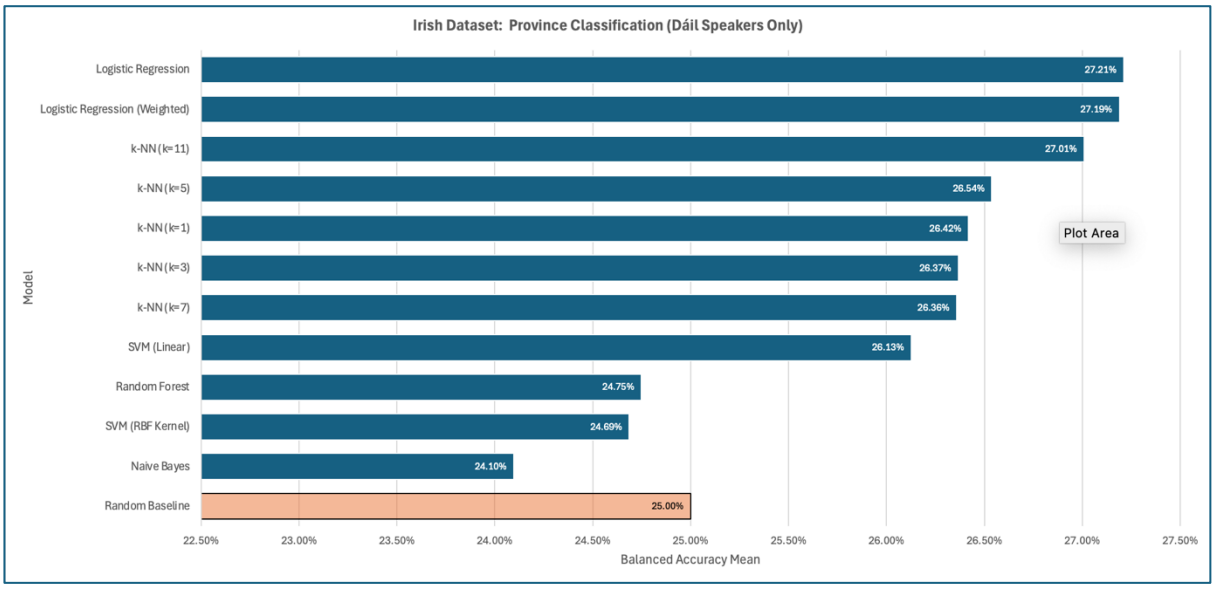


*Figure 50 – B1.1.4 Balanced Accuracy: Four Provinces (Dáil dataset only).*

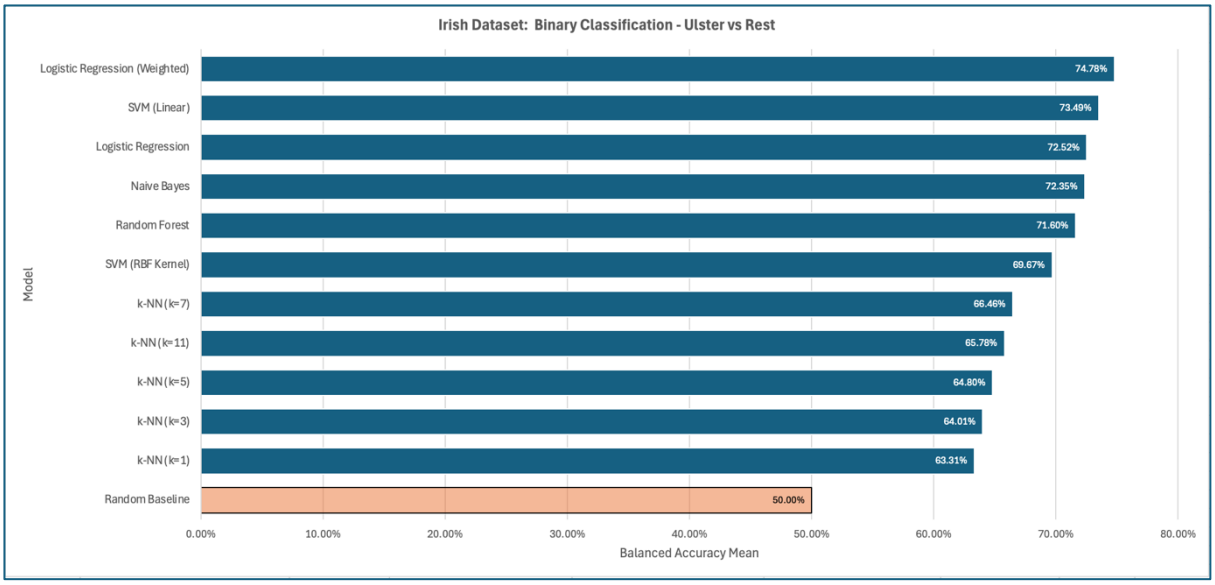


*Figure 51 – B1.1.5 Balanced Accuracy: Ulster vs Rest*

## B1.2 Best Model Per Task

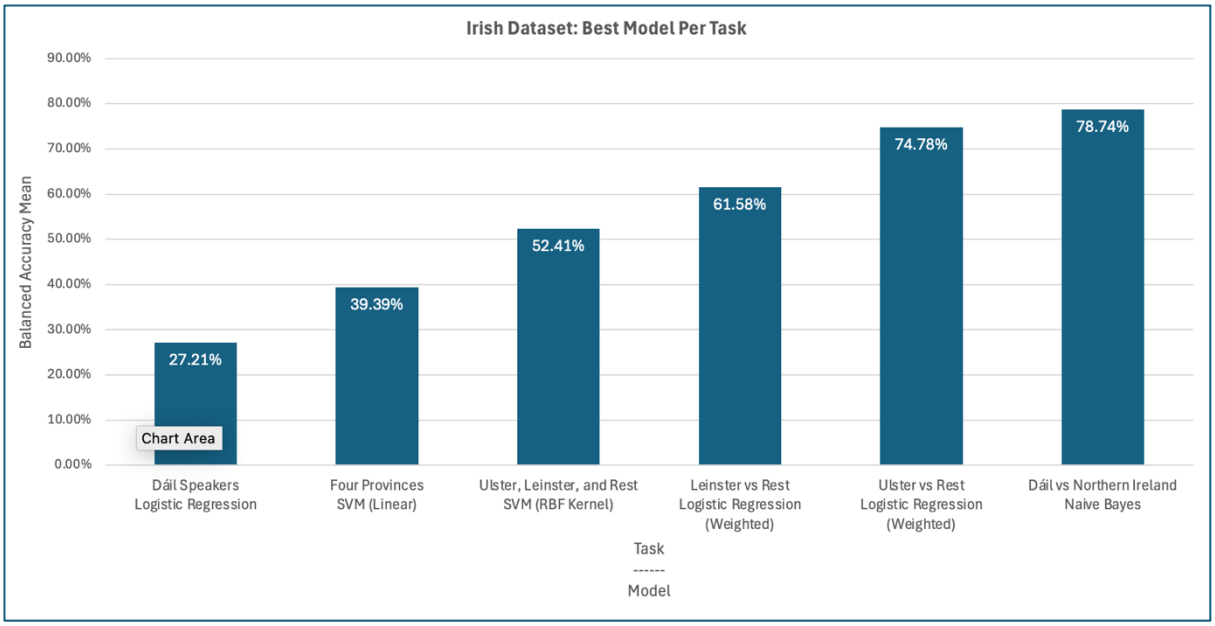


*Figure 52 - B1.2.1 Best Model Per Task*

### B1.3 Balanced Accuracy Per Task

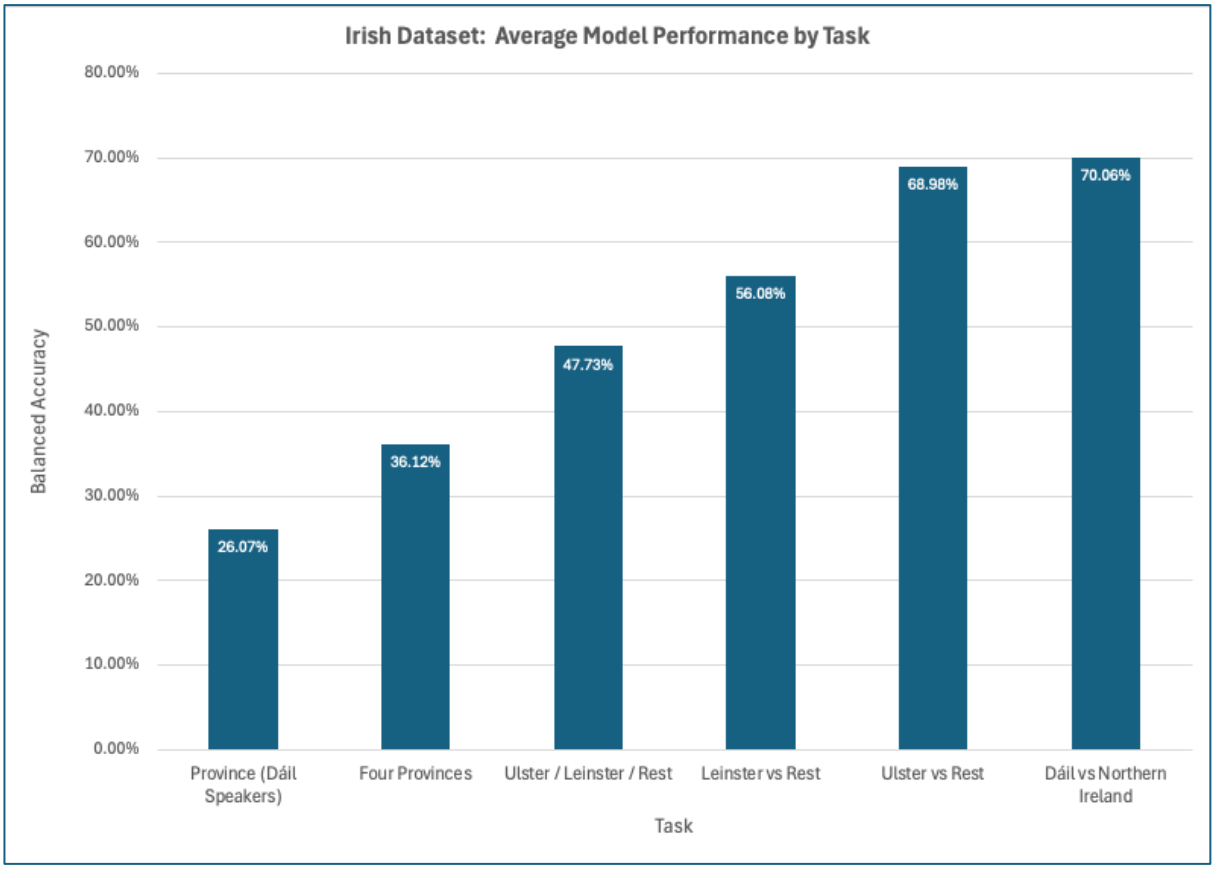


*Figure 53 - B1.3.1 Balanced Accuracy Per Task*

### B1.4 Average Improvement over the Baseline by Model

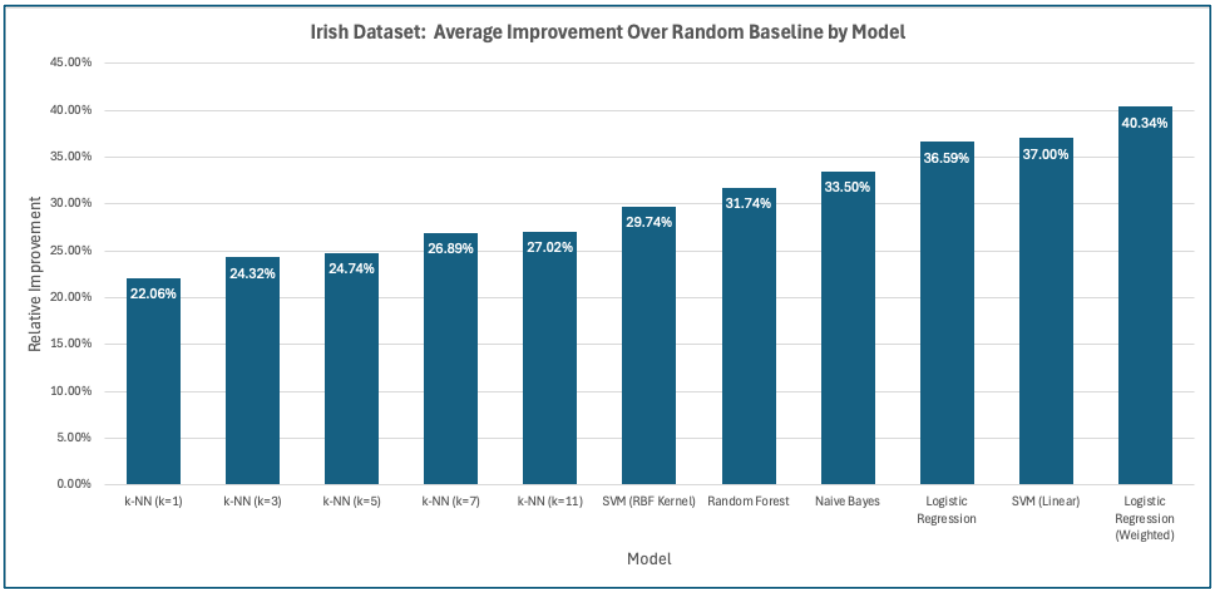


*Figure 54 – B1.4.1 Average Improvement over the Baseline by Model*

## B2. Irish Wav2Vec Results

### B2.1 Balanced Accuracy

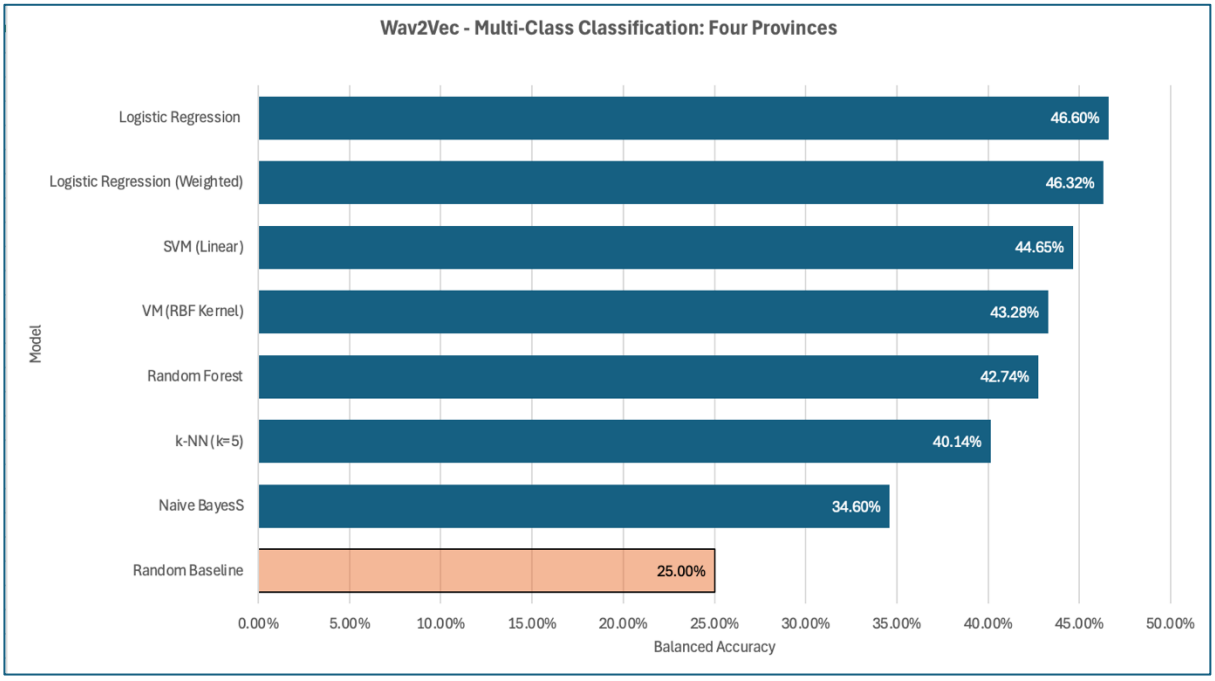


*Figure 55 – B2.1.1 Balanced Accuracy: Four Provinces (Dáil and NI datasets).*

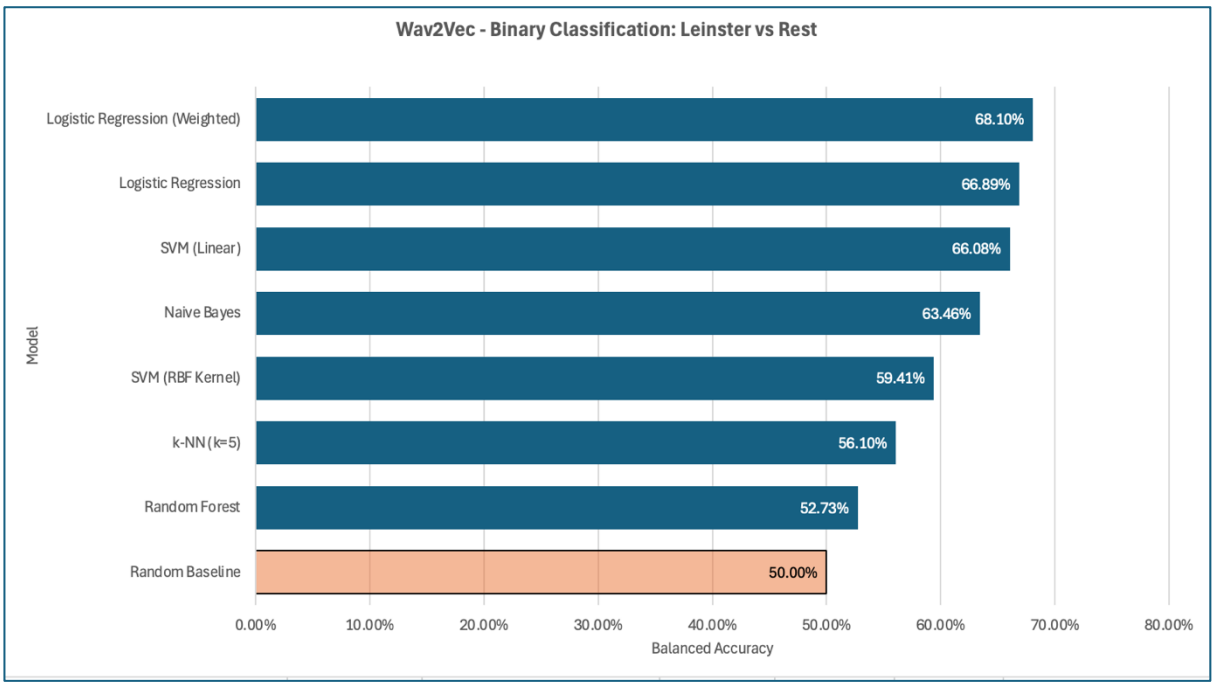


*Figure 56 – B2.1.2 Balanced Accuracy: Leinster vs Rest.*

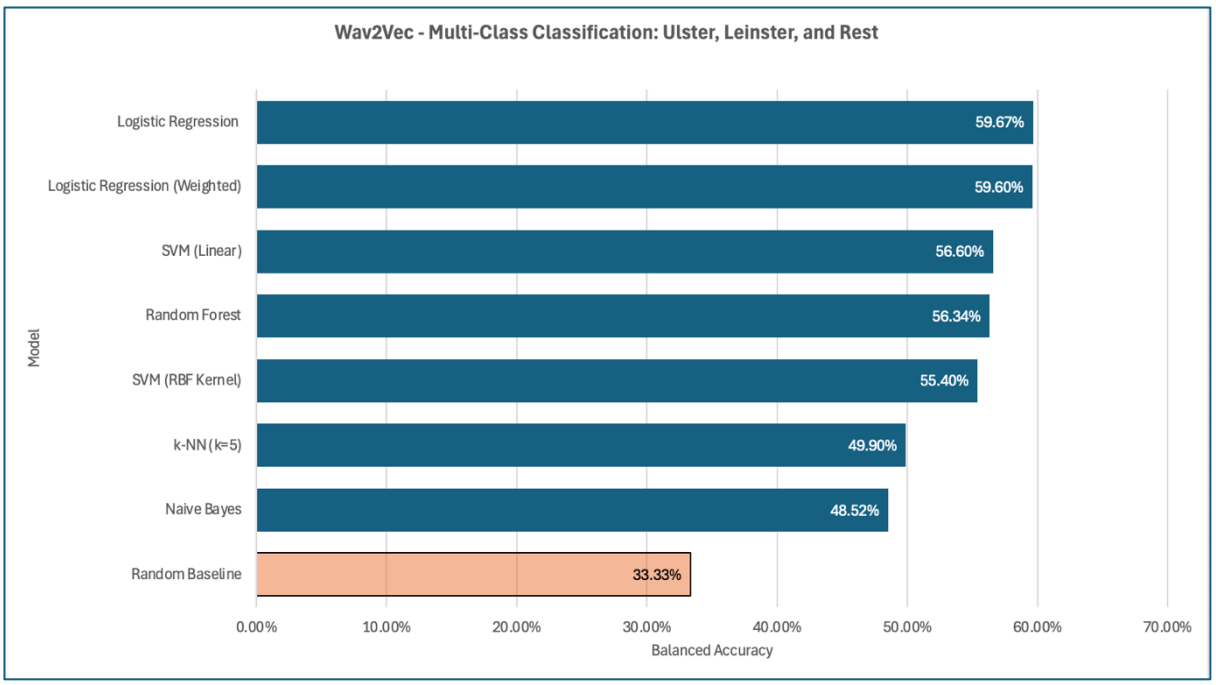


*Figure 57 – B2.1.3 Balanced Accuracy: Ulster, Leinster vs Rest.*

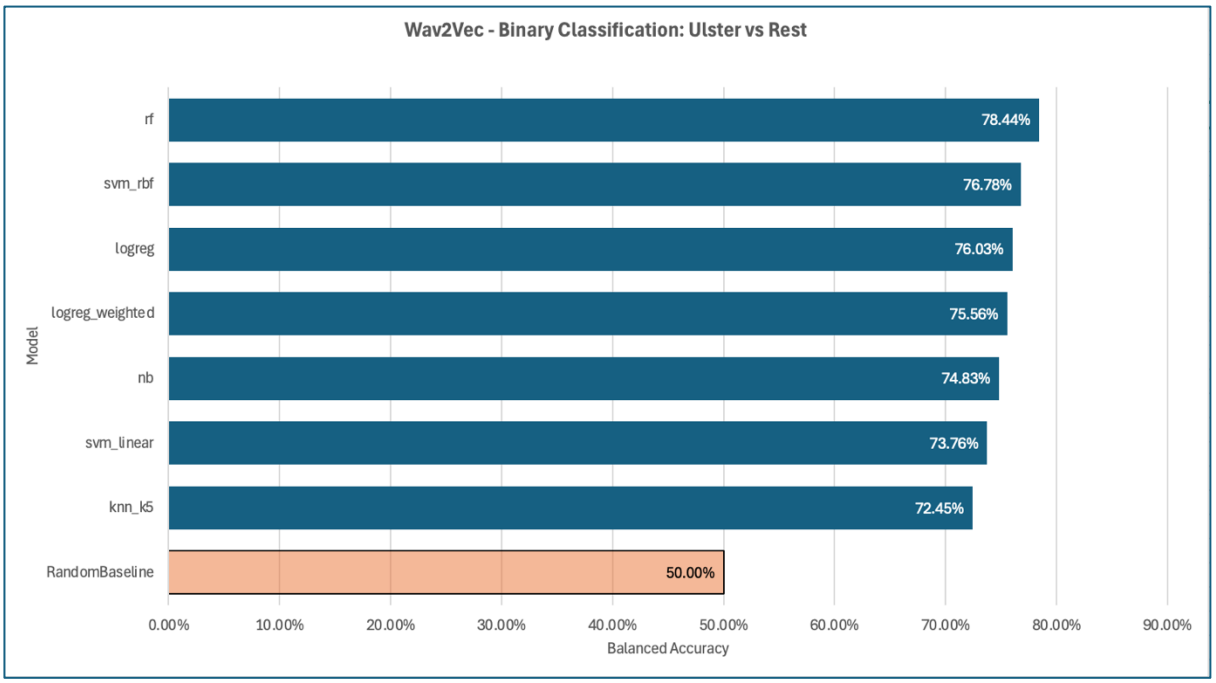


*Figure 58 – B2.1.4 Balanced Accuracy: Ulster vs Rest.*

## B2.2 Macro F1 Per Task

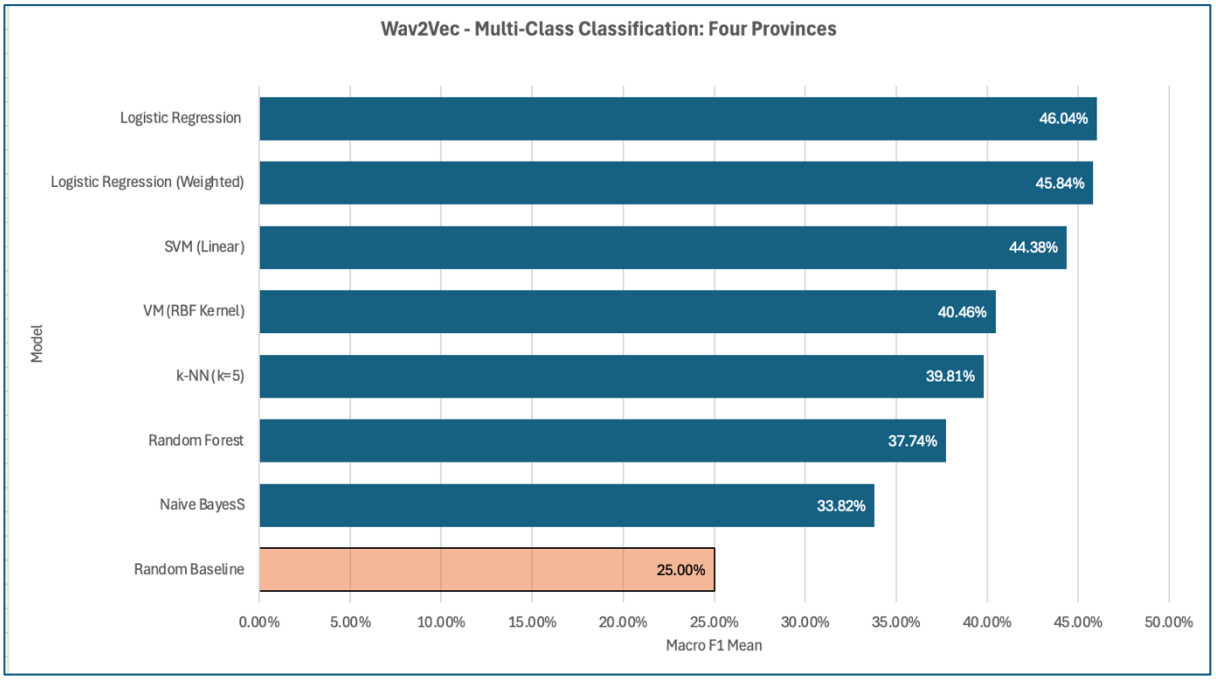


*Figure 59 - B2.2.1 Macro F1: Four Provinces (Dáil and NI datasets).*

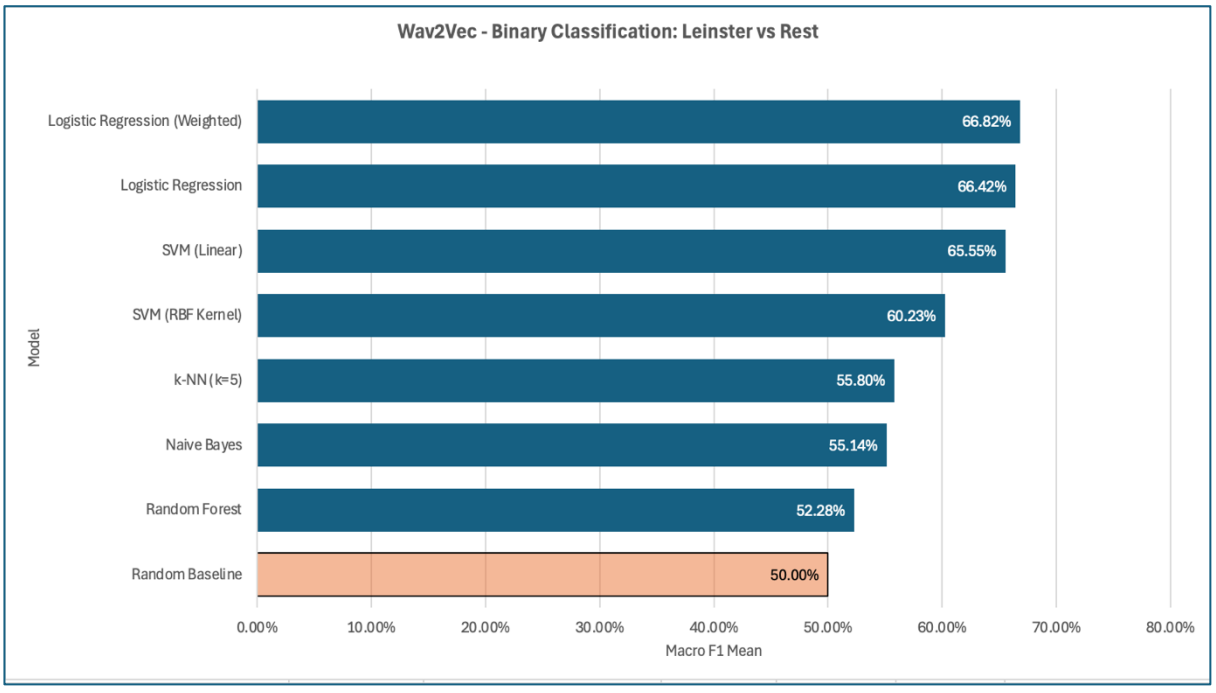


*Figure 60 - B2.2.2 Macro F1: Leinster vs Rest.*

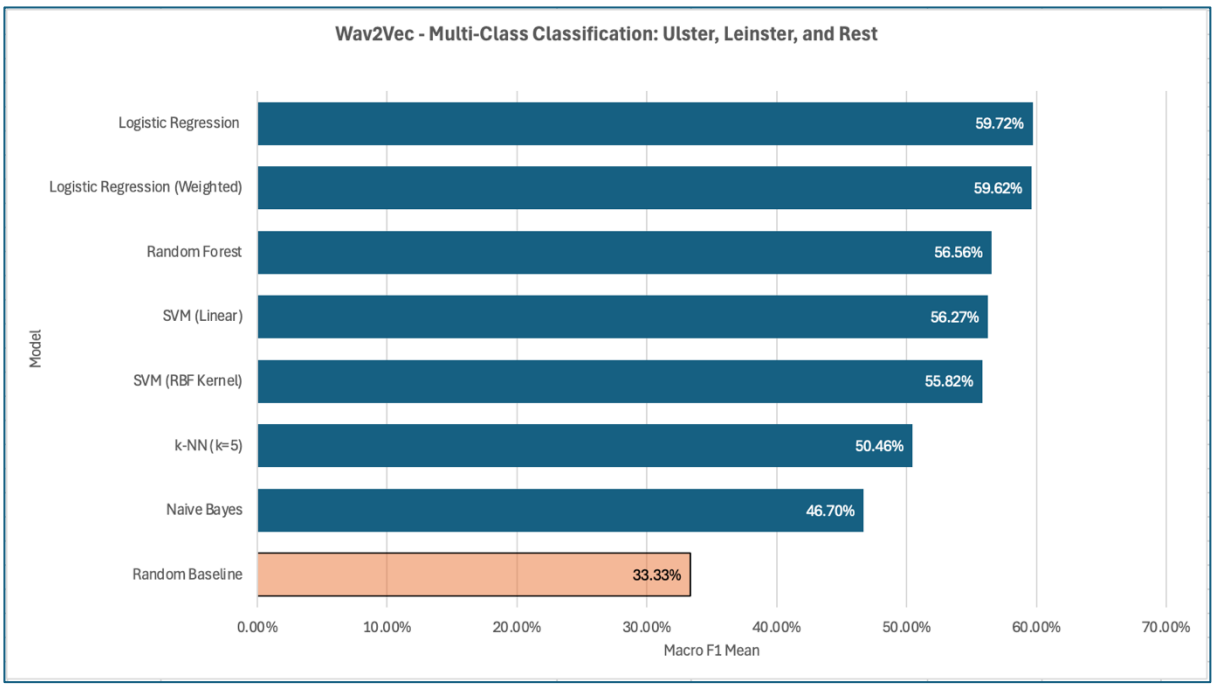


*Figure 61 - B2.2.3 Macro F1: Ulster, Leinster vs Rest.*

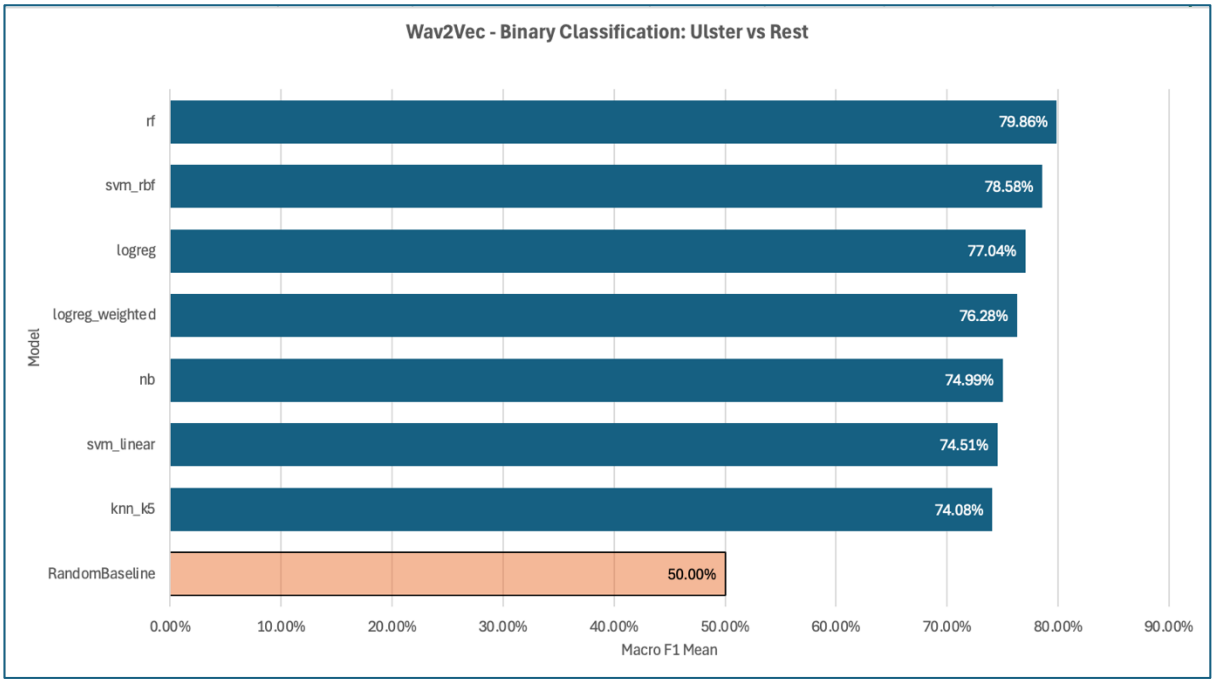


*Figure 62 - B2.2.4 Macro F1: Ulster vs Rest.*

### B2.3 Averaged Accuracy vs Averaged Balanced Accuracy per Task

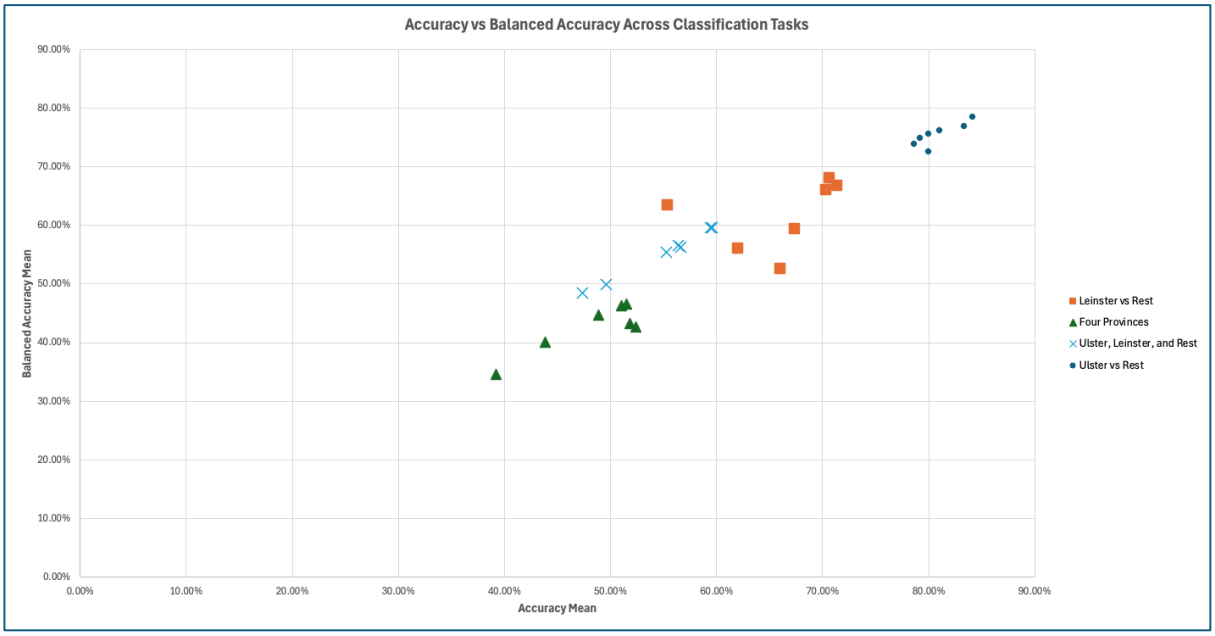


*Figure 63 - B2.3.1 Averaged Accuracy vs Averaged Balanced Accuracy: All Tasks.*

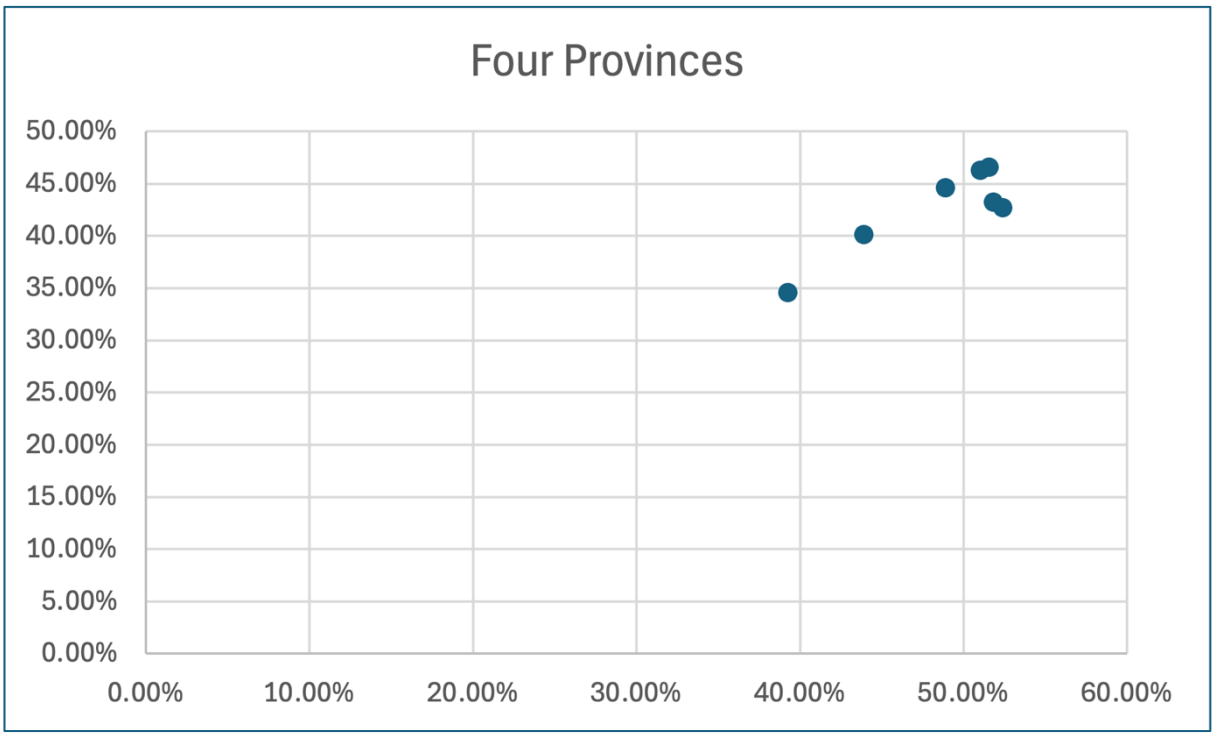


*Figure 64 - B2.3.2 Averaged Accuracy vs Averaged Balanced Accuracy: Four Provinces (Dáil and NI datasets).*

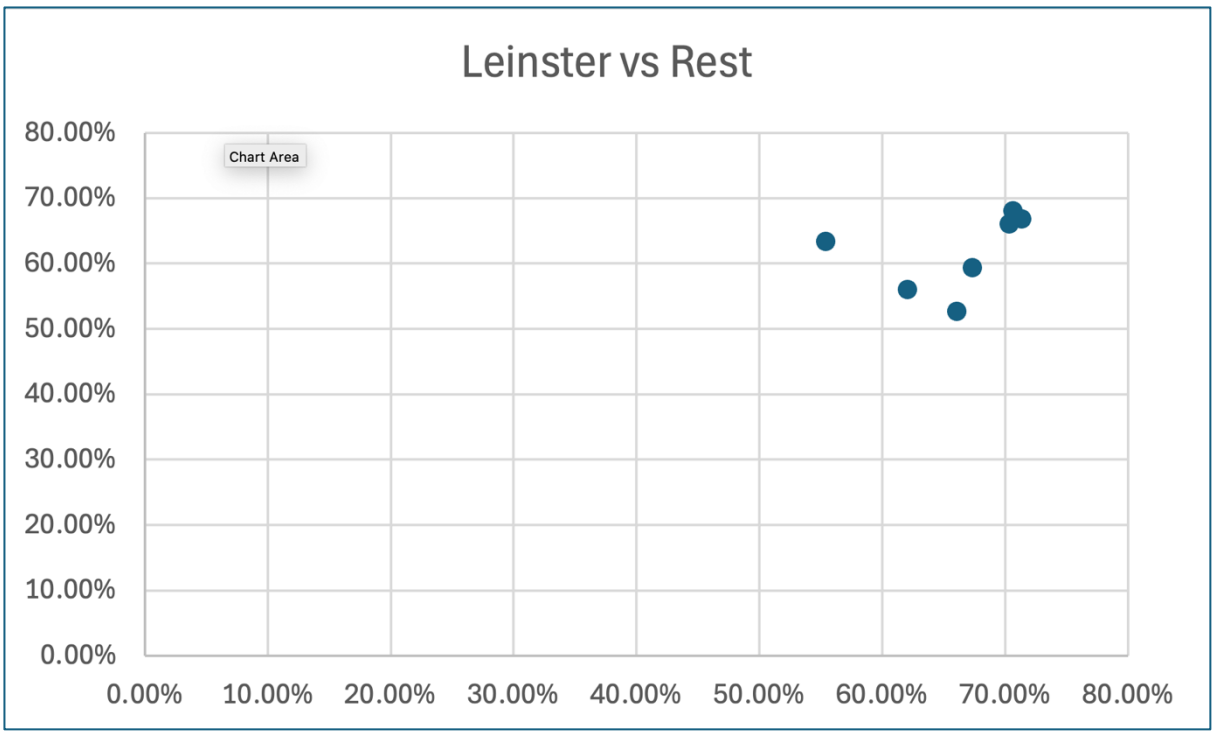


*Figure 65 - B2.3.3 Averaged Accuracy vs Averaged Balanced Accuracy: Leinster vs Rest.*

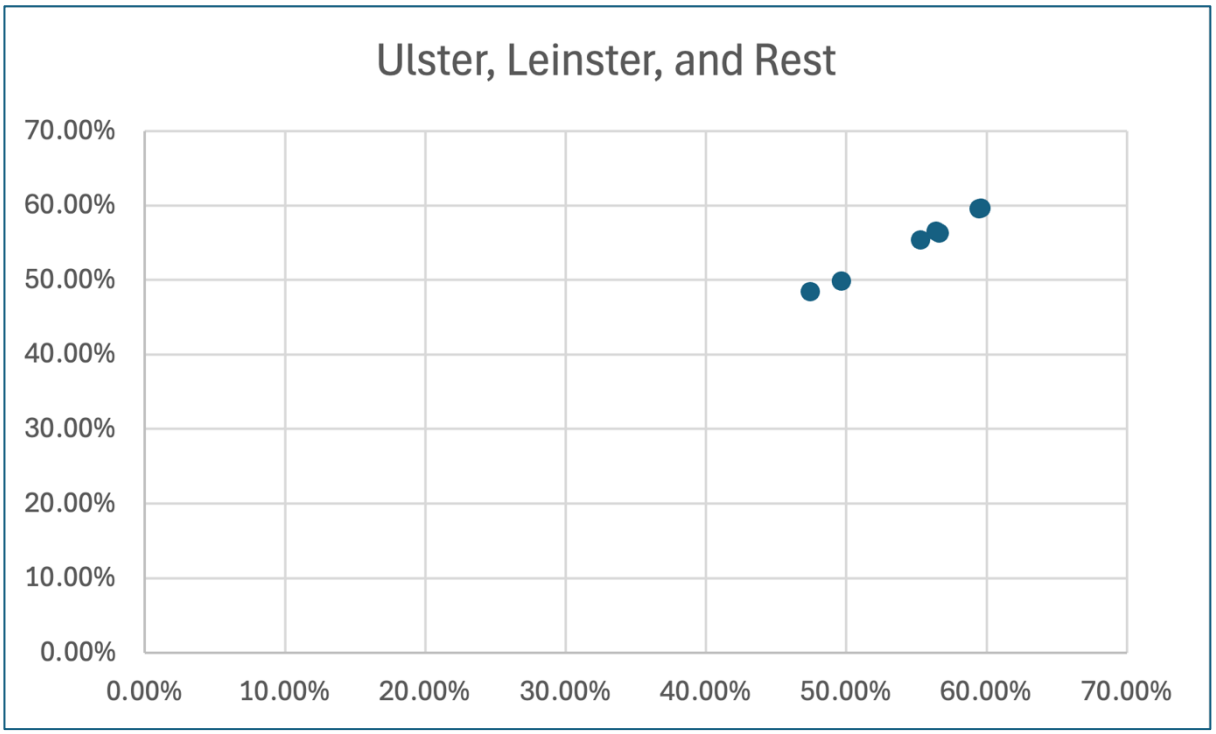


*Figure 66 - B2.3.4 Averaged Accuracy vs Averaged Balanced Accuracy: Ulster, Leinster vs Rest.*

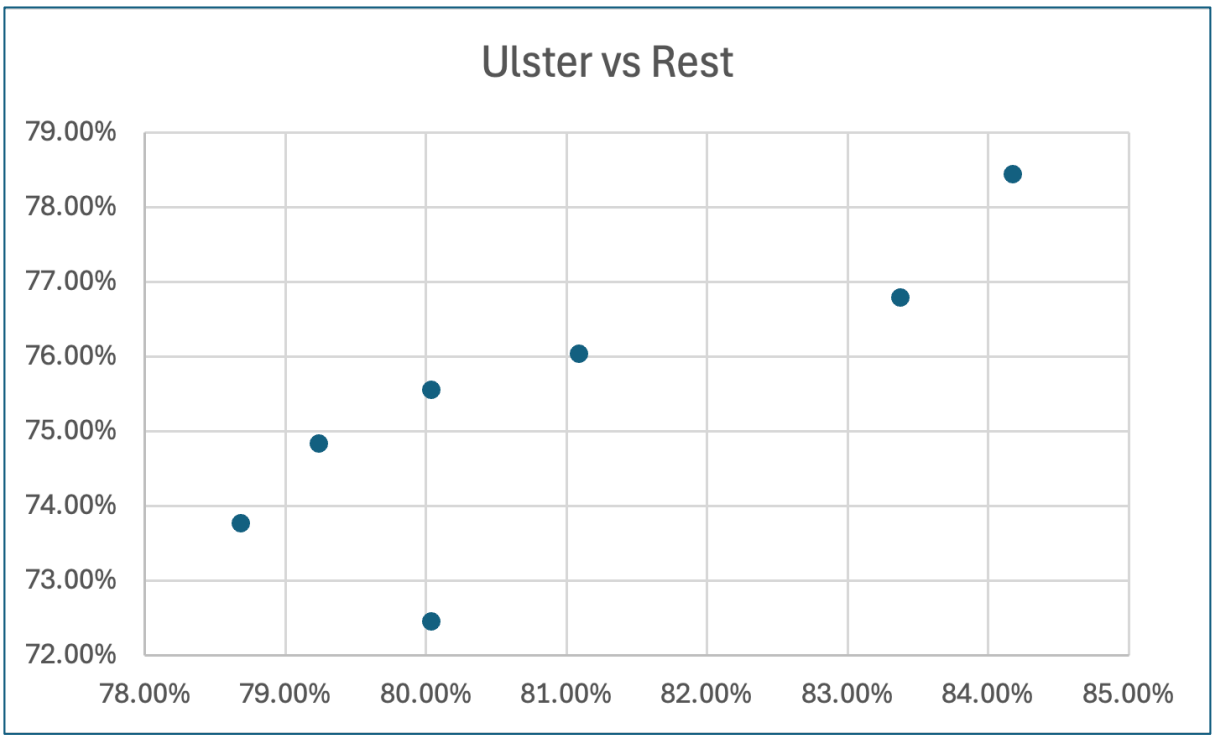


*Figure 67 - B2.3.5 Averaged Accuracy vs Averaged Balanced Accuracy: Ulster vs Rest.*

### B2.4 Average Balanced Accuracy per Task

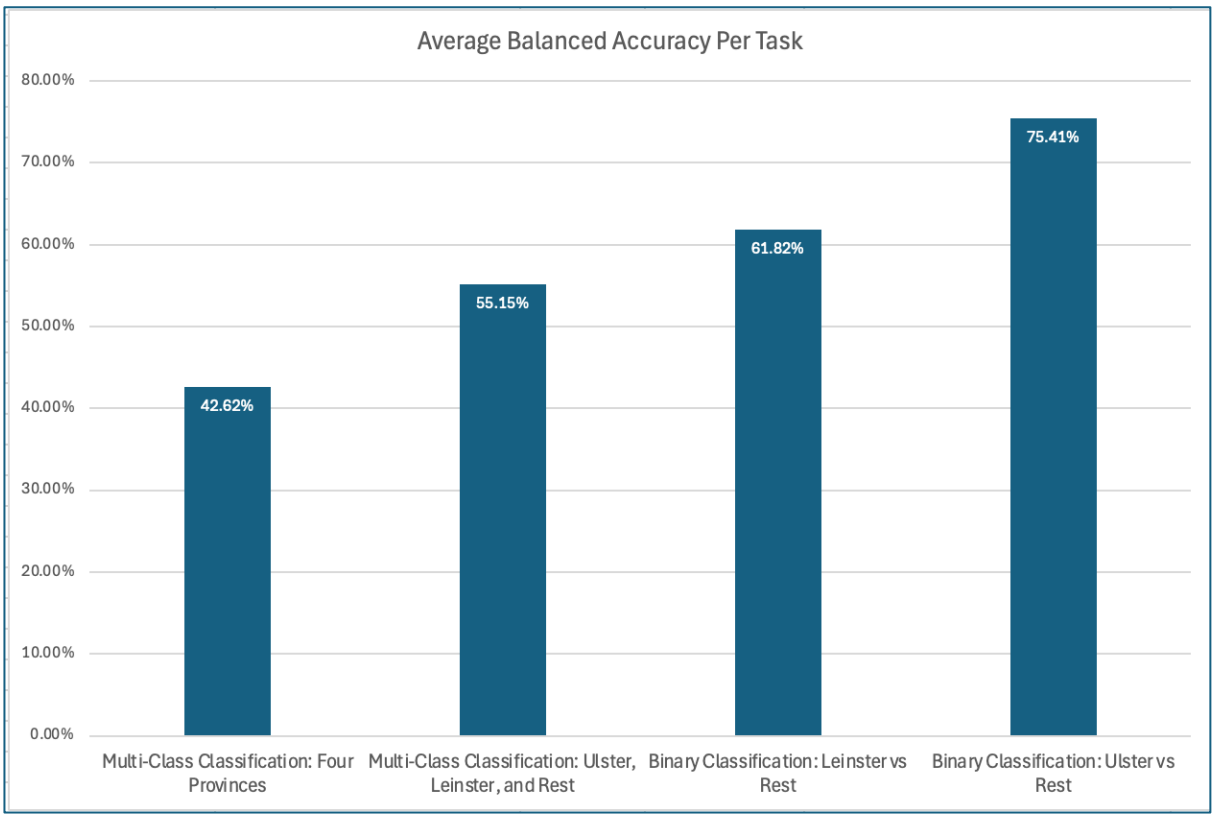


*Figure 68 - B2.4.1 Averaged Balanced Accuracy per Task.*

## B2.5 Best Model Per Task

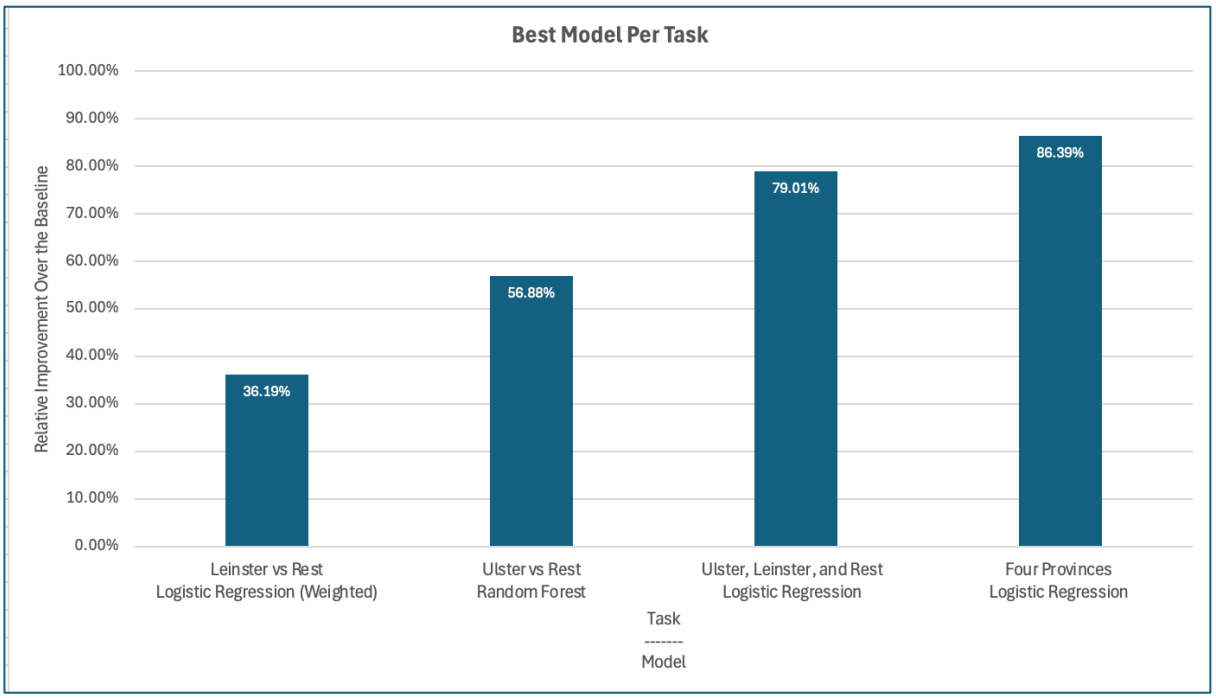


*Figure 69 - B2.5.1 Best Model Per Task, using Relative Improvement over the baseline.*

## B3. Common Voice MFCC Results

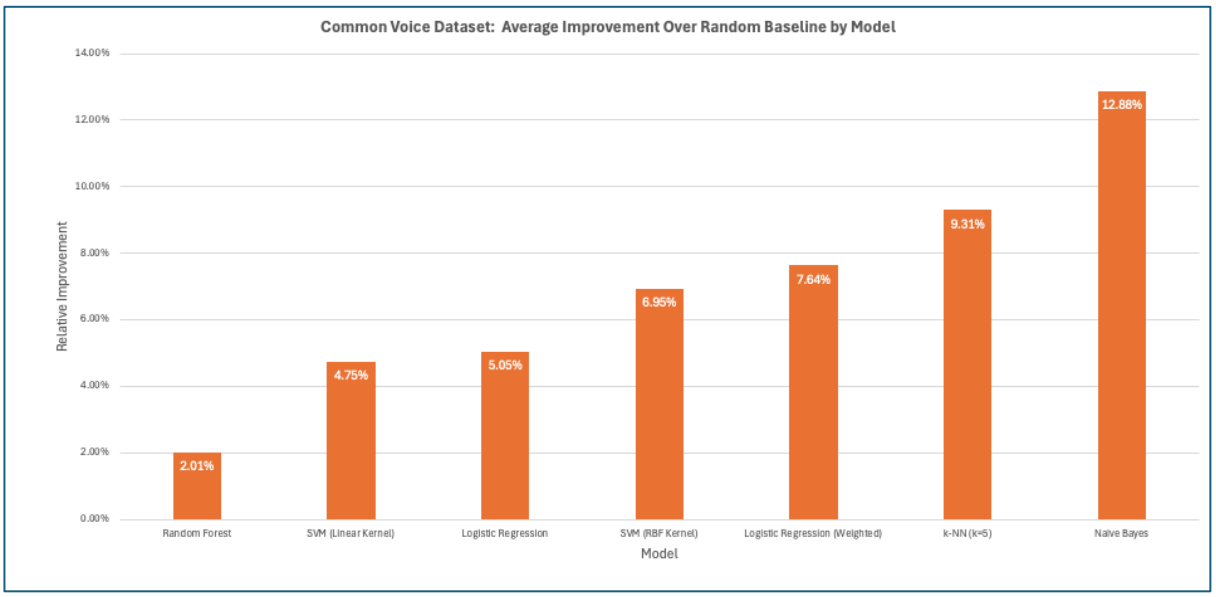


*Figure 70 - B3.1 Average Improvement over the Baseline per Model.*

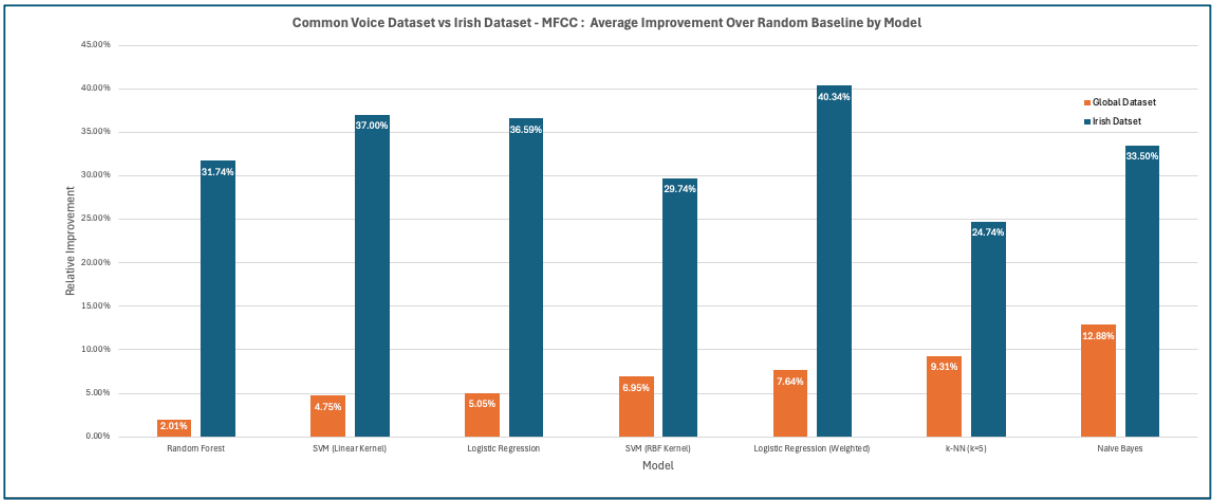


*Figure 71 - B3.2 Average Improvement over the Baseline per Model: Irish Dataset vs Common Voice Dataset Comparison.*

## B.4 Confusion Matrices Irish Data MFCC

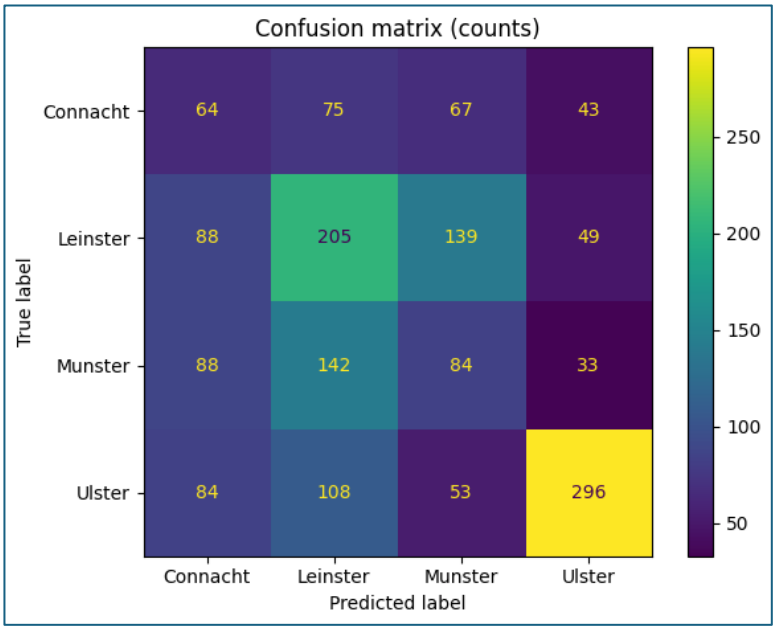


*Figure 72 - B4.1 Confusion Matrix for MFCC-based four-province classification.*

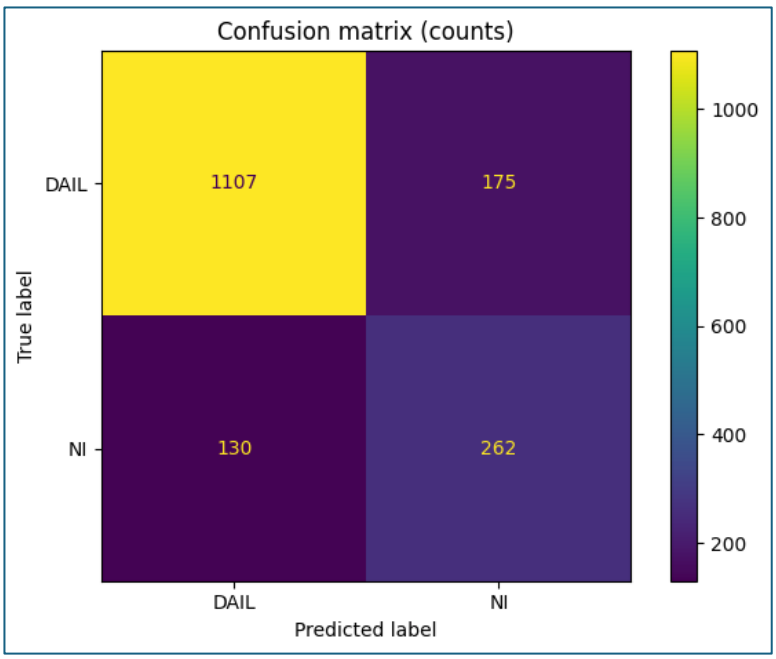


*Figure 73 - B4.2 Confusion Matrix for MFCC-based Dáil Dataset vs Northern Ireland Dataset classification.*

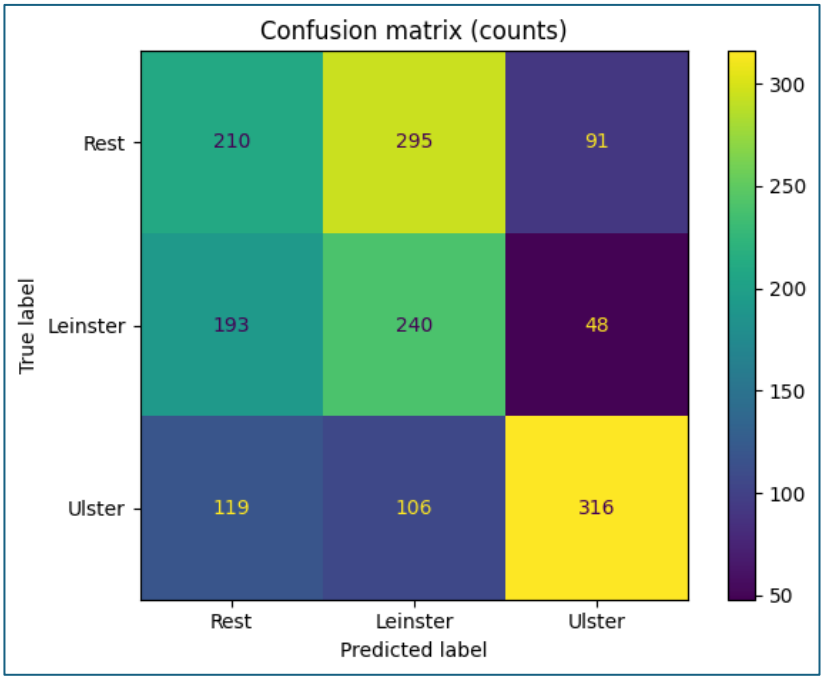


*Figure 74 - B4.3 Confusion Matrix for MFCC-based Ulster, Leinster vs Rest classification.*

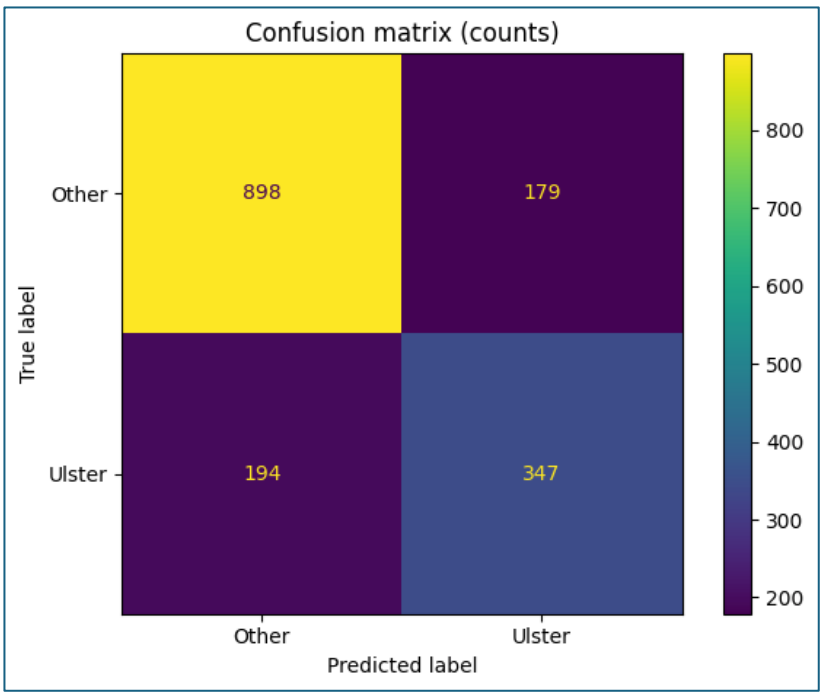


*Figure 75 - B4.4 Confusion Matrix for MFCC-based Ulster vs Rest classification.*

## B.5 Summary of Data Distribution in the Irish Dataset

### B5.1 Summary Across All Tasks

| Task | Classes | Samples | Speakers |
|---|---|---|---|
| Ulster vs Rest | 2 | 1675 | 212 |
| Leinster vs Rest | 2 | 1675 | 212 |
| Ulster, Leinster vs Rest | 3 | 1675 | 212 |
| Four Provinces (Dáil and NI) | 4 | 1675 | 212 |
| Four Provinces (Dáil Only) | 4 | 1282 | 195 |
| Dáil vs Northern Ireland | 2 | 1675 | 212 |

*Figure 76 – B5.1.1 Summary of classification tasks, dataset size and speaker count.*

### B5.2 Class Distribution Across Tasks

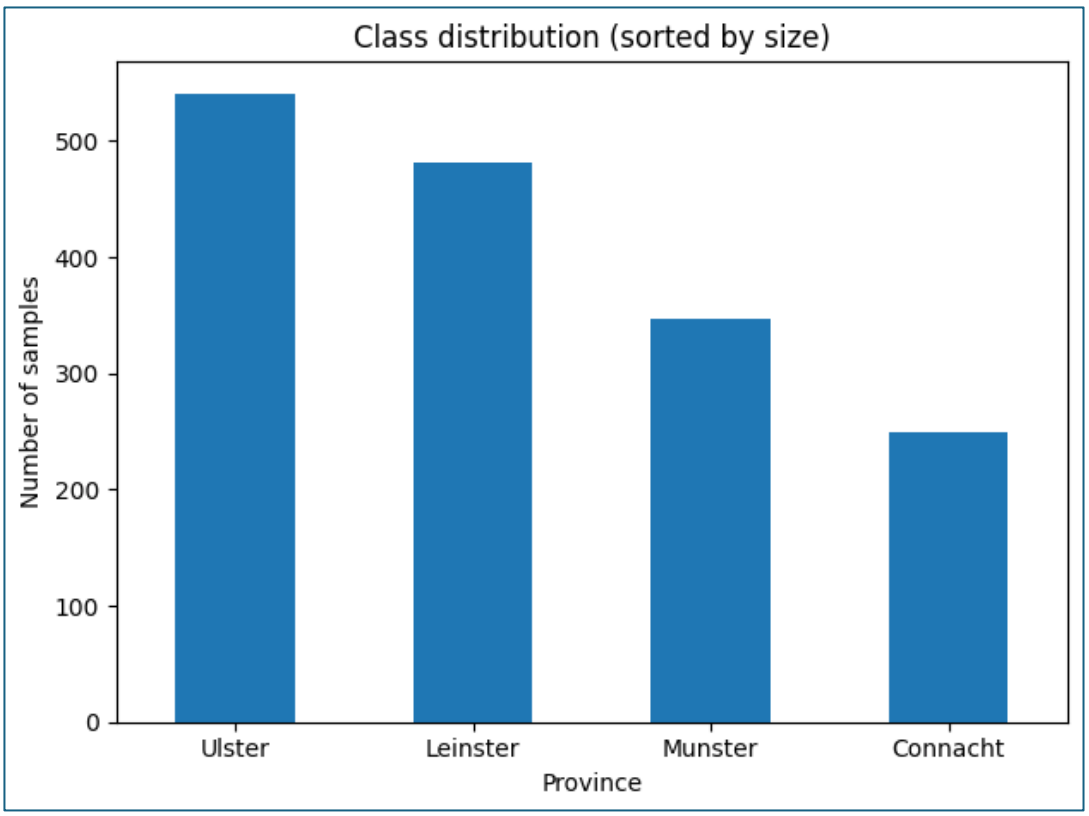


*Figure 77 - B5.2.1 Class Distribution across Four Provinces (Bar Chart).*

| Dáil vs Northern Ireland | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Dáil | 1282 | 195 |
| Northern Ireland | 393 | 11 |

*Figure 78 - B5.2.2 Class Distribution of Dáil Dataset vs Northern Ireland Dataset.*

| Four Provinces (Dáil and NI) | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Leinster | 481 | 91 |
| Ulster | 541 | 30 |
| Connacht | 249 | 33 |
| Munster | 347 | 52 |

*Figure 79 - B5.2.3 Class Distribution of Four Provinces (Dáil and NI).*

| Four Provinces (Dáil Only) | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Leinster | 481 | 91 |
| Ulster | 165 | 14 |
| Connacht | 233 | 32 |
| Munster | 347 | 52 |

*Figure 80 - B5.2.4 Class Distribution of Four Provinces (Dáil only).*

| Ulster, Leinster vs Rest | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Ulster | 541 | 30 |
| Leinster | 481 | 91 |
| Rest | 653 | 85 |

*Figure 81 - B5.2.5 Class Distribution of Ulster, Leinster vs Rest.*

| Leinster vs Rest | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Leinster | 481 | 91 |
| Rest | 1194 | 115 |

*Figure 82 - B5.2.6 Class Distribution of Leinster vs Rest.*

| Ulster vs Rest | | |
|---|---|---|
| **Class** | **Samples** | **Speakers** |
| Ulster | 541 | 30 |
| Rest | 1134 | 176 |

*Figure 83 - B5.2.7 Class Distribution of Ulster vs Rest.*

| task_name | model_name | n_samples | n_speakers | n_classes | class_counts | label_classes | accuracy_mean | accuracy_std | balanced_accuracy_mean | macro_f1_mean |
|---|---|---|---|---|---|---|---|---|---|---|
| dail_only_provinc | knn_k1 | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.31165807457213 | 0.030508867940 | 0.26418717623214105 | 0.25284943007307914 |
| dail_only_provinc | knn_k11 | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.32645344285260 | 0.046151997454 | 0.27008932574240363 | 0.24835835713765028 |
| dail_only_provinc | knn_k3 | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.30309589316921 | 0.024456747 | 0.2636679032820017 | 0.2490282190677308 |
| dail_only_provinc | knn_k5 | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.31846378182223 | 0.039592596 | 0.2653770321509325 | 0.2477735585404667 |
| dail_only_provinc | knn_k7 | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.32799296585510 | 0.025132097 | 0.26357770750863263 | 0.24536388535089299 |
| dail_only_provinc | logreg | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.34544897212106 | 0.045557658 | 0.2721088672581671 | 0.2539634476578346 |
| dail_only_provinc | logreg_weighted | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.299438459 | 0.006941452 | 0.27188960719951955 | 0.2562142555873223 |
| dail_only_provinc | nb | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.29623350566272 | 0.033466667915 | 0.24096703872637568 | 0.23062354425393078 |
| dail_only_provinc | rf | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.33841970809797 | 0.051191308 | 0.24748213816978462 | 0.21378579755165247 |
| dail_only_provinc | svm_linear | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.33756797487448 | 0.066309254 | 0.2612561010787591 | 0.2369943969119785 |
| dail_only_provinc | svm_rbf | 1226 | 189 | 4 | {"Leinster": 481, "Muns | ["Connacht", "Lei | 0.38058831265263 | 0.059185826374 | 0.2468638129049962 | 0.16633684373164648 |

*Figure 84 - B5.3 Summary Table Created Irish Dataset MFCC.*

## Appendix C: System Tests

| Test ID | Objective | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| ST-01 | Verify that the full system starts correctly. | • Main frontend loads successfully.<br>• Swagger docs load successfully.<br>• No immediate console or backend startup errors. | The frontend interface loaded successfully without errors, and the Swagger documentation was accessible. No issues were observed during backend startup. | Pass |
| ST-02 | Verify that available backend models appear in the UI. | • "Loading models…" is replaced with actual model names.<br>• A default model is selected.<br>• Status message updates to selected model. | The model dropdown populated correctly with available models. A default model was automatically selected, and the interface updated to reflect the selected model. | Pass |
| ST-03 | Verify end-to-end prediction using uploaded audio. | • Predict button becomes enabled after upload.<br>• Request is sent to /api/predict.<br>• Prediction result appears in the prediction panel.<br>• Province highlight updates on the map.<br>• No frontend or backend error occurs. | End-to-end prediction was successful for most tested formats, including AIFF, FLAC, AAC, M4A, MP3, OGG, WebM, AMR and WMA. All supported formats produced predictions without system errors. | Pass |
| ST-04 | Verify end-to-end prediction using browser microphone recording. | • Recording countdown works correctly.<br>• Predict button becomes enabled after valid recording.<br>• Audio playback preview is shown.<br>• Prediction result appears.<br>• Map updates correctly. | Audio recording functioned as expected, including countdown enforcement and playback preview. Predictions were successfully generated, and the map visualisation updated correctly. | Pass |
| ST-05 | Verify frontend validation for minimum clip length. | • Recording is rejected.<br>• Predict button remains disabled.<br>• Status message clearly explains minimum 10-second rule.<br>• No prediction request is sent. | Recordings shorter than 10 seconds were correctly rejected. The predict button remained disabled, and an appropriate message informed the user of the minimum duration requirement. | Pass |
| ST-06 | Verify correct handling when no model is available or selected. | • System does not attempt prediction.<br>• Predict button remains grey/ unclickable.<br>• No crash occurs. | The frontend prevented prediction without a selected model by automatically assigning a default model. Direct API testing without a valid model resulted in an | Pass |

| | | | | |
|---|---|---|---|---|
| | | | appropriate HTTP error response. | |
| ST-07 | Verify graceful handling of invalid file types through recording. | • Backend rejects the file safely.<br>• Frontend shows clear error message.<br>• System remains usable afterward. | Non-speech or invalid recorded audio did not cause system failure. The backend processed the input and returned a prediction, although results were not meaningful. System stability was maintained. | Pass |
| ST-08 | Verify graceful handling of invalid file types through upload. | • Backend rejects the file safely.<br>• Frontend shows clear error message.<br>• System remains usable afterward. | The frontend restricted file selection to supported formats. When invalid files were submitted via API testing, the backend returned a clear error response without crashing. | Pass |
| ST-09 | Verify correct rendering of prediction data. | • Selected model name is shown.<br>• Predicted label is shown.<br>• Confidence value is shown.<br>• Probability list is populated correctly. | The prediction panel displayed all expected values, including model name, predicted label, confidence score, and probability distribution. A minor UI visibility issue was identified and resolved. | Pass |
| ST-10 | Verify that the correct province is highlighted after prediction. | • Only the predicted province is highlighted.<br>• Previous highlight is cleared.<br>• Highlight matches prediction text exactly. | The map correctly highlighted the predicted province, cleared any previous highlights, and remained consistent with the displayed prediction output. | Pass |
| ST-11 | Verify modular plug-and-play architecture. | • Frontend updated after reload.<br>• No frontend code changes required.<br>• Prediction still worked. | After modifying backend plugins, the /api/models response updated accordingly. The frontend reflected these changes upon reload without requiring code modifications. Prediction functionality remained fully operational. | Pass |

# Appendix D: Code Snippets

**D1 Experimental Pipeline**

For examples, I am only showing the original Irish dataset MFCC model experimentation pipeline.

**D1.1 - run_batch.py**

```python
def main() -> None:
    print("Loading metadata...")
    metadata_df = load_metadata(str(CSV_PATH))
    print(f"Loaded {len(metadata_df)} rows from {CSV_PATH}")

    all_results: list[dict] = []
    failures: list[dict] = []

    for task_name in TASKS.keys():
        for model_name in MODELS.keys():
            try:
                result = run_single_experiment(
                    metadata_df=metadata_df,
                    task_name=task_name,
                    model_name=model_name,
                )
                all_results.append(result)

            except Exception as exc:
                failure = {
                    "task_name": task_name,
                    "model_name": model_name,
                    "error": str(exc),
                }
                failures.append(failure)
                print(f"FAILED: task={task_name} model={model_name} error={exc}")

    if all_results:
        summary_df = pd.DataFrame(all_results)

        summary_df = summary_df.sort_values(
            by=["task_name", "balanced_accuracy_mean", "macro_f1_mean"],
            ascending=[True, False, False],
        )

        summary_df.to_csv(RESULTS_DIR / "irish_model_summary.csv", index=False)
        print(f"\nSaved summary CSV to {RESULTS_DIR / 'irish_model_summary.csv'}")

    if failures:
        failure_df = pd.DataFrame(failures)
        failure_df.to_csv(LOGS_DIR / "failures.csv", index=False)
        print(f"Saved failures log to {LOGS_DIR / 'failures.csv'}")

    print("\nBatch complete.")
    print(f"Successful experiments: {len(all_results)}")
    print(f"Failed experiments: {len(failures)}")


if __name__ == "__main__":
    main()
```

*Figure 85 - D1.1 /FYP/Experiments/AutoModels/scripts/run_batch.py, the main batch execution script used in the MFCC experimental pipeline. The script loads the training metadata, iterates through all task and model combinations defined in task_defs.py and model_defs.py.*

**D1.2 - pipeline_utils.py**

```python
def load_metadata(csv_path: str) -> pd.DataFrame:
    df = pd.read_csv(csv_path)

    required = {"filepath", "speaker", "label", "dataset"}
    missing = required - set(df.columns)

    if missing:
        raise ValueError(f"Missing required columns: {missing}")

    return df
```

*Figure 86 - D1.2 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py load_metadata(), used to load the experiment metadata CSV and validate that the required columns are present before training begins.*

```python
def extract_mfcc_features(filepath: str, n_mfcc: int = 13):
    """
    Extract MFCC summary statistics from audio file.
    """

    y, sr = librosa.load(filepath, sr=16000)

    mfcc = librosa.feature.mfcc(
        y=y,
        sr=sr,
        n_mfcc=n_mfcc,
    )

    # summarize MFCC across time
    mfcc_mean = np.mean(mfcc, axis=1)
    mfcc_std = np.std(mfcc, axis=1)

    features = np.concatenate([mfcc_mean, mfcc_std])

    return features
```

*Figure 87 - D1.2.2 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py extract_mfcc_features(), used to load an audio file and convert it into a fixed-length MFCC feature vector using mean and standard deviation summary statistics.*

```python
def run_cv_experiment(X, y, speakers, model, n_splits=5):
    """
    Run StratifiedGroupKFold cross-validation.
    """

    le = LabelEncoder()
    y_encoded = le.fit_transform(y)

    cv = StratifiedGroupKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=42,
    )

    fold_results = []

    for fold, (train_idx, test_idx) in enumerate(
        cv.split(X, y_encoded, groups=speakers),
        start=1,
    ):

        X_train = X[train_idx]
        X_test = X[test_idx]

        y_train = y_encoded[train_idx]
        y_test = y_encoded[test_idx]

        model.fit(X_train, y_train)

        preds = model.predict(X_test)

        acc = accuracy_score(y_test, preds)
        bal_acc = balanced_accuracy_score(y_test, preds)
        f1 = f1_score(y_test, preds, average="macro")

        fold_results.append({
            "fold": fold,
            "accuracy": acc,
            "balanced_accuracy": bal_acc,
            "macro_f1": f1,
        })

    return fold_results, le
```

*Figure 88 - D1.2.3 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py run_cv_experiment(), used to perform speaker-safe cross-validation with StratifiedGroupKFold, train the selected model, and calculate fold-level evaluation metrics.*

```python
def build_feature_matrix(X_df: pd.DataFrame):
    """
    Convert dataframe of filepaths to feature matrix.
    """

    features = []

    for path in X_df["filepath"]:
        feat = extract_mfcc_features(path)
        features.append(feat)

    X = np.vstack(features)

    return X
```

*Figure 89 - D1.2.4 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py build_feature_matrix(), used to iterate through all file paths in the metadata and build the full MFCC feature matrix for model training and evaluation.*

```python
def run_cv_experiment(X, y, speakers, model, n_splits=5):
    """
    Run StratifiedGroupKFold cross-validation.
    """

    le = LabelEncoder()
    y_encoded = le.fit_transform(y)

    cv = StratifiedGroupKFold(
        n_splits=n_splits,
        shuffle=True,
        random_state=42,
    )

    fold_results = []

    for fold, (train_idx, test_idx) in enumerate(
        cv.split(X, y_encoded, groups=speakers),
        start=1,
    ):

        X_train = X[train_idx]
        X_test = X[test_idx]

        y_train = y_encoded[train_idx]
        y_test = y_encoded[test_idx]

        model.fit(X_train, y_train)

        preds = model.predict(X_test)

        acc = accuracy_score(y_test, preds)
        bal_acc = balanced_accuracy_score(y_test, preds)
        f1 = f1_score(y_test, preds, average="macro")

        fold_results.append({
            "fold": fold,
            "accuracy": acc,
            "balanced_accuracy": bal_acc,
            "macro_f1": f1,
        })

    return fold_results, le
```

*Figure 90 - D1.2.5 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py run_cv_experiment(), used to perform speaker-safe cross-validation with StratifiedGroupKFold, train the selected model, and calculate fold-level evaluation metrics.*

```python
def summarise_results(fold_results):
    df = pd.DataFrame(fold_results)

    summary = {
        "accuracy_mean": df["accuracy"].mean(),
        "accuracy_std": df["accuracy"].std(),
        "balanced_accuracy_mean": df["balanced_accuracy"].mean(),
        "macro_f1_mean": df["macro_f1"].mean(),
    }

    return summary
```

*Figure 91 - D1.2.6 /FYP/Experiments/AutoModels/scripts/pipeline_utils.py summarise_results(), used to aggregate fold-level results into summary performance metrics, including mean accuracy, mean balanced accuracy, and mean macro F1 score.*

### D1.3 – task_defs.py

```python
# ------------------------------------------------------------
# Task registry
# ------------------------------------------------------------

TASKS = {
    "province_4way": "task_province_4way",
    "dail_only_province": "task_dail_only_province",
    "dail_vs_ni": "task_dail_vs_ni",
    "ulster_leinster_rest": "task_ulster_leinster_rest",
    "ulster_vs_rest": "task_ulster_vs_rest",
    "leinster_vs_rest": "task_leinster_vs_rest",
}
```

*Figure 92 - D1.3.1 task_defs.py: task registry and example task definition used to map the full metadata table into a specific classification problem for batch experimentation.*

```python
def task_leinster_vs_rest(df: pd.DataFrame):
    """
    Binary classification: Leinster vs Rest
    """

    df = df[df["label"] != "Other"].copy()

    df["target"] = df["label"].apply(
        lambda x: "Leinster" if x == "Leinster" else "Rest"
    )

    X_df = df[["filepath", "speaker"]]
    y = df["target"]

    return X_df, y


def task_province_4way(df: pd.DataFrame):
    """
    4-way province classification
    Drops 'Other'
    """

    df = df[df["label"] != "Other"].copy()

    X_df = df[["filepath", "speaker"]]
    y = df["label"]

    return X_df, y

# ------------------------------------------------------------
# Helper
# ------------------------------------------------------------

def get_task(task_name: str):
    """
    Returns task function by name
    """

    if task_name not in TASKS:
        raise ValueError(f"Unknown task: {task_name}")

    return globals()[TASKS[task_name]]
```

*Figure 93 - D1.3.1 task_defs.py: Task implementations and helper function used to retrieve experiment tasks dynamically during batch execution.*

### D1.4 – model_defs.py

```python
MODELS = {
    "logreg": "make_logreg",
    "logreg_weighted": "make_logreg_weighted",
    "rf": "make_rf",
    "knn_k1": "make_knn_k1",
    "knn_k3": "make_knn_k3",
    "knn_k5": "make_knn_k5",
    "knn_k7": "make_knn_k7",
    "knn_k11": "make_knn_k11",
    "svm_linear": "make_svm_linear",
    "svm_rbf": "make_svm_rbf",
    "nb": "make_nb",
}


def make_logreg():
    return LogisticRegression(
        max_iter=5000,
        random_state=42,
    )


def make_logreg_weighted():
    return LogisticRegression(
        max_iter=5000,
        class_weight="balanced",
        random_state=42,
    )


def make_rf():
    return RandomForestClassifier(
        n_estimators=300,
        max_depth=None,
        min_samples_split=2,
        min_samples_leaf=1,
        random_state=42,
        n_jobs=-1,
    )
```

*Figure 94 - D1.4.1 model_defs.py: model registry and example classifier factory functions used to define reusable experiment configurations.*

```python
def make_knn_k11():
    return KNeighborsClassifier(n_neighbors=11)


def make_svm_linear():
    return SVC(
        kernel="linear",
        probability=True,
        random_state=42,
    )


def make_svm_rbf():
    return SVC(
        kernel="rbf",
        probability=True,
        random_state=42,
    )


def make_nb():
    return GaussianNB()


def get_model(model_name: str):
    if model_name not in MODELS:
        raise ValueError(f"Unknown model: {model_name}")

    return globals()[MODELS[model_name]]()
```

*Figure 95 - D1.4.2 model_defs.py: additional model configurations and the helper function used to instantiate a selected classifier during batch experimentation.*

### D1.5 – train_wav2vec_deploy.py

```python
BACKEND_DIR = Path(__file__).resolve().parents[1]
ARTIFACTS_DIR = BACKEND_DIR / "artifacts"
ARTIFACTS_DIR.mkdir(parents=True, exist_ok=True)

CSV_PATH = Path("/Users/cianan/Documents/College/GitHub/FYP/Data/Dail_NI_Metadata/training_minimal_clean.csv")

EXPERIMENTS_SCRIPTS_DIR = Path("/Users/cianan/Documents/College/GitHub/FYP/Experiments/AutoWav2VecIrish/scripts")
if str(EXPERIMENTS_SCRIPTS_DIR) not in sys.path:
    sys.path.insert(0, str(EXPERIMENTS_SCRIPTS_DIR))

from model_defs import get_model
from task_defs import get_task
from pipeline_utils_wav2vec import build_feature_matrix


DEPLOY_CONFIGS = [
    ("ulster_vs_rest", "rf"),
    ("leinster_vs_rest", "logreg"),
    ("ulster_leinster_rest", "logreg"),
    ("province_4way", "logreg"),
]
```

*Figure 96 - D1.5.1 train_wav2vec_deploy.py: deployment configuration defining the selected task–model combinations and reuse of the experimental pipeline utilities for wav2vec feature extraction.*

```python
def train_one(df: pd.DataFrame, task_name: str, model_name: str):
    # Materialise the subset and labels for one deployment task.
    task_fn = get_task(task_name)
    X_df, y = task_fn(df)

    if len(X_df) == 0:
        raise ValueError(f"No rows returned for task '{task_name}'")

    print(f"\n=== Training {task_name} | {model_name} ===")

    # Build wav2vec features using the external experiment utilities.
    X = build_feature_matrix(X_df)

    label_encoder = LabelEncoder()
    y_encoded = label_encoder.fit_transform(np.array(y)).astype(np.int64)

    # Train the selected classifier on the task-specific embeddings.
    model = get_model(model_name)
    model.fit(X, y_encoded)

    artifact_prefix = f"wav2vec_{task_name}_{model_name}"

    model_path = ARTIFACTS_DIR / f"{artifact_prefix}_model.joblib"
    le_path = ARTIFACTS_DIR / f"{artifact_prefix}_label_encoder.joblib"
    meta_path = ARTIFACTS_DIR / f"{artifact_prefix}_meta.json"

    joblib.dump(model, model_path)
    joblib.dump(label_encoder, le_path)

    # Save enough metadata for inspection and later deployment checks.
    metadata = {
        "task_name": task_name,
        "model_name": model_name,
        "label_classes": label_encoder.classes_.tolist(),
        "feature_type": "wav2vec2-base_mean_pool",
    }

    with open(meta_path, "w", encoding="utf-8") as f:
        json.dump(metadata, f, indent=2)

    print(f"Saved: {artifact_prefix}")
```

*Figure 97 - D1.5.2 train_wav2vec_deploy.py: training and export routine used to generate deployable model artefacts, including trained classifiers, label encoders, and metadata for integration into the backend plugin system.*

## F2 Backend

### D2.1 - app.py

```python
load_plugins()

BASE_DIR = Path(__file__).resolve().parent
FRONTEND_DIR = BASE_DIR.parent / "frontend"

app = FastAPI(title="Prototype4 Accent API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.mount("/static", StaticFiles(directory=FRONTEND_DIR), name="static")


@app.get("/")
def read_index():
    # Serve the single-page frontend shell.
    return FileResponse(FRONTEND_DIR / "index.html")


@app.get("/api/health")
def health() -> dict:
    # Lightweight liveness check for the backend service.
    return {"status": "ok"}


@app.get("/api/models")
def api_models() -> dict:
    # Expose model metadata for the frontend selector and model descriptions.
    return {"models": list_models()}
```

*Figure 98 - D2.1.1 Initialises FastAPI app, loads model plugins, serves the frontend interface and exposes endpoints for health and model discovery.*

```python
@app.post("/api/predict")
async def api_predict(
    audio: UploadFile = File(...),
    model_id: str = Form(...),
) -> dict:
    # Resolve the selected model before reading and processing the upload.
    try:
        model = get_model(model_id)
    except KeyError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc

    # Read the uploaded file into memory for the downstream plugin pipeline.
    audio_bytes = await audio.read()

    if not audio_bytes:
        raise HTTPException(status_code=400, detail="Empty audio upload")

    # Each plugin performs its own preprocessing and inference steps.
    try:
        result = model.predict(audio_bytes)
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except Exception as exc:
        raise HTTPException(
            status_code=500,
            detail=f"Prediction failed for model '{model_id}'.",
        ) from exc

    # Return a consistent response shape for the frontend renderer.
    return {
        "request_id": str(uuid.uuid4()),
        "model_id": model_id,
        **result,
    }
```

*Figure 99 - D2.1.2 Implements the core inference API, handling audio uploads, dynamically selecting the requested model plugin, executing prediction, and returning structured results to the frontend.*

### D2.2 - plugin_loader.py

```python
from registry import register
from plugins.dummy_model import plugin as dummy_plugin
from plugins.mfcc_logreg_v1_01 import plugin as mfcc_logreg_plugin
from plugins.wav2vec_ulster_vs_rest_rf import Wav2VecUlsterVsRestRF
from plugins.wav2vec_leinster_vs_rest_logreg import Wav2VecLeinsterVsRestLogReg
from plugins.wav2vec_ulster_leinster_rest_logreg import Wav2VecUlsterLeinsterRestLogReg
from plugins.wav2vec_province_4way_logreg import Wav2VecProvince4WayLogReg


def load_plugins() -> None:
    # Register the test plugin and the deployed models in their display order.
    register(dummy_plugin)
    register(mfcc_logreg_plugin)
    register(Wav2VecUlsterVsRestRF())
    register(Wav2VecLeinsterVsRestLogReg())
    register(Wav2VecUlsterLeinsterRestLogReg())
    register(Wav2VecProvince4WayLogReg())
```

*Figure 100 - D2.2 Registers all available model plugins during application startup, enabling dynamic model selection by exposing each classifier through the backend registry and API.*

**D2.3 - registry.py**

```python
from __future__ import annotations

from typing import Dict

from plugins.base import ModelPlugin


_REGISTRY: Dict[str, ModelPlugin] = {}


def register(plugin: ModelPlugin) -> None:
    # Guard against duplicate IDs so `/api/predict` stays unambiguous.
    if plugin.id in _REGISTRY:
        raise ValueError(f"Duplicate model id: {plugin.id}")
    _REGISTRY[plugin.id] = plugin


def get_model(model_id: str) -> ModelPlugin:
    # Return the plugin instance chosen by the frontend.
    if model_id not in _REGISTRY:
        raise KeyError(f"Unknown model_id: {model_id}")
    return _REGISTRY[model_id]


def list_models() -> list[dict]:
    # Build the lightweight metadata payload returned to the UI.
    return [
        {
            "id": plugin.id,
            "name": plugin.name,
            "description": plugin.description,
        }
        for plugin in _REGISTRY.values()
    ]
```

*Figure 101 - D2.3 Implements the in-memory model registry used by the backend to store, retrieve, and list available model plugins, enabling dynamic model selection and metadata exposure through the API. register(), get_mode(), list_models()*

### D2.4 - wav2vec_ulster_vs_rest_rf.py

```python
from pathlib import Path

import joblib
import numpy as np

from plugins.base import ModelPlugin
from services.wav2vec_features import audio_bytes_to_embedding

BASE_DIR = Path(__file__).resolve().parents[1]
ARTIFACTS_DIR = BASE_DIR / "artifacts"


class Wav2VecUlsterVsRestRF(ModelPlugin):
    id = "wav2vec_ulster_vs_rest_rf"
    name = "Ulster Detection (Wav2Vec)"
    description = "Detects whether the speaker is from Ulster."

    def __init__(self):
        # Load the trained classifier artefacts once when the backend starts.
        self.model = joblib.load(
            ARTIFACTS_DIR / "wav2vec_ulster_vs_rest_rf_model.joblib"
        )
        self.label_encoder = joblib.load(
            ARTIFACTS_DIR / "wav2vec_ulster_vs_rest_rf_label_encoder.joblib"
        )

    def predict(self, audio_bytes: bytes):
        # Decode and embed the clip before handing it to the classifier.
        embedding = audio_bytes_to_embedding(audio_bytes)

        probs = self.model.predict_proba([embedding])[0]
        pred_idx = int(np.argmax(probs))
        label = self.label_encoder.inverse_transform([pred_idx])[0]
        # Rebuild labels from the encoder so the response stays model-driven.
        probs_list = [
            {
                "label": str(self.label_encoder.inverse_transform([i])[0]),
                "p": float(p),
            }
            for i, p in enumerate(probs)
        ]
        probs_list.sort(key=lambda item: item["p"], reverse=True)

        return {
            "label": str(label),
            "confidence": float(probs[pred_idx]),
            "probs": probs_list,
        }
```

*Figure 102 - D2.4 Example backend model plugin implementing a wav2vec-based classifier, showing how audio input is converted into embeddings, passed through a trained model, and returned as a structured prediction for the frontend.*

### D2.5 - wav2vec_ulster_vs_rest_rf_meta.json

```json
{
  "task_name": "ulster_vs_rest",
  "model_name": "rf",
  "label_classes": [
    "Rest",
    "Ulster"
  ],
  "feature_type": "wav2vec2-base_mean_pool"
}
```

*Figure 103 - D2.5 Example model metadata file storing task configuration, label mappings, and feature extraction type, used to support deployment, inspection, and validation of trained model artefacts.*

**D2.6 – audio.py**

```python
TARGET_SR = 16000
REQUIRED_CLIP_SECONDS = 10


def _guess_audio_suffix(audio_bytes: bytes) -> str:
    # Pick a temporary file suffix that gives fallback decoders a better chance.
    if audio_bytes.startswith(b"RIFF") and audio_bytes[8:12] == b"WAVE":
        return ".wav"
    if audio_bytes.startswith(b"fLaC"):
        return ".flac"
    if audio_bytes.startswith(b"OggS"):
        return ".ogg"
    if audio_bytes.startswith(b"ID3"):
        return ".mp3"
    if audio_bytes.startswith(b"\x1A\x45\xDF\xA3"):
        return ".webm"
    if len(audio_bytes) > 12 and audio_bytes[4:8] == b"ftyp":
        return ".m4a"
    return ".bin"


def _load_with_ffmpeg(audio_bytes: bytes, target_sr: int) -> tuple[np.ndarray, int]:
    """Use ffmpeg as the final decoding fallback for difficult formats."""
    input_path: Path | None = None
    output_path: Path | None = None

    try:
        suffix = _guess_audio_suffix(audio_bytes)

        with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp_in:
            tmp_in.write(audio_bytes)
            input_path = Path(tmp_in.name)

        with tempfile.NamedTemporaryFile(suffix=".wav", delete=False) as tmp_out:
            output_path = Path(tmp_out.name)

        # Convert to a simple mono WAV so the rest of the pipeline is consistent.
        cmd = [
            "ffmpeg",
            "-y",
            "-i",
            str(input_path),
            "-ac",
            "1",
            "-ar",
            str(target_sr),
            str(output_path),
        ]

        result = subprocess.run(
            cmd,
            capture_output=True,
            text=True,
            check=False,
        )

        if result.returncode != 0:
            raise ValueError(f"ffmpeg conversion failed: {result.stderr.strip()}")

        waveform, sr = sf.read(output_path)
        waveform = np.asarray(waveform, dtype=np.float32)

        if waveform.ndim > 1:
            waveform = np.mean(waveform, axis=1)

        return waveform, sr

    finally:
        if input_path and input_path.exists():
            input_path.unlink()
        if output_path and output_path.exists():
            output_path.unlink()
```

*Figure 104 - D2.6.1 Implements backend audio decoding support by identifying likely input formats and providing a fallback conversion path through ffmpeg for difficult or unsupported uploads. _load_with_ffmpeg()*

```python
def load_audio_from_bytes(
    audio_bytes: bytes, target_sr: int = TARGET_SR
) -> tuple[np.ndarray, int]:
    """
    Load uploaded audio bytes and convert to mono float waveform at target_sr.
    """
    if not audio_bytes:
        raise ValueError("Empty audio payload")

    # 1. Fast path: decode directly from memory when the format is simple enough.
    try:
        with io.BytesIO(audio_bytes) as buffer:
            waveform, sr = sf.read(buffer)
    except Exception:
        # 2. Fallback: give librosa a temporary file if in-memory decode fails.
        tmp_path: Path | None = None
        try:
            suffix = _guess_audio_suffix(audio_bytes)
            with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
                tmp.write(audio_bytes)
                tmp_path = Path(tmp.name)

            waveform, sr = librosa.load(tmp_path, sr=None, mono=True)

        except Exception:
            # 3. Final fallback: let ffmpeg transcode awkward inputs to WAV first.
            try:
                waveform, sr = _load_with_ffmpeg(audio_bytes, target_sr=target_sr)
            except Exception as exc:
                raise ValueError("Unsupported or unreadable audio format.") from exc

        finally:
            if tmp_path and tmp_path.exists():
                tmp_path.unlink()

    waveform = np.asarray(waveform, dtype=np.float32)

    if waveform.ndim > 1:
        waveform = np.mean(waveform, axis=1)

    # Resample once so MFCC and wav2vec paths receive the same input format.
    if sr != target_sr:
        waveform = librosa.resample(waveform, orig_sr=sr, target_sr=target_sr)
        sr = target_sr

    required_samples = int(target_sr * REQUIRED_CLIP_SECONDS)
    if waveform.size < required_samples:
        raise ValueError(
            f"Audio must be at least {REQUIRED_CLIP_SECONDS} seconds long."
        )

    # Keep a fixed clip length so downstream models see consistent inputs.
    waveform = np.ascontiguousarray(waveform[:required_samples], dtype=np.float32)

    return waveform, sr
```

*Figure 105 - D2.6.2 Normalises uploaded audio into a consistent format for model inference by decoding raw bytes, converting to mono, resampling to 16 kHz, validating minimum duration, and trimming clips to the fixed input length used by the models. load_audio_from_bytes()*

**D2.7 – features.py**

```python
from __future__ import annotations

import numpy as np
import librosa


def extract_mfcc_summary_features(
    waveform: np.ndarray,
    sr: int,
    n_mfcc: int = 13,
) -> np.ndarray:
    """
    Extract MFCCs and summarise them into a fixed-length feature vector.

    Output shape:
    [mfcc_means..., mfcc_stds...] => length = n_mfcc * 2
    """
    # Compute frame-level MFCCs, then compress them into summary statistics.
    mfcc = librosa.feature.mfcc(y=waveform, sr=sr, n_mfcc=n_mfcc)

    mfcc_means = np.mean(mfcc, axis=1)
    mfcc_stds = np.std(mfcc, axis=1)

    features = np.concatenate([mfcc_means, mfcc_stds]).astype(np.float32)
    return features
```

*Figure 106 - D2.7 Implements MFCC-based feature extraction by converting the input waveform into Mel-frequency cepstral coefficients and summarising them using mean and standard deviation to produce a fixed-length feature vector for classical machine learning mode. extract_mfcc_summary()*

**D2.8 wav2vec_features.py**

```python
import numpy as np
import torch
from transformers import Wav2Vec2Processor, Wav2Vec2Model

from services.audio import load_audio_from_bytes

MODEL_NAME = "facebook/wav2vec2-base"

_processor = None
_model = None


def _load_model():
    # Load the shared encoder only when a wav2vec-backed prediction is requested.
    global _processor, _model

    if _processor is None or _model is None:
        print("Loading wav2vec model...")
        _processor = Wav2Vec2Processor.from_pretrained(MODEL_NAME)
        _model = Wav2Vec2Model.from_pretrained(MODEL_NAME)
        _model.eval()


def audio_bytes_to_embedding(audio_bytes: bytes) -> np.ndarray:
    # Reuse the shared audio loader so wav2vec sees the same 10 s clip as the
    # rest of the system.
    _load_model()

    waveform, _sr = load_audio_from_bytes(audio_bytes)

    # Tokenise the waveform for the Hugging Face wav2vec model.
    inputs = _processor(waveform, sampling_rate=16000, return_tensors="pt")

    with torch.no_grad():
        outputs = _model(**inputs)

    # Mean-pool the time dimension to produce one fixed-size embedding.
    hidden_states = outputs.last_hidden_state
    embedding = hidden_states.mean(dim=1).squeeze().cpu().numpy().astype(np.float32)

    return embedding
```

*Figure 107 - D2.8 Implements the wav2vec 2.0 feature extraction process by loading a pre-trained model, converting normalised audio into contextualised speech representations, and applying mean pooling to produce fixed-length embeddings for downstream classification. _load_model(), audio_bytes_to_embedding()*

**D2.8 wav2vec_features.py**

## D3 Frontend

### D3.1 - app.js

```
function setStatus(message) {
  // Centralise status updates so the page reports the current pipeline step.
  document.getElementById("statusMessage").textContent = message;
}

function renderSelectedModelDescription(modelId) {
  // Show the selected model descriptor beneath the dropdown in the controls panel.
  const modelDescription = document.getElementById("modelDescription");
  if (!modelDescription) {
    return;
  }

  const model = store.models.find((item) => item.id === modelId);

  if (!model) {
    modelDescription.textContent = "No model description available.";
    return;
  }

  modelDescription.textContent =
    model.description || "No model description available.";
}
```

*Figure 108 - D3.1.1 setStatus() and renderSelectedModelDescription(), Provides helper functions for updating the user interface, including displaying system status messages and rendering descriptive information for the currently selected model. These functions support user feedback and system transparency.*

```
async function initModels() {
  // Populate the model selector from the backend metadata endpoint.
  const modelSelect = document.getElementById("modelSelect");

  try {
    setStatus("Loading models...");
    const data = await fetchModels();

    const rawModels = Array.isArray(data.models) ? data.models : [];

    const models = rawModels.map((model) => {
      if (typeof model === "string") {
        return { id: model, name: model, description: "" };
      }
      return {
        id: model.id,
        name: model.name ?? model.id,
        description: model.description ?? "",
      };
    });

    store.models = models;
    modelSelect.innerHTML = "";

    if (models.length === 0) {
      const option = document.createElement("option");
      option.value = "";
      option.textContent = "No models available";
      modelSelect.appendChild(option);
      store.selectedModelId = null;
      renderSelectedModelDescription(null);
      setStatus("No models found.");
      return;
    }

    // Keep the backend order so the UI reflects the registered plugin order.
    for (const model of models) {
      const option = document.createElement("option");
      option.value = model.id;
      option.textContent = model.name;
      modelSelect.appendChild(option);
    }

    store.selectedModelId = models[0].id;
    modelSelect.value = store.selectedModelId;
    renderSelectedModelDescription(store.selectedModelId);

    modelSelect.addEventListener("change", (event) => {
      store.selectedModelId = event.target.value;
      renderSelectedModelDescription(store.selectedModelId);
      setStatus(`Selected model: ${store.selectedModelId}`);
    });

    setStatus(`Selected model: ${store.selectedModelId}`);
  } catch (error) {
    console.error(error);
    setStatus("Failed to load models.");
  }
}
```

*Figure 109 - D3.1.2 initModels(). Fetches available model metadata from the backend API, populates the model selection dropdown, and updates the global application state. It also handles user interaction with model selection and dynamically displays model descriptions.*

```
async function initApp() {
  // Start the supporting modules in the same order the page needs them.
  try {
    await initMap();
  } catch (error) {
    console.error("Map failed to load:", error);
  }

  await initModels();
  initRecording();

  const resetMapBtn = document.getElementById("resetMapBtn");
  if (resetMapBtn) {
    resetMapBtn.addEventListener("click", () => {
      resetMapView();
    });
  }

  const clearHistoryBtn = document.getElementById("clearHistoryBtn");
  if (clearHistoryBtn) {
    clearHistoryBtn.addEventListener("click", () => {
      clearPredictionResults();
      setStatus("Previous prediction results cleared.");
    });
  }
}

initApp();
```

*Figure 110 - D3.1.3 initApp(). Initialises the frontend application by coordinating the startup sequence of core modules, including the map visualisation, model loading, and audio recording system. It also binds global user interface controls such as resetting the map view and clearing prediction history.*

**D3.2 - api.js**

```
const API_BASE =
  window.location.hostname === "localhost"
    ? "http://127.0.0.1:8000"
    : "";

function guessRecordingFilename(audioSource) {
  // Preserve a sensible extension for recorded blobs sent through FormData.
  const mimeType = audioSource?.type ?? "";

  if (mimeType.includes("mp4")) {
    return "recording.m4a";
  }
  if (mimeType.includes("ogg")) {
    return "recording.ogg";
  }
  if (mimeType.includes("wav")) {
    return "recording.wav";
  }

  return "recording.webm";
}

export async function fetchModels() {
  // Used during page startup to populate the model selector and descriptions.
  const response = await fetch(`${API_BASE}/api/models`);

  if (!response.ok) {
    throw new Error("Failed to fetch models");
  }

  return await response.json();
}

export async function predictAudio(audioSource, modelId) {
  // Submit either an uploaded file or a recorded blob to the prediction route.
  const formData = new FormData();

  if (audioSource instanceof File) {
    formData.append("audio", audioSource, audioSource.name);
  } else {
    formData.append("audio", audioSource, guessRecordingFilename(audioSource));
  }

  formData.append("model_id", modelId);

  const response = await fetch(`${API_BASE}/api/predict`, {
    method: "POST",
    body: formData,
  });

  if (!response.ok) {
    const err = await response.json().catch(() => ({}));
    throw new Error(err.detail || "Prediction failed");
  }

  return await response.json();
}
```

*Figure 111 D3.2 - api.js Provides a lightweight abstraction over the backend API, handling requests to fetch available models and submit audio for prediction. It constructs multipart form data for file uploads, manages response handling, and ensures consistent communication between the frontend and the backend services.*

**D3.3 - recording.js**

```
import { store } from "./store.js?v=20260410e";
import { predictAudio } from "./api.js?v=20260410e";
import {
  renderPrediction,
  renderError,
  renderFeedbackState,
  renderPredictionHistory,
  savePrediction,
  updatePredictionFeedback,
} from "./predictions.js?v=20260415b";
import { updateMap, clearMapHighlight } from "./map.js?v=20260415c";

let mediaStream = null;
let recordingTrack = null;
let playbackObjectUrl = null;
let recordingStartTime = null;
let recordingTimerId = null;
let recordingAutoStopId = null;
let recordingAudioContext = null;
let recordingSourceNode = null;
let recordingProcessorNode = null;
let recordingMonitorNode = null;
let recordedPcmChunks = [];
let recordedSampleRate = 16000;
let isStoppingRecording = false;
let recordingPeak = 0;
let recordingRmsSumSquares = 0;
let recordingRmsSamples = 0;
let recordingLastLevelLogAt = 0;

const REQUIRED_CLIP_SECONDS = 10;
const RECORDING_COUNTDOWN_SECONDS = 11;
const MICROPHONE_CONSTRAINT_CANDIDATES = [
  true,
  {
    channelCount: 1,
  },
];

function setStatus(message) {
  // Keep status updates local to the recording and prediction workflow.
  document.getElementById("statusMessage").textContent = message;
}
```

*Figure 112 - D3.3.1 Defines the recording module imports, persistent state variables, and local status update helper used throughout the browser-side audio capture and prediction workflow.*

```
export function initRecording() {
  // Wire the controls panel once the page has loaded and models are available.
  const recordBtn = document.getElementById("recordBtn");
  const stopBtn = document.getElementById("stopBtn");
  const predictBtn = document.getElementById("predictBtn");
  const audioPlayback = document.getElementById("audioPlayback");
  const audioFileInput = document.getElementById("audioFile");
  const inputDeviceSelect = document.getElementById("inputDeviceSelect");
  const recordingCountdown = document.getElementById("recordingCountdown");

  const feedbackYesBtn = document.getElementById("feedbackYesBtn");
  const feedbackNoBtn = document.getElementById("feedbackNoBtn");
  const correctLabelSection = document.getElementById("correctLabelSection");
  const correctLabelSelect = document.getElementById("correctLabelSelect");
  const saveCorrectionBtn = document.getElementById("saveCorrectionBtn");
  const feedbackMessage = document.getElementById("feedbackMessage");

  if (!recordBtn || !stopBtn || !predictBtn || !audioPlayback) {
    console.error("Recording controls are missing from the page.");
    return;
  }

  function setPlaybackSource(source) {
    // Rebuild the audio preview element whenever the active source changes.
    if (playbackObjectUrl) {
      URL.revokeObjectURL(playbackObjectUrl);
      playbackObjectUrl = null;
    }

    audioPlayback.pause();
    audioPlayback.currentTime = 0;

    if (!source) {
      audioPlayback.removeAttribute("src");
      audioPlayback.load();
      return;
    }

    playbackObjectUrl = URL.createObjectURL(source);
    audioPlayback.src = playbackObjectUrl;
    audioPlayback.muted = false;
    audioPlayback.volume = 1;
    audioPlayback.load();
  }

  function refreshPredictButtonState() {
    // Enable prediction only when there is a clip ready to submit.
    const hasAudioSource = Boolean(
      store.uploadedAudioFile || store.latestAudioBlob
    );
    predictBtn.disabled = !hasAudioSource;
  }
```

*Figure 113 - D3.3.2 Initialises the main recording module by binding user interface controls, preparing browser audio playback, and enabling prediction only when a valid audio source is available.*

```
async function loadInputDevices(preferredDeviceId = store.selectedInputDeviceId) {
  // Refresh the microphone dropdown using the browser's current device list.
  if (
    !inputDeviceSelect ||
    !navigator.mediaDevices ||
    typeof navigator.mediaDevices.enumerateDevices !== "function"
  ) {
    return;
  }

  try {
    const devices = await navigator.mediaDevices.enumerateDevices();
    const audioInputs = devices.filter((device) => device.kind === "audioinput");

    console.log(
      "Audio input devices:",
      audioInputs.map((device, index) => ({
        index,
        deviceId: device.deviceId,
        label: device.label || "(label unavailable until permission granted)",
      }))
    );

    inputDeviceSelect.innerHTML = "";

    const defaultOption = document.createElement("option");
    defaultOption.value = "";
    defaultOption.textContent = "Default microphone";
    inputDeviceSelect.appendChild(defaultOption);

    for (const [index, device] of audioInputs.entries()) {
      const option = document.createElement("option");
      option.value = device.deviceId;
      option.textContent = device.label || `Microphone ${index + 1}`;
      inputDeviceSelect.appendChild(option);
    }

    const hasPreferredDevice = audioInputs.some(
      (device) => device.deviceId === preferredDeviceId
    );

    inputDeviceSelect.value =
      hasPreferredDevice && preferredDeviceId ? preferredDeviceId : "";
    store.selectedInputDeviceId = inputDeviceSelect.value || null;
  } catch (error) {
    console.warn("Failed to enumerate audio input devices.", error);
  }
}
```

*Figure 114 - D3.3.3 Enumerates available audio input devices and populates the microphone selection interface, allowing users to choose a recording source.*

```
function stopRecordingTimer() {
  // Stop the countdown interval used while recording.
  if (recordingTimerId) {
    clearInterval(recordingTimerId);
    recordingTimerId = null;
  }
}

function stopAutoStopTimer() {
  // Cancel the auto-stop timeout if recording ends early.
  if (recordingAutoStopId) {
    clearTimeout(recordingAutoStopId);
    recordingAutoStopId = null;
  }
}

function setCountdownDefault() {
  // Restore the helper text shown before recording starts.
  if (!recordingCountdown) {
    return;
  }

  recordingCountdown.textContent =
    `Auto-stop after ${RECORDING_COUNTDOWN_SECONDS} seconds (${REQUIRED_CLIP_SECONDS}s minimum clip)`;
  recordingCountdown.classList.remove("text-danger", "text-success");
  recordingCountdown.classList.add("text-body-secondary");
}

function renderRecordingTimer(elapsedSeconds) {
  // Update the live countdown while a recording is in progress.
  if (!recordingCountdown) {
    return;
  }

  const remainingSeconds = Math.max(
    0,
    RECORDING_COUNTDOWN_SECONDS - elapsedSeconds
  );

  if (remainingSeconds > 0) {
    recordingCountdown.textContent =
      `Recording: ${remainingSeconds.toFixed(1)}s remaining`;
    recordingCountdown.classList.remove("text-danger", "text-success");
    recordingCountdown.classList.add("text-body-secondary");
    return;
  }

  recordingCountdown.textContent =
    "Recording complete. Stopping automatically...";
  recordingCountdown.classList.remove("text-danger", "text-body-secondary");
  recordingCountdown.classList.add("text-success");
}

function startRecordingTimer() {
  // Start the UI timer that mirrors the recording window.
  stopRecordingTimer();
  renderRecordingTimer(0);
  recordingTimerId = setInterval(() => {
    if (!recordingStartTime) {
      return;
    }
    const elapsedSeconds = (Date.now() - recordingStartTime) / 1000;
    renderRecordingTimer(elapsedSeconds);
  }, 100);
}
```

*Figure 115 - D3.3.4 Implements countdown and timer logic to enforce recording duration constraints and provide real-time feedback to the user.*

```
function writeAscii(view, offset, text) {
  // Helper used when manually writing the WAV header.
  for (let index = 0; index < text.length; index += 1) {
    view.setUint8(offset + index, text.charCodeAt(index));
  }
}

function encodeMonoWav(samples, sampleRate) {
  // Export captured PCM samples to a browser-playable WAV blob.
  const bytesPerSample = 2;
  const blockAlign = bytesPerSample;
  const buffer = new ArrayBuffer(44 + samples.length * bytesPerSample);
  const view = new DataView(buffer);

  writeAscii(view, 0, "RIFF");
  view.setUint32(4, 36 + samples.length * bytesPerSample, true);
  writeAscii(view, 8, "WAVE");
  writeAscii(view, 12, "fmt ");
  view.setUint32(16, 16, true);
  view.setUint16(20, 1, true);
  view.setUint16(22, 1, true);
  view.setUint32(24, sampleRate, true);
  view.setUint32(28, sampleRate * blockAlign, true);
  view.setUint16(32, blockAlign, true);
  view.setUint16(34, 16, true);
  writeAscii(view, 36, "data");
  view.setUint32(40, samples.length * bytesPerSample, true);

  let offset = 44;
  for (let index = 0; index < samples.length; index += 1) {
    const clampedSample = Math.max(-1, Math.min(1, samples[index]));
    const intSample =
      clampedSample < 0
        ? clampedSample * 0x8000
        : clampedSample * 0x7fff;
    view.setInt16(offset, intSample, true);
    offset += bytesPerSample;
  }

  return new Blob([buffer], { type: "audio/wav" });
}

function downmixToMono(audioBuffer) {
  // Collapse browser audio buffers to mono before export and logging.
  const monoSamples = new Float32Array(audioBuffer.length);

  for (let channel = 0; channel < audioBuffer.numberOfChannels; channel += 1) {
    const channelData = audioBuffer.getChannelData(channel);
    for (let index = 0; index < channelData.length; index += 1) {
      monoSamples[index] += channelData[index];
    }
  }

  if (audioBuffer.numberOfChannels > 1) {
    for (let index = 0; index < monoSamples.length; index += 1) {
      monoSamples[index] /= audioBuffer.numberOfChannels;
    }
  }

  return monoSamples;
}
```

*Figure 116 - D3.3.5 Implements low-level functions for converting raw audio samples into a mono WAV format suitable for backend processing.*

```
function mergeFloat32Chunks(chunks) {
  // Combine recorded audio blocks into one contiguous sample array.
  const totalLength = chunks.reduce((sum, chunk) => sum + chunk.length, 0);
  const merged = new Float32Array(totalLength);
  let offset = 0;

  for (const chunk of chunks) {
    merged.set(chunk, offset);
    offset += chunk.length;
  }

  return merged;
}

function logTrackDiagnostics(track) {
  // Log browser microphone metadata to help with capture debugging.
  console.log("Mic track label:", track?.label ?? "(unknown)");
  console.log("Mic track settings:", track?.getSettings?.() ?? {});
  console.log("Mic track constraints:", track?.getConstraints?.() ?? {});
  console.log("Mic track state:", {
    enabled: track?.enabled,
    muted: track?.muted,
    readyState: track?.readyState,
  });
}

async function requestMicrophoneStream() {
  // Try the selected microphone first, then fall back to simpler constraints.
  let lastError = null;

  const constraintCandidates = [];
  if (store.selectedInputDeviceId) {
    constraintCandidates.push({
      deviceId: { exact: store.selectedInputDeviceId },
    });
  }
  constraintCandidates.push(...MICROPHONE_CONSTRAINT_CANDIDATES);

  for (const audioConstraints of constraintCandidates) {
    try {
      console.log("Requesting microphone with constraints:", audioConstraints);
      return await navigator.mediaDevices.getUserMedia({
        audio: audioConstraints,
      });
    } catch (error) {
      console.warn(
        "Microphone request failed for constraints:",
        audioConstraints,
        error
      );
      lastError = error;
    }
  }

  throw lastError ?? new Error("Unable to acquire microphone stream.");
}
```

*Figure 117 - D3.3.6 Handles merging of recorded audio chunks and acquisition of microphone input streams using browser media APIs.*

```
async function cleanupCaptureGraph() {
  // Tear down any existing Web Audio graph before starting a new recording.
  if (recordingProcessorNode) {
    recordingProcessorNode.onaudioprocess = null;
    recordingProcessorNode.disconnect();
    recordingProcessorNode = null;
  }

  if (recordingSourceNode) {
    recordingSourceNode.disconnect();
    recordingSourceNode = null;
  }

  if (recordingMonitorNode) {
    recordingMonitorNode.disconnect();
    recordingMonitorNode = null;
  }

  if (recordingAudioContext) {
    await recordingAudioContext.close().catch(() => {});
    recordingAudioContext = null;
  }
}
```

*Figure 118 - D3.3.7 Ensures proper teardown of the Web Audio API processing graph to prevent resource leaks between recordings.*

```
async function startPcmCapture(stream) {
  // Capture raw PCM samples so recorded clips are not tied to MediaRecorder codecs.
  const AudioContextClass = window.AudioContext || window.webkitAudioContext;
  if (!AudioContextClass) {
    throw new Error("Audio recording is not supported in this browser.");
  }

  await cleanupCaptureGraph();

  recordingAudioContext = new AudioContextClass();
  await recordingAudioContext.resume();

  recordedSampleRate = recordingAudioContext.sampleRate;
  recordedPcmChunks = [];
  recordingPeak = 0;
  recordingRmsSumSquares = 0;
  recordingRmsSamples = 0;
  recordingLastLevelLogAt = 0;

  recordingSourceNode =
    recordingAudioContext.createMediaStreamSource(stream);
  recordingProcessorNode =
    recordingAudioContext.createScriptProcessor(4096, 1, 1);
  recordingMonitorNode = recordingAudioContext.createGain();
  recordingMonitorNode.gain.value = 0;

  recordingProcessorNode.onaudioprocess = (event) => {
    // Buffer the incoming samples and track simple level diagnostics.
    const monoChunk = downmixToMono(event.inputBuffer);
    recordedPcmChunks.push(monoChunk);

    let chunkPeak = 0;
    for (let index = 0; index < monoChunk.length; index += 1) {
      const sample = monoChunk[index];
      const absSample = Math.abs(sample);
      chunkPeak = Math.max(chunkPeak, absSample);
      recordingRmsSumSquares += sample * sample;
    }

    recordingPeak = Math.max(recordingPeak, chunkPeak);
    recordingRmsSamples += monoChunk.length;

    const now = Date.now();
    if (now - recordingLastLevelLogAt > 1000) {
      recordingLastLevelLogAt = now;
      console.log("Mic level snapshot:", {
        peak: Number(chunkPeak.toFixed(5)),
        trackMuted: recordingTrack?.muted ?? null,
        samplesCaptured: recordingRmsSamples,
      });
    }
  };

  recordingSourceNode.connect(recordingProcessorNode);
  recordingProcessorNode.connect(recordingMonitorNode);
  recordingMonitorNode.connect(recordingAudioContext.destination);
}
```

*Figure 119 - D3.3.8 Initialises raw PCM audio capture using the Web Audio API, buffering microphone input for later processing.*

```
async function stopPcmCapture() {
  // Finish PCM capture and return the final WAV blob.
  const mergedSamples = mergeFloat32Chunks(recordedPcmChunks);
  const sampleRate = recordedSampleRate;

  await cleanupCaptureGraph();
  recordedPcmChunks = [];

  if (mergedSamples.length === 0) {
    throw new Error("No microphone audio samples were captured.");
  }

  return encodeMonoWav(mergedSamples, sampleRate);
}

function stopMediaStream() {
  // Release the active browser microphone stream once recording ends.
  if (mediaStream) {
    mediaStream.getTracks().forEach((track) => track.stop());
    mediaStream = null;
  }
  recordingTrack = null;
}
```

*Figure 120 - D3.3.9 Stops audio capture, processes buffered samples, and releases the active microphone stream.*

```
async function finalizeRecording() {
  // Complete the recording flow, validate the clip, and prepare playback/upload.
  if (!recordingStartTime || isStoppingRecording) {
    return;
  }

  isStoppingRecording = true;
  recordBtn.disabled = false;
  stopBtn.disabled = true;
  stopRecordingTimer();
  stopAutoStopTimer();

  const durationSeconds = (Date.now() - recordingStartTime) / 1000;
  let audioBlob = null;

  try {
    setStatus("Preparing recorded audio...");
    audioBlob = await stopPcmCapture();
  } catch (error) {
    console.error(error);
  }

  const overallRms =
    recordingRmsSamples > 0
      ? Math.sqrt(recordingRmsSumSquares / recordingRmsSamples)
      : 0;

  console.log("Mic recording summary:", {
    durationSeconds: Number(durationSeconds.toFixed(2)),
    peak: Number(recordingPeak.toFixed(5)),
    rms: Number(overallRms.toFixed(5)),
    samplesCaptured: recordingRmsSamples,
    trackMutedAtEnd: recordingTrack?.muted ?? null,
    trackReadyStateAtEnd: recordingTrack?.readyState ?? null,
  });

  stopMediaStream();

  // Reject recordings that do not meet the minimum duration.
  if (durationSeconds + 0.05 < REQUIRED_CLIP_SECONDS) {
    if (recordingCountdown) {
      recordingCountdown.textContent =
        `Too short: ${durationSeconds.toFixed(1)}s recorded. At least ${REQUIRED_CLIP_SECONDS}s is required.`;
      recordingCountdown.classList.remove(
        "text-success",
        "text-body-secondary"
      );
      recordingCountdown.classList.add("text-danger");
    }
    store.latestAudioBlob = null;
    setPlaybackSource(null);
    refreshPredictButtonState();
    setStatus(
      `Recording too short (${durationSeconds.toFixed(1)}s). Please record at least ${REQUIRED_CLIP_SECONDS} seconds.`
    );
    recordingStartTime = null;
    isStoppingRecording = false;
    return;
  }
```

*Figure 121 - D3.3.10 Begins the recording finalisation process, including duration calculation and validation against minimum recording length requirements.*

```
  // Stop here if capture failed before a playable blob could be produced.
  if (!audioBlob) {
    if (recordingCountdown) {
      recordingCountdown.textContent =
        "Recording failed. Please try again.";
      recordingCountdown.classList.remove(
        "text-success",
        "text-body-secondary"
      );
      recordingCountdown.classList.add("text-danger");
    }
    store.latestAudioBlob = null;
    setPlaybackSource(null);
    refreshPredictButtonState();
    setStatus("Recording failed. No usable microphone audio was captured.");
    recordingStartTime = null;
    isStoppingRecording = false;
    return;
  }

  console.log("Recorded blob type:", audioBlob.type);
  console.log("Recorded blob size:", audioBlob.size);

  if (recordingPeak < 0.001 && overallRms < 0.0005) {
    console.warn(
      "Captured audio appears effectively silent despite successful recording."
    );
  }

  if (recordingCountdown) {
    recordingCountdown.textContent =
      `Recorded ${durationSeconds.toFixed(1)}s clip.`;
    recordingCountdown.classList.remove(
      "text-danger",
      "text-body-secondary"
    );
    recordingCountdown.classList.add("text-success");
  }

  // Make the new clip available for preview and prediction.
  store.latestAudioBlob = audioBlob;
  store.uploadedAudioFile = null;
  setPlaybackSource(audioBlob);
  refreshPredictButtonState();

  setStatus(
    `Recording ready (${durationSeconds.toFixed(1)}s). The first ${REQUIRED_CLIP_SECONDS} seconds will be used.`
  );

  recordingStartTime = null;
  isStoppingRecording = false;
}
```

*Figure 122 - D3.3.11 Completes recording finalisation by preparing audio for playback, updating application state, and enabling prediction.*

```
function startAutoStopTimer() {
  // End the recording automatically once the countdown window expires.
  stopAutoStopTimer();
  recordingAutoStopId = setTimeout(async () => {
    if (!recordingStartTime || isStoppingRecording) {
      return;
    }
    await finalizeRecording();
  }, RECORDING_COUNTDOWN_SECONDS * 1000);
}

setCountdownDefault();
refreshPredictButtonState();
renderPredictionHistory();
void loadInputDevices();

if (inputDeviceSelect) {
  inputDeviceSelect.addEventListener("change", () => {
    // Persist the chosen microphone for the rest of the current session.
    store.selectedInputDeviceId = inputDeviceSelect.value || null;
    setStatus(
      store.selectedInputDeviceId
        ? "Microphone selected."
        : "Using default microphone."
    );
  });
}
```

*Figure 123 - D3.3.12 Implements automatic stopping of recording after a fixed duration to ensure consistent input length.*

```
recordBtn.addEventListener("click", async () => {
  try {
    // Start a fresh recording from the chosen input device.
    mediaStream = await requestMicrophoneStream();
    recordingTrack = mediaStream.getAudioTracks()[0] ?? null;

    if (!recordingTrack) {
      throw new Error("No audio track was returned from getUserMedia.");
    }

    logTrackDiagnostics(recordingTrack);
    await loadInputDevices(recordingTrack.getSettings?.().deviceId ?? store.selectedInputDeviceId);
    recordingTrack.onmute = () => console.warn("Mic track muted");
    recordingTrack.onunmute = () => console.log("Mic track unmuted");
    recordingTrack.onended = () => console.warn("Mic track ended");

    await startPcmCapture(mediaStream);

    recordingStartTime = Date.now();
    isStoppingRecording = false;
    store.latestAudioBlob = null;
    store.uploadedAudioFile = null;
    setPlaybackSource(null);

    startRecordingTimer();
    startAutoStopTimer();
    recordBtn.disabled = true;
    stopBtn.disabled = false;
    refreshPredictButtonState();
    setStatus(
      `Recording... stop any time after ${REQUIRED_CLIP_SECONDS} seconds, or it will auto-stop after ${RECORDING_COUNTDOWN_SECONDS} seconds.`
    );
  } catch (error) {
    console.error(error);
    stopRecordingTimer();
    stopAutoStopTimer();
    await cleanupCaptureGraph().catch(() => {});
    stopMediaStream();
    recordingStartTime = null;
    isStoppingRecording = false;
    setCountdownDefault();
    setStatus("Could not access microphone.");
  }
});

stopBtn.addEventListener("click", async () => {
  // Manual stop follows the same completion path as auto-stop.
  await finalizeRecording();
});
```

*Figure 124 - D3.3.13 Defines event listeners for starting and stopping microphone recording through user interface controls.*

```
if (audioFileInput) {
  const onFileSelected = () => {
    // Switch the active audio source from recorded audio to the chosen file.
    const file = audioFileInput.files?.[0] ?? null;

    if (!file) {
      refreshPredictButtonState();
      return;
    }

    store.uploadedAudioFile = file;
    store.latestAudioBlob = null;

    setPlaybackSource(file);
    stopRecordingTimer();
    setCountdownDefault();
    refreshPredictButtonState();

    setStatus(`Audio file selected: ${file.name}`);
  };

  audioFileInput.addEventListener("change", onFileSelected);
  audioFileInput.addEventListener("input", onFileSelected);
}

predictBtn.addEventListener("click", async () => {
  // Submit whichever clip is currently active in the controls panel.
  if (!store.selectedModelId) {
    setStatus("No model selected.");
    return;
  }

  let audioSource = null;

  if (store.uploadedAudioFile) {
    audioSource = store.uploadedAudioFile;
  } else if (store.latestAudioBlob) {
    audioSource = store.latestAudioBlob;
  }

  if (!audioSource) {
    setStatus("No recording or uploaded audio available.");
    return;
  }

  try {
    clearMapHighlight();
    setStatus("Running prediction...");

    const result = await predictAudio(audioSource, store.selectedModelId);

    // Save and render the result before syncing the map highlight.
    savePrediction(result);
    renderPrediction(result);
    renderPredictionHistory();

    const predictedLabel = result.label ?? result.predicted_label ?? null;
    const predictionModelId = result.model_id ?? store.selectedModelId ?? null;

    if (predictionModelId && predictedLabel) {
      updateMap(predictionModelId, predictedLabel);
    }

    setStatus("Prediction complete.");
  } catch (error) {
    console.error(error);
    renderError(error.message);
    setStatus(`Prediction failed: ${error.message}`);
  }
});
```

*Figure 125 - D3.3.14 Handles audio file selection and submits recorded or uploaded audio to the backend prediction endpoint.*

**D3.4 - map.js**

```
let map;
let provinceLayers = {};
let geojsonLayer;
let mapReady = false;

const defaultStyle = {
  color: "#6c757d",
  weight: 1,
  fillColor: "#dee2e6",
  fillOpacity: 0.5
};

const highlightStyle = {
  color: "#0d6efd",
  weight: 2,
  fillColor: "#0d6efd",
  fillOpacity: 0.6
};

const INITIAL_CENTER = [53.4, -8.2];
const INITIAL_ZOOM = 7;

function normalizeName(value) {
  // Keep label matching stable across map data and model outputs.
  return String(value || "").trim().toLowerCase();
}

function getRegionsToHighlight(modelId, label) {
  // Translate model outputs into one or more province polygons on the map.
  const cleanLabel = String(label || "").trim();

  if (modelId === "wav2vec_ulster_vs_rest_rf") {
    if (cleanLabel === "Ulster") return ["Ulster"];
    if (cleanLabel === "Rest") return ["Leinster", "Munster", "Connacht"];
  }

  if (modelId === "wav2vec_leinster_vs_rest_logreg") {
    if (cleanLabel === "Leinster") return ["Leinster"];
    if (cleanLabel === "Rest") return ["Ulster", "Munster", "Connacht"];
  }

  if (modelId === "wav2vec_ulster_leinster_rest_logreg") {
    if (cleanLabel === "Ulster") return ["Ulster"];
    if (cleanLabel === "Leinster") return ["Leinster"];
    if (cleanLabel === "Rest") return ["Munster", "Connacht"];
  }

  if (
    modelId === "wav2vec_province_4way_logreg" ||
    modelId === "mfcc_logreg_v1_01"
  ) {
    if (["Ulster", "Leinster", "Munster", "Connacht"].includes(cleanLabel)) {
      return [cleanLabel];
    }
  }

  return [];
}
```

*Figure 126 - D3.4.1 Defines map styling, normalisation of region labels, and logic for translating model predictions into one or more geographic regions for visualisation.*

```
export async function initMap() {
  // Create the base map and index each province layer by name for later lookup.
  map = L.map("map").setView(INITIAL_CENTER, INITIAL_ZOOM);

  L.tileLayer(
    "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png",
    {
      attribution: "&copy; OpenStreetMap contributors"
    }
  ).addTo(map);

  const response = await fetch("/static/data/provinces.geojson");
  const geojson = await response.json();

  geojsonLayer = L.geoJSON(geojson, {
    style: defaultStyle,
    onEachFeature: (feature, layer) => {
      const name = feature.properties.NAME;
      provinceLayers[normalizeName(name)] = layer;
      layer.bindTooltip(name);
    }
  }).addTo(map);

  mapReady = true;
}

export function updateMap(modelId, label) {
  // Called after prediction and history restore to apply the correct highlight.
  clearMapHighlight();

  const regions = getRegionsToHighlight(modelId, label);

  regions.forEach((regionName) => {
    const layer = provinceLayers[normalizeName(regionName)];
    if (layer) {
      layer.setStyle(highlightStyle);
    }
  });
}

export function resetMapView() {
  // Return the map to the default Ireland-wide view without changing highlights.
  if (!mapReady || !map) {
    return;
  }

  map.setView(INITIAL_CENTER, INITIAL_ZOOM);
}

export function clearMapHighlight() {
  // Remove any active region highlight before applying a new one.
  Object.values(provinceLayers).forEach((layer) => {
    layer.setStyle(defaultStyle);
  });
}
```

*Figure 127 - D3.4.2 Initialises the Leaflet map, loads province GeoJSON data, and updates the map by highlighting predicted regions and managing map state during user interaction.*

**D3.5 - store.js**

```
const STORAGE_KEY = "fyp_predictions";

function loadStoredPredictions() {
  // Restore saved prediction history when the page boots.
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return [];
    const parsed = JSON.parse(raw);
    return Array.isArray(parsed) ? parsed : [];
  } catch (e) {
    console.error("Failed to load stored predictions", e);
    return [];
  }
}

function saveStoredPredictions(predictions) {
  // Persist the history list after prediction or feedback changes.
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(predictions));
  } catch (e) {
    console.error("Failed to save predictions", e);
  }
}

export const store = {
  models: [],
  selectedModelId: null,
  selectedInputDeviceId: null,
  latestAudioBlob: null,
  uploadedAudioFile: null,

  predictions: loadStoredPredictions(),
  currentPredictionId: null,

  persist() {
    // Save the current prediction history snapshot.
    saveStoredPredictions(this.predictions);
  },

  clearPredictions() {
    // Remove all saved history entries and reset the current selection.
    this.predictions = [];
    this.currentPredictionId = null;
    saveStoredPredictions(this.predictions);
  }
};
```

*Figure 128 - D3.5 Implements a shared state container for the frontend, tracking selected models, audio inputs, and prediction history. It also provides persistence through localStorage, allowing prediction results and user feedback to be retained across sessions.*

**D3.6 index.html**

```
<!-- 1,2 Current Prediction -->
<div class="col-12 col-md-6 d-flex">
  <div class="card shadow-sm border-0 w-100 h-100">
    <div class="card-body">
      <h2 class="h5 card-title mb-3">Current Prediction</h2>

      <p class="mb-1"><strong>Model:</strong> <span id="resultModel">-</span></p>
      <p id="resultModelDescription" class="small text-body-secondary mb-2 d-none"></p>
      <p class="mb-2"><strong>Prediction:</strong> <span id="resultLabel">-</span></p>
      <p class="mb-3"><strong>Confidence:</strong> <span id="resultConfidence">-</span></p>

      <h3 class="h6 text-uppercase text-body-secondary mb-2">Class Probabilities</h3>
      <ul id="probList" class="list-group list-group-flush"></ul>
    </div>
  </div>
</div>
```

*Figure 129 - D3.6.1 Current Prediction Div: defines the frontend panel used to display the selected model, predicted label, confidence score, and class probability breakdown returned by the backend.*

```
<!-- 2,2 Feedback -->
<div class="col-12 col-md-6 d-flex">
  <div class="card shadow-sm border-0 w-100 h-100">
    <div class="card-body">
      <h2 class="h5 card-title mb-3">Feedback</h2>

      <div id="feedbackSection" class="d-none">
        <p class="mb-2">Was this prediction correct?</p>

        <div class="d-flex gap-2 mb-3">
          <button
            id="feedbackYesBtn"
            type="button"
            class="btn btn-outline-success btn-sm w-100"
          >
            Yes
          </button>
          <button
            id="feedbackNoBtn"
            type="button"
            class="btn btn-outline-danger btn-sm w-100"
          >
            No
          </button>
        </div>

        <div id="correctLabelSection" class="d-none">
          <label for="correctLabelSelect" class="form-label">
            Select the correct province
          </label>
          <select id="correctLabelSelect" class="form-select form-select-sm">
            <option value="">Choose province</option>
            <option value="Leinster">Leinster</option>
            <option value="Munster">Munster</option>
            <option value="Connacht">Connacht</option>
            <option value="Ulster">Ulster</option>
          </select>

          <button
            id="saveCorrectionBtn"
            type="button"
            class="btn btn-sm btn-primary mt-2"
          >
            Save correction
          </button>
        </div>

        <div id="feedbackMessage" class="form-text mt-2"></div>
      </div>

      <div id="feedbackPlaceholder" class="text-body-secondary small">
        Run a prediction to give feedback.
      </div>
    </div>
```

*Figure 130 - D3.6.2 Feedback Div : defines the user feedback interface, allowing predictions to be marked as correct or incorrect and enabling corrected province labels to be submitted when required.*

```
<!-- Right area: Map -->
<section class="col-12 col-lg-4 d-flex">
  <div class="card shadow-sm border-0 w-100 h-100">
    <div class="card-body d-flex flex-column">
      <div class="d-flex justify-content-between align-items-center mb-3">
        <h2 class="h5 card-title mb-0">Map</h2>
        <button
          id="resetMapBtn"
          class="btn btn-sm btn-outline-secondary"
          type="button"
        >
          Reset View
        </button>
      </div>

      <div id="map" class="map-box flex-grow-1"></div>
```

*Figure 131 - D3.6.3 Map Div: Defines the dedicated map container and associated controls used for geospatial visualisation of prediction results within the frontend layout.*

```
<!-- 2,1 Prediction History -->
<div class="col-12 col-md-6 d-flex">
  <div class="card shadow-sm border-0 w-100 h-100">
    <div class="card-body">
      <div class="d-flex justify-content-between align-items-center mb-3">
        <h2 class="h5 card-title mb-0">Prediction History</h2>
        <button
          id="clearHistoryBtn"
          type="button"
          class="btn btn-sm btn-outline-secondary"
          disabled
        >
          Clear Results
        </button>
      </div>
      <div id="predictionHistoryPlaceholder" class="text-body-secondary small">
        No previous predictions yet.
      </div>
      <div id="predictionHistoryList" class="d-none"></div>
    </div>
  </div>
</div>
```

*Figure 132 - D3.6.4 Prediction History Div: Defines the frontend section used to display stored prediction results, allowing users to revisit and interact with previous classifications.*